Human review packet: KR21Monoculture

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: KR21Monoculture
Paper id: KR21Monoculture
Source version: arXiv:2101.05853
Source record: audit/paper_statement_map.json
Rows: 49
Generated: 2026-09-07
Source URL: official source

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper KR21Monoculture --serve
```

Open http://127.0.0.1:8765/ in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- equation5_from_literal_definition1_removalSpec
- equation6_outer_payoff_indifference_from_source_conditionsSpec
- theorem2_gaussian_laplace_source_semantic_completeSpec
- equation7_plackettLuce_choice_probabilitySpec
- section31_literalUnitVarianceGumbel_outer_zero_effect_semantic_completeSpec
- equation8_source_concrete_mallows_probabilitySpec
- theorem3_source_complete_semantic_completeSpec
- theorem4_source_mallows_outer_horizon_lt_literal_semantic_completeSpec
- appendixA_theorem5_corrected_source_completeSpec
- equationA1_source_w11_iid_literal_event_conditionalTail_integralSpec
- appendixB1_source_discrete_definition2_counterexample_semantic_completeSpec
- appendixB2_source_discrete_definition3_counterexample_semantic_completeSpec
- equationB1_counterexample_first_choice_x1_semantic_completeSpec
- equationB2_counterexample_first_choice_x2_semantic_completeSpec
- appendixC1_source_pairwise_full_eventsSpec
- appendixC2_source_pairwise_full_eventsSpec
- theorem6_source_raw_iid_density_literal_semantic_completeSpec
- lemma1_corrected_gaussian_laplace_kernel_targetSpec
- lemma2_source_arbitraryFinite_iid_bottom_first_probability_semantic_completeSpec
- lemma3_source_arbitraryFinite_iid_delta_le_top_delta_semantic_completeSpec
- theorem7_source_semantic_completeSpec
- equationC3_laplace_strict_conditional_ratio_eq_density_cdf_semantic_completeSpec
- equationC4_laplace_case3_expression_posSpec
- theorem8_source_semantic_completeSpec
- equationC5_gaussian_source_semantic_completeSpec
- equationC6_gaussian_reduced_expression_posSpec
- equationC7_gaussian_reduced_expression_tendsto_atBot_zeroSpec
- equationC8_gaussian_reduced_expression_hasDerivAtSpec
- equationC8_C9_corrected_gaussian_targetSpec
- equationC10_gaussian_g_inv_derivative_lower_boundSpec
- theorem9_source_mallows_definition1_literal_semantic_completeSpec
- equationD1_source_phi_likelihood_ratio_completeSpec
- equationE1_source_phi_literal_conditional_semantic_completeSpec
- equationE2_source_phi_literal_conditional_semantic_completeSpec
- equationE3_source_phi_completeSpec
- lemma4_weighted_average_lt_of_cross_ratioSpec
- lemma5_source_phi_completeSpec
- lemma6_source_phi_rank_power_completeSpec
- equationF2_source_phi_closed_form_completeSpec
- lemma7_source_phi_completeSpec
- lemma8_source_mallows_phi_pairwise_correct_probability_ltSpec
- section31_corrected_gumbel_plackett_luce_targetSpec
- section31_plackettLuce_outer_best_available_weakly_dominatesSpec
- appendixB_smoothing_source_coreSpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Probability.gaussianVarianceFromStd

Declaration location: reusable library

Lean declaration body

type:
```
ℝ → NNReal
```

value:
```
fun (σ : ℝ) => NNReal.mk (σ ^ 2) (AppliedModelingLib.Probability.gaussianVarianceFromStd._proof_1 σ)
```

AppliedModelingLib.Probability.independentGaussianPairMeasureWithStd

Declaration location: reusable library

Lean declaration body

type:
```
ℝ → ℝ → ℝ → MeasureTheory.Measure (ℝ × ℝ)
```

value:
```
fun (σ xi xj : ℝ) =>
  (ProbabilityTheory.gaussianReal xi (AppliedModelingLib.Probability.gaussianVarianceFromStd σ)).prod
    (ProbabilityTheory.gaussianReal xj (AppliedModelingLib.Probability.gaussianVarianceFromStd σ))
```

Direct retained dependencies

- AppliedModelingLib.Probability.gaussianVarianceFromStd

AppliedModelingLib.SocialChoice.Ranking.RankingPair

Declaration location: reusable library

Lean declaration body

type:
```
ℕ → Type
```

value:
```
fun (n : ℕ) => AppliedModelingLib.SocialChoice.Ranking.Ranking n × AppliedModelingLib.SocialChoice.Ranking.Ranking n
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.firstChoice

Declaration location: reusable library

Lean declaration body

type:
```
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Candidate n
```

value:
```
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) => π 0
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.disagreementEvent

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.RankingPair n → Prop

value:
fun {n : ℕ} (a : AppliedModelingLib.SocialChoice.Ranking.RankingPair n) =>
  AppliedModelingLib.SocialChoice.Ranking.firstChoice a.1 ≠ AppliedModelingLib.SocialChoice.Ranking.firstChoice a.2

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.RankingPair
- AppliedModelingLib.SocialChoice.Ranking.firstChoice

AppliedModelingLib.SocialChoice.Ranking.decidableDisagreementEvent

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} →
  (a : AppliedModelingLib.SocialChoice.Ranking.RankingPair n) →
    Decidable (AppliedModelingLib.SocialChoice.Ranking.disagreementEvent a)

value:
fun {n : ℕ} (a : AppliedModelingLib.SocialChoice.Ranking.RankingPair n) => id inferInstance

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.RankingPair
- AppliedModelingLib.SocialChoice.Ranking.disagreementEvent
- AppliedModelingLib.SocialChoice.Ranking.firstChoice

AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking

Declaration location: reusable library

Lean declaration body

type:
(n : ℕ) → MeasurableSpace (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun (n : ℕ) => ⊤

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rankByScore

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Ranking n

value:
fun {n : ℕ} (score : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  Tuple.sort fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => -score i

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rankOf

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n

value:
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  (Equiv.symm π) c
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.WeaklyOrderedBy

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop

value:
fun {n : ℕ} (ρ : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  ∀ {a b : AppliedModelingLib.SocialChoice.Ranking.Candidate n},
    AppliedModelingLib.SocialChoice.Ranking.rankOf ρ a < AppliedModelingLib.SocialChoice.Ranking.rankOf ρ b →
      value b ≤ value a
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankOf

AppliedModelingLib.SocialChoice.Ranking.invertedPair

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n →
      AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n → Prop

value:
fun {n : ℕ} (ρ π : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (ab : AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.SocialChoice.Ranking.rankOf ρ ab.1 < AppliedModelingLib.SocialChoice.Ranking.rankOf ρ ab.2 ∧
    AppliedModelingLib.SocialChoice.Ranking.rankOf π ab.2 < AppliedModelingLib.SocialChoice.Ranking.rankOf π ab.1
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankOf

AppliedModelingLib.SocialChoice.Ranking.inversionFinset

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n →
      Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n)

value:
fun {n : ℕ} (ρ π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  {ab : AppliedModelingLib.SocialChoice.Ranking.Candidate n × AppliedModelingLib.SocialChoice.Ranking.Candidate n |
    AppliedModelingLib.SocialChoice.Ranking.invertedPair ρ π ab}
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.invertedPair

AppliedModelingLib.SocialChoice.Ranking.kendallTau

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℕ

value:
fun {n : ℕ} (ρ π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  (AppliedModelingLib.SocialChoice.Ranking.inversionFinset ρ π).card
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.inversionFinset

AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  {Ω : Type u_1} →
    [inst : MeasurableSpace Ω] →
      (μ : MeasureTheory.Measure Ω) →
        [MeasureTheory.IsProbabilityMeasure μ] →
          (rank : Ω → AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
            Measurable rank → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {n : ℕ} {Ω : Type u_1} [MeasurableSpace Ω] (μ : MeasureTheory.Measure Ω) [MeasureTheory.IsProbabilityMeasure μ]
    (rank : Ω → AppliedModelingLib.SocialChoice.Ranking.Ranking n) (hrank : Measurable rank) =>
  (MeasureTheory.Measure.map rank μ).toPMF
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking

AppliedModelingLib.SocialChoice.Ranking.rum3Ranking012

Declaration location: reusable library

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
Equiv.refl (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rum3Ranking021

Declaration location: reusable library

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
Equiv.swap 1 2

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rum3Ranking102

Declaration location: reusable library

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
Equiv.swap 0 1

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rum3Ranking120

Declaration location: reusable library

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
Equiv.trans (Equiv.swap 1 2) (Equiv.swap 0 1)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rum3Ranking201

Declaration location: reusable library

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
Equiv.trans (Equiv.swap 0 2) (Equiv.swap 0 1)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rum3Ranking210

Declaration location: reusable library

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
Equiv.swap 0 2

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.rum3RankByScores

Declaration location: reusable library

Lean declaration body

type:
ℝ → ℝ → ℝ → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
```
fun (s1 s2 s3 : ℝ) =>
  if h0 : s2 ≤ s1 ∧ s3 ≤ s1 then
    if s3 ≤ s2 then AppliedModelingLib.SocialChoice.Ranking.rum3Ranking012
    else AppliedModelingLib.SocialChoice.Ranking.rum3Ranking021
  else
    if h1 : s1 < s2 ∧ s3 ≤ s2 then
      if s3 ≤ s1 then AppliedModelingLib.SocialChoice.Ranking.rum3Ranking102
      else AppliedModelingLib.SocialChoice.Ranking.rum3Ranking120
    else
      if s2 ≤ s1 then AppliedModelingLib.SocialChoice.Ranking.rum3Ranking201
      else AppliedModelingLib.SocialChoice.Ranking.rum3Ranking210
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking012
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking021
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking102
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking120
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking201
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking210

AppliedModelingLib.SocialChoice.Ranking.secondChoice

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Candidate n

value:
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) => π 1

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

AppliedModelingLib.SocialChoice.Ranking.bestRemainingAfter

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n

value:
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  if AppliedModelingLib.SocialChoice.Ranking.firstChoice π = c then
    AppliedModelingLib.SocialChoice.Ranking.secondChoice π
  else AppliedModelingLib.SocialChoice.Ranking.firstChoice π
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.secondChoice

AppliedModelingLib.SocialChoice.Ranking.rerankingGainOnPair

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ

value:
fun {n : ℕ} (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (π σ : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  if AppliedModelingLib.SocialChoice.Ranking.firstChoice π = AppliedModelingLib.SocialChoice.Ranking.firstChoice σ then
    0
  else
    value (AppliedModelingLib.SocialChoice.Ranking.firstChoice π) -
      value (AppliedModelingLib.SocialChoice.Ranking.secondChoice π)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.secondChoice

AppliedModelingLib.SocialChoice.Ranking.pairRerankingGain

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → AppliedModelingLib.SocialChoice.Ranking.RankingPair n → ℝ

value:
fun {n : ℕ} (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (a : AppliedModelingLib.SocialChoice.Ranking.RankingPair n) =>
  AppliedModelingLib.SocialChoice.Ranking.rerankingGainOnPair value a.1 a.2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.RankingPair
- AppliedModelingLib.SocialChoice.Ranking.rerankingGainOnPair

AppliedModelingLib.finCons

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → {k : ℕ} → α → (Fin k → α) → Fin (k + 1) → α

value:
fun {α : Type u_1} {k : ℕ} (head : α) (tail : Fin k → α) (a : Fin (k + 1)) =>
  match a with
  | ⟨0, _h⟩ => head
  | ⟨n.succ, h⟩ => tail ⟨n, AppliedModelingLib.finCons._proof_1 n h⟩
```

AppliedModelingLib.finiteAvailableWeight

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → [DecidableEq α] → (α → ℝ) → Finset α → ℝ

value:
fun {α : Type u_1} [Fintype α] [DecidableEq α] (baseWeight : α → ℝ) (forbidden : Finset α) =>
  ∑ a : α, if a ∈ forbidden then 0 else baseWeight a
```

AppliedModelingLib.finiteFreshList

Declaration location: reusable library

Lean declaration body

```
type:
(α : Type u_1) → ℕ → Finset α → Type (max 0 u_1)

value:
fun (α : Type u_1) (k : ℕ) (forbidden : Finset α) =>
  { list : Fin k → α // Function.Injective list ∧ ∀ (slot : Fin k), list slot ∉ forbidden }
```

AppliedModelingLib.finiteFreshListCons

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
  [inst : DecidableEq α] →
    {k : ℕ} →
      {forbidden : Finset α} →
        (head : { a : α // a ∉ forbidden }) →
          AppliedModelingLib.finiteFreshList α k (insert (↑head) forbidden) →
            AppliedModelingLib.finiteFreshList α (k + 1) forbidden

value:
fun {α : Type u_1} [DecidableEq α] {k : ℕ} {forbidden : Finset α} (head : { a : α // a ∉ forbidden })
    (tail : AppliedModelingLib.finiteFreshList α k (insert (↑head) forbidden)) =>
  ⟨AppliedModelingLib.finCons ↑head ↑tail, AppliedModelingLib.finiteFreshListCons._proof_1 head tail⟩
```

Direct retained dependencies

- AppliedModelingLib.finCons
- AppliedModelingLib.finiteFreshList

AppliedModelingLib.finiteFreshListNil

Declaration location: reusable library

Lean declaration body

```
type:
(α : Type u_1) → (forbidden : Finset α) → AppliedModelingLib.finiteFreshList α 0 forbidden

value:
fun (α : Type u_1) (forbidden : Finset α) =>
  ⟨fun (i : Fin 0) => i.elim0, AppliedModelingLib.finiteFreshListNil._proof_1 α forbidden⟩
```

Direct retained dependencies

- AppliedModelingLib.finiteFreshList

AppliedModelingLib.finiteWeightedPMF

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [inst : Fintype α] → (weight : α → ℝ) → (∀ (a : α), 0 ≤ weight a) → 0 < ∑ a : α, weight a → PMF α

value:
fun {α : Type u_1} [Fintype α] (weight : α → ℝ) (hweight_nonneg : ∀ (a : α), 0 ≤ weight a)
    (htotal_pos : 0 < ∑ a : α, weight a) =>
  PMF.ofFintype (fun (a : α) => ENNReal.ofReal (weight a / ∑ b : α, weight b))
    (AppliedModelingLib.finiteWeightedPMF._proof_1 weight hweight_nonneg htotal_pos)
```

AppliedModelingLib.finiteWeightedPMFAvailable

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
  [inst : Fintype α] →
    [inst_1 : DecidableEq α] →
      (baseWeight : α → ℝ) →
        (forbidden : Finset α) →
          (∀ (a : α), 0 ≤ baseWeight a) →
            0 < AppliedModelingLib.finiteAvailableWeight baseWeight forbidden → PMF { a : α // a ∉ forbidden }

value:
fun {α : Type u_1} [Fintype α] [DecidableEq α] (baseWeight : α → ℝ) (forbidden : Finset α)
    (hbase_nonneg : ∀ (a : α), 0 ≤ baseWeight a)
    (havailable_pos : 0 < AppliedModelingLib.finiteAvailableWeight baseWeight forbidden) =>
  AppliedModelingLib.finiteWeightedPMF (fun (a : { a : α // a ∉ forbidden }) => baseWeight ↑a)
    (AppliedModelingLib.finiteWeightedPMFAvailable._proof_1 baseWeight forbidden hbase_nonneg)
    (AppliedModelingLib.finiteWeightedPMFAvailable._proof_2 baseWeight forbidden havailable_pos)
```

Direct retained dependencies

- AppliedModelingLib.finiteAvailableWeight
- AppliedModelingLib.finiteWeightedPMF

AppliedModelingLib.finiteWithoutReplacementPMF

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
  [inst : Fintype α] →
    [inst_1 : DecidableEq α] →
      (baseWeight : α → ℝ) →
        (∀ (a : α), 0 ≤ baseWeight a) →
          (∀ (forbidden : Finset α),
             forbidden.card < Fintype.card α → 0 < AppliedModelingLib.finiteAvailableWeight baseWeight forbidden) →
            (k : ℕ) →
              (forbidden : Finset α) →
                forbidden.card + k ≤ Fintype.card α → PMF (AppliedModelingLib.finiteFreshList α k forbidden)

value:
```

```
fun {α : Type u_1} [Fintype α] [DecidableEq α] (baseWeight : α → ℝ) (hbase_nonneg : ∀ (a : α), 0 ≤ baseWeight a)
    (havailable :
      ∀ (forbidden : Finset α),
        forbidden.card < Fintype.card α → 0 < AppliedModelingLib.finiteAvailableWeight baseWeight forbidden)
    (k : ℕ) (forbidden : Finset α) (a : forbidden.card + k ≤ Fintype.card α) =>
  AppliedModelingLib.finiteWithoutReplacementPMF._unary baseWeight hbase_nonneg havailable (k, ⟨forbidden, a⟩)
```

Direct retained dependencies

- AppliedModelingLib.finiteAvailableWeight
- AppliedModelingLib.finiteFreshList
- AppliedModelingLib.finiteFreshListCons
- AppliedModelingLib.finiteFreshListNil
- AppliedModelingLib.finiteWeightedPMFAvailable

AppliedModelingLib.measureProb

Declaration location: reusable library

Lean declaration body

type:
```
{α : Type u_1} → [inst : MeasurableSpace α] → MeasureTheory.Measure α → (α → Prop) → ℝ
```

value:
```
fun {α : Type u_1} [MeasurableSpace α] (μ : MeasureTheory.Measure α) (p : α → Prop) => (μ {a : α | p a}).toReal
```

AppliedModelingLib.pmfExp

Declaration location: reusable library

Lean declaration body

type:
```
{α : Type u_1} → [Fintype α] → PMF α → (α → ℝ) → ℝ
```

value:
```
fun {α : Type u_1} [Fintype α] (μ : PMF α) (f : α → ℝ) => ∑ a : α, (μ a).toReal * f a
```

AppliedModelingLib.SocialChoice.Ranking.expectedFirstMoverUtility

Declaration location: reusable library

Lean declaration body

type:
```
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → ℝ
```

value:
```
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.pmfExp μ fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    value (AppliedModelingLib.SocialChoice.Ranking.firstChoice π)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.pmfExp

AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.pmfExp μ fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    value (AppliedModelingLib.SocialChoice.Ranking.secondChoice π)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.pmfExp

AppliedModelingLib.pmfPairExp

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
  {β : Type u_2} → [Fintype α] → [DecidableEq α] → [Fintype β] → [DecidableEq β] → PMF α → PMF β → (α → β → ℝ) → ℝ

value:
fun {α : Type u_1} {β : Type u_2} [Fintype α] [DecidableEq α] [Fintype β] [DecidableEq β] (μ : PMF α) (ν : PMF β)
    (f : α → β → ℝ) =>
  AppliedModelingLib.pmfExp μ fun (a : α) => AppliedModelingLib.pmfExp ν fun (b : β) => f a b
```

Direct retained dependencies

- AppliedModelingLib.pmfExp

AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → ℝ

value:
fun {n : ℕ} (μ₂ μ₁ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.pmfPairExp μ₂ μ₁ fun (π σ : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value π σ
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfPairExp

AppliedModelingLib.SocialChoice.Ranking.PrefersIndependentReranking

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared μ value <
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent μ μ value

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared

AppliedModelingLib.SocialChoice.Ranking.PrefersWeakerCompetition

Declaration location: reusable library

Lean declaration body

type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop

value:
fun {n : ℕ} (μBetter μWorse : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent μWorse μBetter value <
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent μWorse μWorse value

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent

AppliedModelingLib.pmfPairIndicatorExp

Declaration location: reusable library

Lean declaration body

type:
{α : Type u_1} →
  {β : Type u_2} →
    [Fintype α] →
      [DecidableEq α] →
        [Fintype β] → [DecidableEq β] → PMF α → PMF β → (p : α × β → Prop) → [DecidablePred p] → (α × β → ℝ) → ℝ

value:
fun {α : Type u_1} {β : Type u_2} [Fintype α] [DecidableEq α] [Fintype β] [DecidableEq β] (μ : PMF α) (ν : PMF β)
    (p : α × β → Prop) [DecidablePred p] (f : α × β → ℝ) =>
  AppliedModelingLib.pmfPairExp μ ν fun (a : α) (b : β) => if p (a, b) then f (a, b) else 0

Direct retained dependencies

- AppliedModelingLib.pmfPairExp

AppliedModelingLib.pmfPairProb

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
  {β : Type u_2} →
    [Fintype α] →
      [DecidableEq α] → [Fintype β] → [DecidableEq β] → PMF α → PMF β → (p : α × β → Prop) → [DecidablePred p] → ℝ

value:
fun {α : Type u_1} {β : Type u_2} [Fintype α] [DecidableEq α] [Fintype β] [DecidableEq β] (μ : PMF α) (ν : PMF β)
    (p : α × β → Prop) [DecidablePred p] =>
  AppliedModelingLib.pmfPairExp μ ν fun (a : α) (b : β) => if p (a, b) then 1 else 0
```

Direct retained dependencies

- AppliedModelingLib.pmfPairExp

AppliedModelingLib.SocialChoice.Ranking.disagreementProb

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)) =>
  AppliedModelingLib.pmfPairProb μ μ AppliedModelingLib.SocialChoice.Ranking.disagreementEvent
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.decidableDisagreementEvent
- AppliedModelingLib.SocialChoice.Ranking.disagreementEvent
- AppliedModelingLib.pmfPairProb

AppliedModelingLib.pmfProb

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → PMF α → (p : α → Prop) → [DecidablePred p] → ℝ

value:
fun {α : Type u_1} [Fintype α] (μ : PMF α) (p : α → Prop) [DecidablePred p] =>
  AppliedModelingLib.pmfExp μ fun (a : α) => if p a then 1 else 0
```

Direct retained dependencies

- AppliedModelingLib.pmfExp

AppliedModelingLib.SocialChoice.Ranking.firstChoiceProb

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.pmfProb μ fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    c = AppliedModelingLib.SocialChoice.Ranking.firstChoice π
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.pmfProb

AppliedModelingLib.SocialChoice.Ranking.firstChoiceMissProb

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  1 - AppliedModelingLib.SocialChoice.Ranking.firstChoiceProb μ c
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoiceProb

AppliedModelingLib.pmfProd

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → {β : Type u_2} → [Fintype α] → [Fintype β] → PMF α → PMF β → PMF (α × β)

value:
fun {α : Type u_1} {β : Type u_2} [Fintype α] [Fintype β] (μ : PMF α) (ν : PMF β) =>
  PMF.ofFintype (fun (p : α × β) => μ p.1 * ν p.2) (AppliedModelingLib.pmfProd._proof_1 μ ν)
```

KR21Monoculture.AppendixB1NoiseAtom

Declaration location: paper-local

Lean declaration body

```
type:
Type

constructor KR21Monoculture.AppendixB1NoiseAtom.plusOne:
KR21Monoculture.AppendixB1NoiseAtom

constructor KR21Monoculture.AppendixB1NoiseAtom.zero:
KR21Monoculture.AppendixB1NoiseAtom

constructor KR21Monoculture.AppendixB1NoiseAtom.minusOne:
KR21Monoculture.AppendixB1NoiseAtom
```

KR21Monoculture.AppendixB1NoiseTriple

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
(KR21Monoculture.AppendixB1NoiseAtom × KR21Monoculture.AppendixB1NoiseAtom) × KR21Monoculture.AppendixB1NoiseAtom
```

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom

KR21Monoculture.AppendixB2NoiseAtom

Declaration location: paper-local

Lean declaration body

type:
Type

constructor KR21Monoculture.AppendixB2NoiseAtom.plusOne:
KR21Monoculture.AppendixB2NoiseAtom

constructor KR21Monoculture.AppendixB2NoiseAtom.minusOne:
KR21Monoculture.AppendixB2NoiseAtom

constructor KR21Monoculture.AppendixB2NoiseAtom.plusTen:
KR21Monoculture.AppendixB2NoiseAtom

constructor KR21Monoculture.AppendixB2NoiseAtom.minusTen:
KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.AppendixB2NoiseTriple

Declaration location: paper-local

Lean declaration body

type:
Type

value:
(KR21Monoculture.AppendixB2NoiseAtom × KR21Monoculture.AppendixB2NoiseAtom) × KR21Monoculture.AppendixB2NoiseAtom

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.AppendixBGaussianTriple

Declaration location: paper-local

Lean declaration body

type:
Type

value:
$\mathbb{R} \times \mathbb{R} \times \mathbb{R}$

KR21Monoculture.AppendixBRankingAtom

Declaration location: paper-local

Lean declaration body

type:
Type

constructor KR21Monoculture.AppendixBRankingAtom.r012:
KR21Monoculture.AppendixBRankingAtom

constructor KR21Monoculture.AppendixBRankingAtom.r021:
KR21Monoculture.AppendixBRankingAtom

constructor KR21Monoculture.AppendixBRankingAtom.r102:
KR21Monoculture.AppendixBRankingAtom

constructor KR21Monoculture.AppendixBRankingAtom.r120:
KR21Monoculture.AppendixBRankingAtom

constructor KR21Monoculture.AppendixBRankingAtom.r201:
KR21Monoculture.AppendixBRankingAtom

constructor KR21Monoculture.AppendixBRankingAtom.r210:
KR21Monoculture.AppendixBRankingAtom

KR21Monoculture.AccuracyFamily.modelAt

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} → KR21Monoculture.AccuracyFamily n → ℝ → ℝ → KR21Monoculture.Model n
```

value:
```
fun {n : ℕ} (F : KR21Monoculture.AccuracyFamily n) (θA θH : ℝ) =>
  { algorithmRanking := F.dist θA, humanRanking := F.dist θH, value := F.value }
```

Direct retained dependencies

- KR21Monoculture.AccuracyFamily
- KR21Monoculture.Model

KR21Monoculture.Model.PrefersIndependentReranking

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop
```

value:
```
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.PrefersIndependentReranking μ value
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.PrefersIndependentReranking
- AppliedModelingLib.SocialChoice.Ranking.Ranking

KR21Monoculture.Model.PrefersWeakerCompetition

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop
```

value:
```
fun {n : ℕ} (μBetter μWorse : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.PrefersWeakerCompetition μBetter μWorse value
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.PrefersWeakerCompetition
- AppliedModelingLib.SocialChoice.Ranking.Ranking

KR21Monoculture.PaperInterface.instMeasurableSpaceAppendixB1NoiseAtom_1

Declaration location: paper-local

Lean declaration body

type:
```
MeasurableSpace KR21Monoculture.AppendixB1NoiseAtom
```

value:
```
⊤
```

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom

KR21Monoculture.PaperInterface.instMeasurableSpaceAppendixB2NoiseAtom_1

Declaration location: paper-local

Lean declaration body

type:
MeasurableSpace KR21Monoculture.AppendixB2NoiseAtom

value:
⊤

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.RankingPair

Declaration location: paper-local

Lean declaration body

type:
ℕ → Type

value:
fun (n : ℕ) => AppliedModelingLib.SocialChoice.Ranking.Ranking n × AppliedModelingLib.SocialChoice.Ranking.Ranking n

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking

KR21Monoculture.Model.rankingDist

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → KR21Monoculture.Model n → KR21Monoculture.Strategy → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
```
fun {n : ℕ} (M : KR21Monoculture.Model n) (a : KR21Monoculture.Strategy) =>
  match a with
  | KR21Monoculture.Strategy.algorithm => M.algorithmRanking
  | KR21Monoculture.Strategy.human => M.humanRanking
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.Model
- KR21Monoculture.Strategy

KR21Monoculture.Model.firstMoverEU

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → KR21Monoculture.Model n → KR21Monoculture.Strategy → ℝ

value:
```
fun {n : ℕ} (M : KR21Monoculture.Model n) (s : KR21Monoculture.Strategy) =>
  AppliedModelingLib.SocialChoice.Ranking.expectedFirstMoverUtility (M.rankingDist s) M.value
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.expectedFirstMoverUtility
- KR21Monoculture.Model
- KR21Monoculture.Model.rankingDist
- KR21Monoculture.Strategy

KR21Monoculture.Model.secondMoverEU

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.Model n → KR21Monoculture.Strategy → KR21Monoculture.Strategy → ℝ

value:
fun {n : ℕ} (M : KR21Monoculture.Model n) (s₁ s₂ : KR21Monoculture.Strategy) =>
  match s₁, s₂ with
  | KR21Monoculture.Strategy.algorithm, KR21Monoculture.Strategy.algorithm =>
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared M.algorithmRanking M.value
  | x, x_1 =>
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (M.rankingDist s₂) (M.rankingDist s₁) M.value
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
- KR21Monoculture.Model
- KR21Monoculture.Model.rankingDist
- KR21Monoculture.Strategy

KR21Monoculture.Model.payoffAgainst

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.Model n → KR21Monoculture.Strategy → KR21Monoculture.Strategy → ℝ

value:
fun {n : ℕ} (M : KR21Monoculture.Model n) (self other : KR21Monoculture.Strategy) =>
  (M.firstMoverEU self + M.secondMoverEU other self) / 2
```

Direct retained dependencies

- KR21Monoculture.Model
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.Strategy

KR21Monoculture.StrictlyWellOrderedNoise

Declaration location: paper-local

Lean declaration body

```
type:
(ℝ → ℝ) → Prop

value:
fun (f : ℝ → ℝ) => AppliedModelingLib.Probability.StrictlyWellOrderedNoise f
```

Direct retained dependencies

- AppliedModelingLib.Probability.StrictlyWellOrderedNoise

KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

Declaration location: paper-local

Lean declaration body

type:
Type

value:
ℝ × ℝ × ℝ

KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace

Declaration location: paper-local

Lean declaration body

type:
Type

value:
KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

Direct retained dependencies

- KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

KR21Monoculture.DistributionalAccuracyFamily.independentPairLaw

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  KR21Monoculture.DistributionalAccuracyFamily n →
    ℝ → KR21Monoculture.ValueProfile n → PMF (KR21Monoculture.RankingPair n)

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n) (theta : ℝ) (value : KR21Monoculture.ValueProfile n) =>
  AppliedModelingLib.pmfProd (F.dist theta value) (F.dist theta value)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.pmfProd
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.DistributionalAccuracyFamily.independentPairKernel

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  (F : KR21Monoculture.DistributionalAccuracyFamily n) →
    (theta : ℝ) →
      (∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
          Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking) →
        ProbabilityTheory.Kernel (KR21Monoculture.ValueProfile n) (KR21Monoculture.RankingPair n)

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n) (theta : ℝ)
    (hatom :
      ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
        Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking) =>
  { toFun := fun (value : KR21Monoculture.ValueProfile n) => (F.independentPairLaw theta value).toMeasure,
    measurable' := KR21Monoculture.DistributionalAccuracyFamily.measurable_independentPairLaw_toMeasure F theta hatom }

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.independentPairLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.DistributionalAccuracyFamily.jointIndependentSecondMoverPayoff

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n → ℝ

value:
fun {n : ℕ} (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
  AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.DistributionalAccuracyFamily.jointSharedSecondMoverPayoff

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n → ℝ

value:
fun {n : ℕ} (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
  AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  (F : KR21Monoculture.DistributionalAccuracyFamily n) →
    MeasureTheory.Measure (KR21Monoculture.ValueProfile n) →
      (theta : ℝ) →
        (∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
            Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking) →
          MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n)

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) (theta : ℝ)
    (hatom :
      ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
        Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking) =>
  D.compProd (F.independentPairKernel theta hatom)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.independentPairKernel
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.DistributionalAccuracyFamily.pointFamily

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  KR21Monoculture.DistributionalAccuracyFamily n → KR21Monoculture.ValueProfile n → KR21Monoculture.AccuracyFamily n

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n) (value : KR21Monoculture.ValueProfile n) =>
  { dist := fun (theta : ℝ) => F.dist theta value, value := value }
```

Direct retained dependencies

- KR21Monoculture.AccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.ValueProfile

KR21Monoculture.PaperInterface.literal_outer_payoff_paradox

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  KR21Monoculture.DistributionalAccuracyFamily n → MeasureTheory.Measure (KR21Monoculture.ValueProfile n) → ℝ → Prop

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) (thetaH : ℝ) =>
  ∃ (thetaA : ℝ),
    thetaH < thetaA ∧
      ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
              KR21Monoculture.Strategy.human ∂D <
        ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
              KR21Monoculture.Strategy.algorithm ∂D ∧
       ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.human ∂D <
        ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.algorithm ∂D ∧
         ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
              KR21Monoculture.Strategy.algorithm ∂D <
        ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.human ∂D
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AccuracyFamily.modelAt
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.pointFamily
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.Strategy
- KR21Monoculture.ValueProfile

KR21Monoculture.SourceDefinition2

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} → KR21Monoculture.DistributionalAccuracyFamily n → MeasureTheory.Measure (KR21Monoculture.ValueProfile n) → Prop
```

value:
```
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) =>
  ∀ (theta : ℝ),
    0 < theta →
      ∀
        (hatom :
          ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
            Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking),
        have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
          F.outerIndependentPairJointLaw D theta hatom;
        have numerator : ℝ :=
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            if
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
              x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
            else 0 ∂J;
        have denominator : ℝ :=
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            if
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
              1
            else 0 ∂J;
        0 < denominator → 0 < numerator / denominator
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.SourceDefinition3

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.DistributionalAccuracyFamily n → MeasureTheory.Measure (KR21Monoculture.ValueProfile n) → Prop

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) =>
  ∀ (thetaA thetaH : ℝ),
    0 < thetaH →
      thetaH < thetaA →
        ∫ (value : KR21Monoculture.ValueProfile n),
            AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaA value)
              fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
              AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D <
          ∫ (value : KR21Monoculture.ValueProfile n),
            AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaH value)
              fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
              AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfPairExp
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.ValueProfile

KR21Monoculture.appendixB1NoiseTripleFunction

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB1NoiseTriple →
  AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB1NoiseAtom

value:
fun (noise : KR21Monoculture.AppendixB1NoiseTriple) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  match a with
  | 0 => noise.1.1
  | 1 => noise.1.2
  | x => noise.2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.AppendixB1NoiseTriple

KR21Monoculture.appendixB1NoiseValue

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB1NoiseAtom → ℝ

value:
fun (a : KR21Monoculture.AppendixB1NoiseAtom) =>
  match a with
  | KR21Monoculture.AppendixB1NoiseAtom.plusOne => 1
  | KR21Monoculture.AppendixB1NoiseAtom.zero => 0
  | KR21Monoculture.AppendixB1NoiseAtom.minusOne => -1
```

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom

KR21Monoculture.appendixB1NoiseWeight

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixB1NoiseAtom → ℝ

value:
```
fun (a : KR21Monoculture.AppendixB1NoiseAtom) =>
  match a with
  | KR21Monoculture.AppendixB1NoiseAtom.plusOne => 1 / 20
  | KR21Monoculture.AppendixB1NoiseAtom.zero => 9 / 10
  | KR21Monoculture.AppendixB1NoiseAtom.minusOne => 1 / 20
```

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom

KR21Monoculture.appendixB1NoisePMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixB1NoiseAtom

value:
```
AppliedModelingLib.finiteWeightedPMF KR21Monoculture.appendixB1NoiseWeight
  KR21Monoculture.appendixB1NoiseWeight_nonneg_for_mixture✝ KR21Monoculture.appendixB1NoisePMF._proof_1
```

Direct retained dependencies

- AppliedModelingLib.finiteWeightedPMF
- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.appendixB1NoiseWeight

KR21Monoculture.appendixB1NoiseTriplePMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixB1NoiseTriple

value:
```
AppliedModelingLib.pmfProd
  (AppliedModelingLib.pmfProd KR21Monoculture.appendixB1NoisePMF KR21Monoculture.appendixB1NoisePMF)
  KR21Monoculture.appendixB1NoisePMF
```

Direct retained dependencies

- AppliedModelingLib.pmfProd
- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.appendixB1NoisePMF

KR21Monoculture.appendixB1RankingWeight

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixBRankingAtom → ℝ

value:
```
fun (a : KR21Monoculture.AppendixBRankingAtom) =>
  match a with
  | KR21Monoculture.AppendixBRankingAtom.r012 => 181 / 200
  | KR21Monoculture.AppendixBRankingAtom.r021 => 721 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r102 => 19 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r120 => 1 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r201 => 19 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r210 => 0
```

Direct retained dependencies

- KR21Monoculture.AppendixBRankingAtom

KR21Monoculture.appendixB1AtomPMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixBRankingAtom

value:
AppliedModelingLib.finiteWeightedPMF KR21Monoculture.appendixB1RankingWeight
  KR21Monoculture.appendixB1RankingWeight_nonneg† KR21Monoculture.appendixB1RankingWeight_sum_pos†

Direct retained dependencies

- AppliedModelingLib.finiteWeightedPMF
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.appendixB1RankingWeight

KR21Monoculture.appendixB1Value

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) => if c = 0 then 7 / 4 else if c = 1 then 1 / 2 else 0

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.appendixB1DiscreteScore

Declaration location: paper-local

Lean declaration body

type:
(AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB1NoiseAtom) →
  AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (noise : AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB1NoiseAtom)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  KR21Monoculture.appendixB1Value c + KR21Monoculture.appendixB1NoiseValue (noise c)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.appendixB1NoiseValue
- KR21Monoculture.appendixB1Value

KR21Monoculture.appendixB2AlgorithmRankingWeight

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixBRankingAtom → ℝ

value:
```
fun (a : KR21Monoculture.AppendixBRankingAtom) =>
  match a with
  | KR21Monoculture.AppendixBRankingAtom.r012 => 4999 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r021 => 361 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r102 => 19 / 80
  | KR21Monoculture.AppendixBRankingAtom.r120 => 361 / 8000
  | KR21Monoculture.AppendixBRankingAtom.r201 => 7 / 200
  | KR21Monoculture.AppendixBRankingAtom.r210 => 99 / 8000
```

Direct retained dependencies

- KR21Monoculture.AppendixBRankingAtom

KR21Monoculture.appendixB2AlgorithmAtomPMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixBRankingAtom

value:
```
AppliedModelingLib.finiteWeightedPMF KR21Monoculture.appendixB2AlgorithmRankingWeight
  KR21Monoculture.appendixB2AlgorithmRankingWeight_nonneg† KR21Monoculture.appendixB2AlgorithmRankingWeight_sum_pos†
```

Direct retained dependencies

- AppliedModelingLib.finiteWeightedPMF
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.appendixB2AlgorithmRankingWeight

KR21Monoculture.appendixB2HumanRankingWeight

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixBRankingAtom → ℝ

value:
```
fun (a : KR21Monoculture.AppendixBRankingAtom) =>
  match a with
  | KR21Monoculture.AppendixBRankingAtom.r012 => 173 / 400
  | KR21Monoculture.AppendixBRankingAtom.r021 => 19 / 80
  | KR21Monoculture.AppendixBRankingAtom.r102 => 19 / 80
  | KR21Monoculture.AppendixBRankingAtom.r120 => 7 / 200
  | KR21Monoculture.AppendixBRankingAtom.r201 => 7 / 200
  | KR21Monoculture.AppendixBRankingAtom.r210 => 9 / 400
```

Direct retained dependencies

- KR21Monoculture.AppendixBRankingAtom

KR21Monoculture.appendixB2HumanAtomPMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixBRankingAtom

value:
```
AppliedModelingLib.finiteWeightedPMF KR21Monoculture.appendixB2HumanRankingWeight
  KR21Monoculture.appendixB2HumanRankingWeight_nonneg† KR21Monoculture.appendixB2HumanRankingWeight_sum_pos†
```

Direct retained dependencies

- AppliedModelingLib.finiteWeightedPMF
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.appendixB2HumanRankingWeight

KR21Monoculture.appendixB2NoiseTripleFunction

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB2NoiseTriple →
  AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB2NoiseAtom

value:
fun (noise : KR21Monoculture.AppendixB2NoiseTriple) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  match a with
  | 0 => noise.1.1
  | 1 => noise.1.2
  | x => noise.2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple

KR21Monoculture.appendixB2NoiseValue

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB2NoiseAtom → ℝ

value:
fun (a : KR21Monoculture.AppendixB2NoiseAtom) =>
  match a with
  | KR21Monoculture.AppendixB2NoiseAtom.plusOne => 1
  | KR21Monoculture.AppendixB2NoiseAtom.minusOne => -1
  | KR21Monoculture.AppendixB2NoiseAtom.plusTen => 10
  | KR21Monoculture.AppendixB2NoiseAtom.minusTen => -10
```

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.appendixB2NoiseWeight

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB2NoiseAtom → ℝ

value:
fun (a : KR21Monoculture.AppendixB2NoiseAtom) =>
  match a with
  | KR21Monoculture.AppendixB2NoiseAtom.plusOne => 9 / 20
  | KR21Monoculture.AppendixB2NoiseAtom.minusOne => 9 / 20
  | KR21Monoculture.AppendixB2NoiseAtom.plusTen => 1 / 20
  | KR21Monoculture.AppendixB2NoiseAtom.minusTen => 1 / 20
```

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.appendixB2NoisePMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixB2NoiseAtom

value:
AppliedModelingLib.finiteWeightedPMF KR21Monoculture.appendixB2NoiseWeight
  KR21Monoculture.appendixB2NoiseWeight_nonneg_for_mixture✝ KR21Monoculture.appendixB2NoisePMF._proof_1

Direct retained dependencies

- AppliedModelingLib.finiteWeightedPMF
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.appendixB2NoiseWeight

KR21Monoculture.appendixB2NoiseTriplePMF

Declaration location: paper-local

Lean declaration body

type:
PMF KR21Monoculture.AppendixB2NoiseTriple

value:
AppliedModelingLib.pmfProd
  (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF KR21Monoculture.appendixB2NoisePMF)
  KR21Monoculture.appendixB2NoisePMF

Direct retained dependencies

- AppliedModelingLib.pmfProd
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.appendixB2NoisePMF

KR21Monoculture.appendixB2Value

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) => if c = 0 then 3 else if c = 1 then 2 else 0

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.appendixB2AlgorithmDiscreteScore

Declaration location: paper-local

Lean declaration body

type:
(AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB2NoiseAtom) →
  AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (noise : AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB2NoiseAtom)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  KR21Monoculture.appendixB2Value c + 10 / 11 * KR21Monoculture.appendixB2NoiseValue (noise c)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixB2Value

KR21Monoculture.appendixB2HumanDiscreteScore

Declaration location: paper-local

Lean declaration body

```
type:
(AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB2NoiseAtom) →
  AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (noise : AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → KR21Monoculture.AppendixB2NoiseAtom)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  KR21Monoculture.appendixB2Value c + 10 / 9 * KR21Monoculture.appendixB2NoiseValue (noise c)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixB2Value

KR21Monoculture.appendixBGaussianTripleFunction

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixBGaussianTriple → AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (z : KR21Monoculture.AppendixBGaussianTriple) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  match a with
  | 0 => z.1
  | 1 => z.2.1
  | x => z.2.2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixBGaussianTriple

KR21Monoculture.appendixB1SourceGaussianMixtureNoise

Declaration location: paper-local

Lean declaration body

```
type:
ℝ →
  KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple →
    AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (s : ℝ) (omega : KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  KR21Monoculture.appendixB1NoiseValue (KR21Monoculture.appendixB1NoiseTripleFunction omega.1 c) +
    s * KR21Monoculture.appendixBGaussianTripleFunction omega.2 c
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB1NoiseTripleFunction
- KR21Monoculture.appendixB1NoiseValue
- KR21Monoculture.appendixBGaussianTripleFunction

KR21Monoculture.appendixB1SourceGaussianMixtureRank

Declaration location: paper-local

Lean declaration body

```
type:
ℝ →
  ℝ →
    KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple →
      AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
fun (s theta : ℝ) (a : KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB1Value c + KR21Monoculture.appendixB1SourceGaussianMixtureNoise s a c / theta
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB1SourceGaussianMixtureNoise
- KR21Monoculture.appendixB1Value

KR21Monoculture.appendixB2SourceGaussianMixtureNoise

Declaration location: paper-local

Lean declaration body

```
type:
ℝ →
  KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple →
    AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (s : ℝ) (omega : KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple)
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction omega.1 c) +
    s * KR21Monoculture.appendixBGaussianTripleFunction omega.2 c
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixBGaussianTripleFunction

KR21Monoculture.appendixB2SourceGaussianMixtureRank

Declaration location: paper-local

Lean declaration body

type:
ℝ →
  ℝ →
    KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple →
      AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
fun (s theta : ℝ) (a : KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c + KR21Monoculture.appendixB2SourceGaussianMixtureNoise s a c / theta

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB2SourceGaussianMixtureNoise
- KR21Monoculture.appendixB2Value

KR21Monoculture.appendixBGaussianVariance

Declaration location: paper-local

Lean declaration body

type:
ℝ → NNReal

value:
fun (s : ℝ) => (s ^ 2, KR21Monoculture.appendixBGaussianVariance._proof_1 s)

KR21Monoculture.appendixBRankingAtomOfScores

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → KR21Monoculture.AppendixBRankingAtom

value:
fun (s0 s1 s2 : ℝ) =>
  if s1 ≤ s0 ∧ s2 ≤ s0 then
    if s2 ≤ s1 then KR21Monoculture.AppendixBRankingAtom.r012 else KR21Monoculture.AppendixBRankingAtom.r021
  else
    if s0 < s1 ∧ s2 ≤ s1 then
      if s2 ≤ s0 then KR21Monoculture.AppendixBRankingAtom.r102 else KR21Monoculture.AppendixBRankingAtom.r120
    else if s1 ≤ s0 then KR21Monoculture.AppendixBRankingAtom.r201 else KR21Monoculture.AppendixBRankingAtom.r210

Direct retained dependencies

- KR21Monoculture.AppendixBRankingAtom

KR21Monoculture.appendixB1DiscreteTripleAtom

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixB1NoiseTriple → KR21Monoculture.AppendixBRankingAtom

value:
fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
  KR21Monoculture.appendixBRankingAtomOfScores
    (KR21Monoculture.appendixB1DiscreteScore (KR21Monoculture.appendixB1NoiseTripleFunction noise) 0)
    (KR21Monoculture.appendixB1DiscreteScore (KR21Monoculture.appendixB1NoiseTripleFunction noise) 1)
    (KR21Monoculture.appendixB1DiscreteScore (KR21Monoculture.appendixB1NoiseTripleFunction noise) 2)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.appendixB1DiscreteScore
- KR21Monoculture.appendixB1NoiseTripleFunction
- KR21Monoculture.appendixBRankingAtomOfScores

KR21Monoculture.appendixB2AlgorithmDiscreteTripleAtom

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB2NoiseTriple → KR21Monoculture.AppendixBRankingAtom

value:
fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  KR21Monoculture.appendixBRankingAtomOfScores
    (KR21Monoculture.appendixB2AlgorithmDiscreteScore (KR21Monoculture.appendixB2NoiseTripleFunction noise) 0)
    (KR21Monoculture.appendixB2AlgorithmDiscreteScore (KR21Monoculture.appendixB2NoiseTripleFunction noise) 1)
    (KR21Monoculture.appendixB2AlgorithmDiscreteScore (KR21Monoculture.appendixB2NoiseTripleFunction noise) 2)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.appendixB2AlgorithmDiscreteScore
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixBRankingAtomOfScores

KR21Monoculture.appendixB2HumanDiscreteTripleAtom

Declaration location: paper-local

Lean declaration body

```
type:
KR21Monoculture.AppendixB2NoiseTriple → KR21Monoculture.AppendixBRankingAtom

value:
fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  KR21Monoculture.appendixBRankingAtomOfScores
    (KR21Monoculture.appendixB2HumanDiscreteScore (KR21Monoculture.appendixB2NoiseTripleFunction noise) 0)
    (KR21Monoculture.appendixB2HumanDiscreteScore (KR21Monoculture.appendixB2NoiseTripleFunction noise) 1)
    (KR21Monoculture.appendixB2HumanDiscreteScore (KR21Monoculture.appendixB2NoiseTripleFunction noise) 2)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.appendixB2HumanDiscreteScore
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixBRankingAtomOfScores

KR21Monoculture.appendixBStandardGaussianTripleMeasure

Declaration location: paper-local

Lean declaration body

type:
MeasureTheory.Measure KR21Monoculture.AppendixBGaussianTriple

value:
(ProbabilityTheory.gaussianReal 0 1).prod
  ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))

Direct retained dependencies

- KR21Monoculture.AppendixBGaussianTriple

KR21Monoculture.bestInSet

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n →
    Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n) → AppliedModelingLib.SocialChoice.Ranking.Candidate n

value:
fun {n : ℕ} (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)) =>
  if h : remaining.Nonempty then
    π
      ((Finset.image (AppliedModelingLib.SocialChoice.Ranking.rankOf π) remaining).min'
        (KR21Monoculture.bestInSet._proof_1 π remaining h))
  else AppliedModelingLib.SocialChoice.Ranking.firstChoice π

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.rankOf

KR21Monoculture.disagreementEvent

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → KR21Monoculture.RankingPair n → Prop

value:
fun {n : ℕ} (a : KR21Monoculture.RankingPair n) =>
  AppliedModelingLib.SocialChoice.Ranking.firstChoice a.1 ≠ AppliedModelingLib.SocialChoice.Ranking.firstChoice a.2

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- KR21Monoculture.RankingPair

KR21Monoculture.decidableDisagreementEvent

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → (a : KR21Monoculture.RankingPair n) → Decidable (KR21Monoculture.disagreementEvent a)

value:
fun {n : ℕ} (a : KR21Monoculture.RankingPair n) => id inferInstance

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- KR21Monoculture.RankingPair
- KR21Monoculture.disagreementEvent

KR21Monoculture.disagreementProb

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)) =>
  AppliedModelingLib.SocialChoice.Ranking.disagreementProb μ
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.disagreementProb

KR21Monoculture.DistributionalAccuracyFamily.OuterIndependentRerankingRegularity

Declaration location: paper-local

Lean declaration body

```
field outer_is_probability:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ},
  F.OuterIndependentRerankingRegularity D theta → MeasureTheory.IsProbabilityMeasure D

field ranking_atom_measurable:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ},
  F.OuterIndependentRerankingRegularity D theta →
    ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
      Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking

field disagreement_integrable:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ},
  F.OuterIndependentRerankingRegularity D theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) => KR21Monoculture.disagreementProb (F.dist theta value)) D

field shared_payoff_integrable:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ},
  F.OuterIndependentRerankingRegularity D theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value) value)
      D

field independent_payoff_integrable:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ},
  F.OuterIndependentRerankingRegularity D theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (F.dist theta value) (F.dist theta value)
          value)
      D
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared

- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.ValueProfile
- KR21Monoculture.disagreementProb

KR21Monoculture.DistributionalAccuracyFamily.OuterIndependentRerankingJointRegularity

Declaration location: paper-local

Lean declaration body

```
field base:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ},
  F.OuterIndependentRerankingJointRegularity D theta → F.OuterIndependentRerankingRegularity D theta

field joint_shared_payoff_integrable:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ}
  (self : F.OuterIndependentRerankingJointRegularity D theta),
  MeasureTheory.Integrable KR21Monoculture.DistributionalAccuracyFamily.jointSharedSecondMoverPayoff
    (F.outerIndependentPairJointLaw D theta self.base.ranking_atom_measurable)

field joint_independent_payoff_integrable:
∀ {n : ℕ} {F : KR21Monoculture.DistributionalAccuracyFamily n}
  {D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)} {theta : ℝ}
  (self : F.OuterIndependentRerankingJointRegularity D theta),
  MeasureTheory.Integrable KR21Monoculture.DistributionalAccuracyFamily.jointIndependentSecondMoverPayoff
    (F.outerIndependentPairJointLaw D theta self.base.ranking_atom_measurable)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.OuterIndependentRerankingRegularity
- KR21Monoculture.DistributionalAccuracyFamily.jointIndependentSecondMoverPayoff
- KR21Monoculture.DistributionalAccuracyFamily.jointSharedSecondMoverPayoff
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

KR21Monoculture.SourceDefinition1NoisyPermutationFamily

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.AccuracyFamily n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {n : ℕ} (F : KR21Monoculture.AccuracyFamily n) (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  (∀ (theta : ℝ),
      0 < theta →
        ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
          ContinuousAt (fun (theta' : ℝ) => ((F.dist theta') pi).toReal) theta ∧
            DifferentiableAt ℝ (fun (theta' : ℝ) => ((F.dist theta') pi).toReal) theta) ∧
    Filter.Tendsto (fun (theta : ℝ) => ((F.dist theta) center).toReal) Filter.atTop (nhds 1) ∧
      ∀ (thetaA thetaH : ℝ),
        0 < thetaH →
          thetaH < thetaA →
            (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                remaining.Nonempty →
                  KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value remaining ≤
                    KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value remaining) ∧
              KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value Finset.univ <
                KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value Finset.univ
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.expectedBestInSet

KR21Monoculture.SourceDefinition1

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → KR21Monoculture.AccuracyFamily n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → Prop

value:
fun {n : ℕ} (F : KR21Monoculture.AccuracyFamily n) (trueRanking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  KR21Monoculture.SourceDefinition1NoisyPermutationFamily F trueRanking
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.SourceDefinition1NoisyPermutationFamily

KR21Monoculture.finiteGaussianMixtureDensity

Declaration location: paper-local

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → PMF α → (α → ℝ) → ℝ → ℝ → ℝ

value:
fun {α : Type u_1} [Fintype α] (law : PMF α) (center : α → ℝ) (s x : ℝ) =>
  ∑ a : α, (law a).toReal * ProbabilityTheory.gaussianPDFReal (center a) (KR21Monoculture.appendixBGaussianVariance s) x
```

Direct retained dependencies

- KR21Monoculture.appendixBGaussianVariance

KR21Monoculture.firstChoiceMissProb

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.SocialChoice.Ranking.firstChoiceMissProb μ c
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoiceMissProb

KR21Monoculture.firstChoiceProb

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.SocialChoice.Ranking.firstChoiceProb μ c

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoiceProb

KR21Monoculture.fullFreshListToRanking

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  AppliedModelingLib.finiteFreshList (AppliedModelingLib.SocialChoice.Ranking.Candidate n) (n + 2) ∅ →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n

value:
fun {n : ℕ}
    (sample : AppliedModelingLib.finiteFreshList (AppliedModelingLib.SocialChoice.Ranking.Candidate n) (n + 2) ∅) =>
  Equiv.ofBijective (↑sample) (KR21Monoculture.fullFreshListToRanking._proof_1 sample)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.finiteFreshList

KR21Monoculture.gumbelArrivalLaw

Declaration location: paper-local

Lean declaration body

type:
(n : ℕ) → MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)

value:
fun (n : ℕ) =>
  MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    ProbabilityTheory.expMeasure 1

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom

Declaration location: paper-local

Lean declaration body

type:
MeasurableSpace KR21Monoculture.AppendixB1NoiseAtom

value:
⊤

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom

KR21Monoculture.appendixB1GaussianLatentMeasure

Declaration location: paper-local

Lean declaration body

type:
MeasureTheory.Measure (KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple)

value:
KR21Monoculture.appendixB1NoiseTriplePMF.toMeasure.prod KR21Monoculture.appendixBStandardGaussianTripleMeasure

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB1NoiseTriplePMF
- KR21Monoculture.appendixBStandardGaussianTripleMeasure
- KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom

KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom_2

Declaration location: paper-local

Lean declaration body

type:
MeasurableSpace KR21Monoculture.AppendixB1NoiseAtom

value:
⊤

Direct retained dependencies

- KR21Monoculture.AppendixB1NoiseAtom

KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom

Declaration location: paper-local

Lean declaration body

type:
MeasurableSpace KR21Monoculture.AppendixB2NoiseAtom

value:
⊤

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.appendixB2GaussianLatentMeasure

Declaration location: paper-local

Lean declaration body

type:
MeasureTheory.Measure (KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple)

value:
KR21Monoculture.appendixB2NoiseTriplePMF.toMeasure.prod KR21Monoculture.appendixBStandardGaussianTripleMeasure

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB2NoiseTriplePMF
- KR21Monoculture.appendixBStandardGaussianTripleMeasure
- KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom

KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom_1

Declaration location: paper-local

Lean declaration body

type:
MeasurableSpace KR21Monoculture.AppendixB2NoiseAtom

value:
⊤

Direct retained dependencies

- KR21Monoculture.AppendixB2NoiseAtom

KR21Monoculture.mallowsInverseAccuracyQ

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ

value:
fun (θ : ℝ) => (θ + 1)⁻¹

KR21Monoculture.mallowsAccuracyQ

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ

value:
fun (θ : ℝ) => if 0 < θ then KR21Monoculture.mallowsInverseAccuracyQ θ else 1

Direct retained dependencies

- KR21Monoculture.mallowsInverseAccuracyQ

KR21Monoculture.mallowsWeight

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → ℝ → AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ

value:
fun {n : ℕ} (q : ℝ) (ρ π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  q ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau ρ π

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau

KR21Monoculture.mallowsPartition

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} → ℝ → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ
```

value:
```
fun {n : ℕ} (q : ℝ) (ρ : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  ∑ π : AppliedModelingLib.SocialChoice.Ranking.Ranking n, KR21Monoculture.mallowsWeight q ρ π
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.mallowsWeight

KR21Monoculture.MallowsSpec

Declaration location: paper-local

Lean declaration body

field center:
```
{n : ℕ} → KR21Monoculture.MallowsSpec n → AppliedModelingLib.SocialChoice.Ranking.Ranking n
```

field q:
```
{n : ℕ} → KR21Monoculture.MallowsSpec n → ℝ
```

field law:
```
{n : ℕ} → KR21Monoculture.MallowsSpec n → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)
```

field partition:
```
{n : ℕ} → KR21Monoculture.MallowsSpec n → ℝ
```

field q_pos:
```
∀ {n : ℕ} (self : KR21Monoculture.MallowsSpec n), 0 < self.q
```

field partition_pos:
```
∀ {n : ℕ} (self : KR21Monoculture.MallowsSpec n), 0 < self.partition
```

field partition_eq_sum:
```
∀ {n : ℕ} (self : KR21Monoculture.MallowsSpec n), self.partition = KR21Monoculture.mallowsPartition self.q self.center
```

field law_apply_toReal:
```
∀ {n : ℕ} (self : KR21Monoculture.MallowsSpec n) (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  (self.law π).toReal = KR21Monoculture.mallowsWeight self.q self.center π / self.partition
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.mallowsPartition
- KR21Monoculture.mallowsWeight

KR21Monoculture.MallowsSpec.firstSecondChoiceProb

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} →
  KR21Monoculture.MallowsSpec n →
    AppliedModelingLib.SocialChoice.Ranking.Candidate n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
```

value:
```
fun {n : ℕ} (M : KR21Monoculture.MallowsSpec n) (c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  AppliedModelingLib.pmfProb M.law fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    c = AppliedModelingLib.SocialChoice.Ranking.firstChoice π ∧
      d = AppliedModelingLib.SocialChoice.Ranking.secondChoice π
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.pmfProb
- KR21Monoculture.MallowsSpec

KR21Monoculture.mallowsPMF

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  (q : ℝ) →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n → 0 < q → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {n : ℕ} (q : ℝ) (ρ : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (hq : 0 < q) =>
  PMF.ofFintype
    (fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
      ENNReal.ofReal (KR21Monoculture.mallowsWeight q ρ π / KR21Monoculture.mallowsPartition q ρ))
    (KR21Monoculture.mallowsPMF._proof_1 q ρ hq)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.mallowsPartition
- KR21Monoculture.mallowsWeight

KR21Monoculture.MallowsSpec.ofQ

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → (q : ℝ) → 0 < q → KR21Monoculture.MallowsSpec n

value:
fun {n : ℕ} (ρ : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (q : ℝ) (hq : 0 < q) =>
  { center := ρ, q := q, law := KR21Monoculture.mallowsPMF q ρ hq, partition := KR21Monoculture.mallowsPartition q ρ,
    q_pos := hq, partition_pos := KR21Monoculture.mallowsPartition_pos hq ρ,
    partition_eq_sum := KR21Monoculture.MallowsSpec.ofQ._proof_1 ρ q,
    law_apply_toReal := fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
      KR21Monoculture.mallowsPMF_apply_toReal hq ρ π }
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.MallowsSpec
- KR21Monoculture.mallowsPMF
- KR21Monoculture.mallowsPartition
- KR21Monoculture.mallowsWeight

KR21Monoculture.concreteMallowsSpec

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ → KR21Monoculture.MallowsSpec n

value:
fun {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (θ : ℝ) =>
  KR21Monoculture.MallowsSpec.ofQ center (KR21Monoculture.mallowsAccuracyQ θ) (KR21Monoculture.mallowsAccuracyQ_pos θ)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.MallowsSpec
- KR21Monoculture.MallowsSpec.ofQ
- KR21Monoculture.mallowsAccuracyQ

KR21Monoculture.fixedCenterMallowsDistributionalFamily

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → AppliedModelingLib.SocialChoice.Ranking.Ranking n → KR21Monoculture.DistributionalAccuracyFamily n

value:
fun {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  {
    dist := fun (theta : ℝ) (x : KR21Monoculture.ValueProfile n) =>
      (KR21Monoculture.concreteMallowsSpec center theta).law }

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.MallowsSpec
- KR21Monoculture.ValueProfile
- KR21Monoculture.concreteMallowsSpec

KR21Monoculture.fixedCenterMallowsPointFamily

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → KR21Monoculture.ValueProfile n → KR21Monoculture.AccuracyFamily n

value:
fun {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (value : KR21Monoculture.ValueProfile n) =>
  { dist := fun (theta : ℝ) => (KR21Monoculture.concreteMallowsSpec center theta).law, value := value }

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.MallowsSpec
- KR21Monoculture.ValueProfile
- KR21Monoculture.concreteMallowsSpec

KR21Monoculture.pairRerankingGain

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → KR21Monoculture.RankingPair n → ℝ

value:
fun {n : ℕ} (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) (a : KR21Monoculture.RankingPair n) =>
  AppliedModelingLib.SocialChoice.Ranking.pairRerankingGain value a
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.pairRerankingGain
- KR21Monoculture.RankingPair

KR21Monoculture.DistributionalAccuracyFamily.jointLawDisagreementConditionalGain

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  (F : KR21Monoculture.DistributionalAccuracyFamily n) →
    MeasureTheory.Measure (KR21Monoculture.ValueProfile n) →
      (theta : ℝ) →
        (∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
           Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking) →
          ℝ

value:
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) (theta : ℝ)
    (hatom :
      ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
        Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking) =>
  have M : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
    F.outerIndependentPairJointLaw D theta hatom;
  have p : ℝ :=
    ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
      if KR21Monoculture.disagreementEvent x.2 then 1 else 0 ∂M;
  if p = 0 then 0
  else
    (∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
       if KR21Monoculture.disagreementEvent x.2 then KR21Monoculture.pairRerankingGain x.1 x.2 else 0 ∂M) /
      p
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile
- KR21Monoculture.decidableDisagreementEvent
- KR21Monoculture.disagreementEvent
- KR21Monoculture.pairRerankingGain

KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  (μ : MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)) →
    [MeasureTheory.IsProbabilityMeasure μ] →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
        ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {n : ℕ} (μ : MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ))
    [MeasureTheory.IsProbabilityMeasure μ] (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) (θ : ℝ) =>
  let noise :
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
      AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ :=
    fun (eps : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
      (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    eps c;
  let rank :
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Ranking n :=
    fun (eps : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
    AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
      value i + noise eps i / θ;
  have hnoise :
    ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
      Measurable fun (eps : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) => noise eps c :=
    KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF._proof_1;
  have hrank : Measurable rank := KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF._proof_2 value θ hnoise;
  AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure μ rank hrank
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure

KR21Monoculture.plackettLuceChoiceProb

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
      Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n) →
        AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n))
    (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  if i ∈ remaining then Real.exp (theta * value i) / ∑ j ∈ remaining, Real.exp (theta * value j) else 0
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.plackettLuceWeight

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  Real.exp (theta * value i)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.plackettLuceFreshRankingPMF

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
      PMF (AppliedModelingLib.finiteFreshList (AppliedModelingLib.SocialChoice.Ranking.Candidate n) (n + 2) ∅)

value:
fun {n : ℕ} (theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.finiteWithoutReplacementPMF (KR21Monoculture.plackettLuceWeight theta value)
    (KR21Monoculture.plackettLuceFreshRankingPMF._proof_1 theta value)
    (KR21Monoculture.plackettLuceAvailableWeight_pos theta value) (n + 2) ∅
    KR21Monoculture.plackettLuceFreshRankingPMF._proof_2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.finiteAvailableWeight
- AppliedModelingLib.finiteFreshList
- AppliedModelingLib.finiteWithoutReplacementPMF
- KR21Monoculture.plackettLuceWeight

KR21Monoculture.plackettLuceRankingPMF

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {n : ℕ} (theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  PMF.map KR21Monoculture.fullFreshListToRanking (KR21Monoculture.plackettLuceFreshRankingPMF theta value)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.finiteFreshList
- KR21Monoculture.fullFreshListToRanking
- KR21Monoculture.plackettLuceFreshRankingPMF

KR21Monoculture.positiveScaleGumbelInnovation

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ

value:
fun (s arrival : ℝ) => -s * Real.log arrival
```

KR21Monoculture.commonLocationPositiveScaleGumbelInnovation

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ

value:
fun (location s arrival : ℝ) => location + KR21Monoculture.positiveScaleGumbelInnovation s arrival
```

Direct retained dependencies

- KR21Monoculture.positiveScaleGumbelInnovation

KR21Monoculture.commonLocationPositiveScaleGumbelMeasure

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → MeasureTheory.Measure ℝ

value:
fun (location s : ℝ) =>
  MeasureTheory.Measure.map (KR21Monoculture.commonLocationPositiveScaleGumbelInnovation location s)
    (ProbabilityTheory.expMeasure 1)
```

Direct retained dependencies

- KR21Monoculture.commonLocationPositiveScaleGumbelInnovation

KR21Monoculture.commonLocationPositiveScaleGumbelNoise

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    ℝ →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
        AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (location s : ℝ) (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (a : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  KR21Monoculture.commonLocationPositiveScaleGumbelInnovation location s (arrival a)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.commonLocationPositiveScaleGumbelInnovation

KR21Monoculture.commonLocationPositiveScaleGumbelScores

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    ℝ →
      ℝ →
        (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
          (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
            AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

value:
fun {n : ℕ} (location s theta : ℝ) (value arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (a : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  value a + KR21Monoculture.commonLocationPositiveScaleGumbelNoise location s arrival a / theta
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.commonLocationPositiveScaleGumbelNoise

KR21Monoculture.commonLocationPositiveScaleGumbelRank

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    ℝ →
      ℝ →
        (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
          (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Ranking n

value:
fun {n : ℕ} (location s theta : ℝ) (value a : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore
    (KR21Monoculture.commonLocationPositiveScaleGumbelScores location s theta value a)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.commonLocationPositiveScaleGumbelScores

KR21Monoculture.commonLocationPositiveScaleGumbelRUMRankingPMF

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    ℝ →
      ℝ →
        (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
          PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {n : ℕ} (location s theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure (KR21Monoculture.gumbelArrivalLaw n)
    (KR21Monoculture.commonLocationPositiveScaleGumbelRank location s theta value)
    (KR21Monoculture.measurable_commonLocationPositiveScaleGumbelRank location s theta value)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure
- KR21Monoculture.commonLocationPositiveScaleGumbelRank
- KR21Monoculture.gumbelArrivalLaw

KR21Monoculture.rightTripleToCandidateFunction

Declaration location: paper-local

Lean declaration body

```
type:
{α : Type u_1} → α × α × α → AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → α

value:
fun {α : Type u_1} (z : α × α × α) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  match a with
  | 0 => z.1
  | 1 => z.2.1
  | x => z.2.2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.rum3RankByScores

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
fun (s1 s2 s3 : ℝ) => AppliedModelingLib.SocialChoice.Ranking.rum3RankByScores s1 s2 s3

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3RankByScores

KR21Monoculture.rum3RankByScoreFns

Declaration location: paper-local

Lean declaration body

type:
{Ω : Type u_1} → (Ω → ℝ) → (Ω → ℝ) → (Ω → ℝ) → Ω → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
fun {Ω : Type u_1} (r1 r2 r3 : Ω → ℝ) (a : Ω) => KR21Monoculture.rum3RankByScores (r1 a) (r2 a) (r3 a)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.rum3RankByScores

KR21Monoculture.rum3Ranking012

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
AppliedModelingLib.SocialChoice.Ranking.rum3Ranking012

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking012

KR21Monoculture.rum3Ranking021

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
AppliedModelingLib.SocialChoice.Ranking.rum3Ranking021

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking021

KR21Monoculture.rum3Ranking102

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
AppliedModelingLib.SocialChoice.Ranking.rum3Ranking102

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking102

KR21Monoculture.rum3Ranking120

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
AppliedModelingLib.SocialChoice.Ranking.rum3Ranking120

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking120

KR21Monoculture.rum3Ranking201

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
AppliedModelingLib.SocialChoice.Ranking.rum3Ranking201

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking201

KR21Monoculture.rum3Ranking210

Declaration location: paper-local

Lean declaration body

type:
AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
AppliedModelingLib.SocialChoice.Ranking.rum3Ranking210

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rum3Ranking210

KR21Monoculture.AppendixBRankingAtom.toRanking

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixBRankingAtom → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
```
fun (a : KR21Monoculture.AppendixBRankingAtom) =>
  match a with
  | KR21Monoculture.AppendixBRankingAtom.r012 => KR21Monoculture.rum3Ranking012
  | KR21Monoculture.AppendixBRankingAtom.r021 => KR21Monoculture.rum3Ranking021
  | KR21Monoculture.AppendixBRankingAtom.r102 => KR21Monoculture.rum3Ranking102
  | KR21Monoculture.AppendixBRankingAtom.r120 => KR21Monoculture.rum3Ranking120
  | KR21Monoculture.AppendixBRankingAtom.r201 => KR21Monoculture.rum3Ranking201
  | KR21Monoculture.AppendixBRankingAtom.r210 => KR21Monoculture.rum3Ranking210
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.rum3Ranking012
- KR21Monoculture.rum3Ranking021
- KR21Monoculture.rum3Ranking102
- KR21Monoculture.rum3Ranking120
- KR21Monoculture.rum3Ranking201
- KR21Monoculture.rum3Ranking210

KR21Monoculture.appendixB1DiscreteTripleRank

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixB1NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
```
fun (noise : KR21Monoculture.AppendixB1NoiseTriple) => (KR21Monoculture.appendixB1DiscreteTripleAtom noise).toRanking
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixBRankingAtom.toRanking
- KR21Monoculture.appendixB1DiscreteTripleAtom

KR21Monoculture.appendixB1RankingPMF

Declaration location: paper-local

Lean declaration body

type:
PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
```
PMF.map KR21Monoculture.AppendixBRankingAtom.toRanking KR21Monoculture.appendixB1AtomPMF
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.AppendixBRankingAtom.toRanking
- KR21Monoculture.appendixB1AtomPMF

KR21Monoculture.appendixB2AlgorithmDiscreteTripleRank

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  (KR21Monoculture.appendixB2AlgorithmDiscreteTripleAtom noise).toRanking

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBRankingAtom.toRanking
- KR21Monoculture.appendixB2AlgorithmDiscreteTripleAtom

KR21Monoculture.appendixB2AlgorithmRankingPMF

Declaration location: paper-local

Lean declaration body

type:
PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
PMF.map KR21Monoculture.AppendixBRankingAtom.toRanking KR21Monoculture.appendixB2AlgorithmAtomPMF

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.AppendixBRankingAtom.toRanking
- KR21Monoculture.appendixB2AlgorithmAtomPMF

KR21Monoculture.appendixB2HumanDiscreteTripleRank

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1

value:
fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  (KR21Monoculture.appendixB2HumanDiscreteTripleAtom noise).toRanking

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBRankingAtom.toRanking
- KR21Monoculture.appendixB2HumanDiscreteTripleAtom

KR21Monoculture.appendixB2HumanRankingPMF

Declaration location: paper-local

Lean declaration body

type:
PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
PMF.map KR21Monoculture.AppendixBRankingAtom.toRanking KR21Monoculture.appendixB2HumanAtomPMF

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.AppendixBRankingAtom
- KR21Monoculture.AppendixBRankingAtom.toRanking
- KR21Monoculture.appendixB2HumanAtomPMF

KR21Monoculture.rumRankingPMFOfMeasure

Declaration location: paper-local

Lean declaration body

type:
{Ω : Type u_1} →
  [inst : MeasurableSpace Ω] →
    (μ : MeasureTheory.Measure Ω) →
      [MeasureTheory.IsProbabilityMeasure μ] →
        (rank : Ω → AppliedModelingLib.SocialChoice.Ranking.Ranking 1) →
          Measurable rank → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
fun {Ω : Type u_1} [MeasurableSpace Ω] (μ : MeasureTheory.Measure Ω) [MeasureTheory.IsProbabilityMeasure μ]
    (rank : Ω → AppliedModelingLib.SocialChoice.Ranking.Ranking 1) (hrank : Measurable rank) =>
  AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure μ rank hrank

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure

KR21Monoculture.appendixB1SourceGaussianMixtureFamily

Declaration location: paper-local

Lean declaration body

type:
ℝ → KR21Monoculture.AccuracyFamily 1

value:
fun (s : ℝ) =>
  {
    dist := fun (theta : ℝ) =>
      KR21Monoculture.rumRankingPMFOfMeasure KR21Monoculture.appendixB1GaussianLatentMeasure
        (KR21Monoculture.appendixB1SourceGaussianMixtureRank s theta)
        (KR21Monoculture.appendixB1SourceGaussianMixtureRank_measurable s theta),
    value := KR21Monoculture.appendixB1Value }

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB1GaussianLatentMeasure
- KR21Monoculture.appendixB1SourceGaussianMixtureRank

- KR21Monoculture.appendixB1Value
- KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom
- KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom_2
- KR21Monoculture.rumRankingPMFOfMeasure

KR21Monoculture.appendixB2SourceGaussianMixtureFamily

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → KR21Monoculture.AccuracyFamily 1

value:
fun (s : ℝ) =>
  {
    dist := fun (theta : ℝ) =>
      KR21Monoculture.rumRankingPMFOfMeasure KR21Monoculture.appendixB2GaussianLatentMeasure
        (KR21Monoculture.appendixB2SourceGaussianMixtureRank s theta)
        (KR21Monoculture.appendixB2SourceGaussianMixtureRank_measurable s theta),
    value := KR21Monoculture.appendixB2Value }
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.appendixB2GaussianLatentMeasure
- KR21Monoculture.appendixB2SourceGaussianMixtureRank
- KR21Monoculture.appendixB2Value
- KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom
- KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom_1
- KR21Monoculture.rumRankingPMFOfMeasure

KR21Monoculture.scaleOneGumbelInnovation

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ

value:
fun (arrival : ℝ) => -Real.log arrival
```

KR21Monoculture.scaleOneGumbelMeasure

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure ℝ

value:
MeasureTheory.Measure.map KR21Monoculture.scaleOneGumbelInnovation (ProbabilityTheory.expMeasure 1)
```

Direct retained dependencies

- KR21Monoculture.scaleOneGumbelInnovation

KR21Monoculture.sourceAppendixAProductNoise

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} → (Fin (n + 1) → ℝ) × ℝ → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
```

value:
```
fun {n : ℕ} (z : (Fin (n + 1) → ℝ) × ℝ) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  Fin.cases z.2 (fun (d : Fin (n + 1)) => z.1 d) a
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.sourceAppendixARestNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
```
(n : ℕ) → MeasureTheory.Measure ℝ → MeasureTheory.Measure (Fin (n + 1) → ℝ)
```

value:
```
fun (n : ℕ) (mu : MeasureTheory.Measure ℝ) => MeasureTheory.Measure.pi fun (x : Fin (n + 1)) => mu
```

KR21Monoculture.sourceRUMNormalizedScalarNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
```
MeasureTheory.Measure ℝ → ℝ → MeasureTheory.Measure ℝ
```

value:
```
fun (E : MeasureTheory.Measure ℝ) (sigma : ℝ) => MeasureTheory.Measure.map (fun (epsilon : ℝ) => epsilon / sigma) E
```

KR21Monoculture.sourceRUMNormalizedIIDNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} → MeasureTheory.Measure ℝ → ℝ → MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
```

value:
```
fun {n : ℕ} (E : MeasureTheory.Measure ℝ) (sigma : ℝ) =>
  MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    KR21Monoculture.sourceRUMNormalizedScalarNoiseLaw E sigma
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.sourceRUMNormalizedScalarNoiseLaw

KR21Monoculture.sourceUnitVarianceGumbelScores

Declaration location: paper-local

Lean declaration body

type:
```
{n : ℕ} →
  ℝ →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
        AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
```

value:
```
fun {n : ℕ} (theta : ℝ) (value epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (a : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
  value a + epsilon a / theta
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.sourceUnitVarianceGumbelRank

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} →
  ℝ →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → AppliedModelingLib.SocialChoice.Ranking.Ranking n

value:
fun {n : ℕ} (theta : ℝ) (value a : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore (KR21Monoculture.sourceUnitVarianceGumbelScores theta value a)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.sourceUnitVarianceGumbelScores

KR21Monoculture.sourceUnitVarianceLaplaceRate

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ

value:
fun (theta : ℝ) => √2 * theta
```

KR21Monoculture.theorem2GaussianBaseDensity

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ

value:
fun (a : ℝ) => KR21Monoculture.finiteGaussianMixtureDensity (PMF.pure ()) (fun (x : Unit) => 0) 1 a
```

Direct retained dependencies

- KR21Monoculture.finiteGaussianMixtureDensity

KR21Monoculture.theorem7LaplacePDF

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ

value:
fun (lam μ x : ℝ) => lam / 2 * Real.exp (-lam * |x - μ|)
```

KR21Monoculture.laplacePDFWeakDerivative

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ

value:
fun (lam x : ℝ) =>
  if x < 0 then lam * KR21Monoculture.theorem7LaplacePDF lam 0 x else -lam * KR21Monoculture.theorem7LaplacePDF lam 0 x

Direct retained dependencies

- KR21Monoculture.theorem7LaplacePDF

KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ

value:
fun (a : ℝ) => KR21Monoculture.theorem7LaplacePDF (√2) 0 a

Direct retained dependencies

- KR21Monoculture.theorem7LaplacePDF

KR21Monoculture.sourceUnitVarianceLaplaceBaseWeakDerivative

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ

value:
fun (a : ℝ) => KR21Monoculture.laplacePDFWeakDerivative (√2) a

Direct retained dependencies

- KR21Monoculture.laplacePDFWeakDerivative

KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity

Declaration location: paper-local

Lean declaration body

field density_integrable:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity →
  MeasureTheory.Integrable KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity MeasureTheory.volume

field derivative_integrable:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity →
  MeasureTheory.Integrable KR21Monoculture.sourceUnitVarianceLaplaceBaseWeakDerivative MeasureTheory.volume

field density_measurable:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity → Measurable KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity

field density_positive:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity →
  ∀ (x : ℝ), 0 < KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity x

field absolute_continuity:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity →
  ∀ (a b : ℝ), AbsolutelyContinuousOnInterval KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity a b

field derivative_ae_eq:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity →
  KR21Monoculture.sourceUnitVarianceLaplaceBaseWeakDerivative =ᵐ[MeasureTheory.volume]
    deriv KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity

field normalized:
KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity →
  ∫⁻ (x : ℝ), ENNReal.ofReal (KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity x) = 1

Direct retained dependencies

- KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity
- KR21Monoculture.sourceUnitVarianceLaplaceBaseWeakDerivative

KR21Monoculture.theorem7LaplaceMeasure

Declaration location: paper-local

Lean declaration body

type:
```
ℝ → ℝ → MeasureTheory.Measure ℝ
```

value:
```
fun (lam μ : ℝ) =>
  MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (KR21Monoculture.theorem7LaplacePDF lam μ x)
```

Direct retained dependencies

- KR21Monoculture.theorem7LaplacePDF

KR21Monoculture.theorem7LaplacianDefinition2Score1

Declaration location: paper-local

Lean declaration body

type:
```
KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace → ℝ
```

value:
```
fun (ω : KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace) => ω.1
```

Direct retained dependencies

- KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace

KR21Monoculture.theorem7LaplacianDefinition2Score2

Declaration location: paper-local

Lean declaration body

type:
```
KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace → ℝ
```

value:
```
fun (ω : KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace) => ω.2.1
```

Direct retained dependencies

- KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace

KR21Monoculture.theorem7LaplacianDefinition2Score3

Declaration location: paper-local

Lean declaration body

type:
```
KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace → ℝ
```

value:
```
fun (ω : KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace) => ω.2.2
```

Direct retained dependencies

- KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace

KR21Monoculture.theorem7LaplacianPairMeasure

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → MeasureTheory.Measure (ℝ × ℝ)

value:
fun (lam xi xj : ℝ) =>
  (KR21Monoculture.theorem7LaplaceMeasure lam xi).prod (KR21Monoculture.theorem7LaplaceMeasure lam xj)
```

Direct retained dependencies

- KR21Monoculture.theorem7LaplaceMeasure

KR21Monoculture.theorem7LaplacianDefinition2ScoreMeasure

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → MeasureTheory.Measure KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace

value:
fun (lam x1 x2 x3 : ℝ) =>
  (KR21Monoculture.theorem7LaplaceMeasure lam x1).prod (KR21Monoculture.theorem7LaplacianPairMeasure lam x2 x3)
```

Direct retained dependencies

- KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace
- KR21Monoculture.theorem7LaplaceMeasure
- KR21Monoculture.theorem7LaplacianPairMeasure

KR21Monoculture.theorem7LaplacianDefinition2RankingPMF

Declaration location: paper-local

Lean declaration body

```
type:
(lam : ℝ) → ℝ → ℝ → ℝ → 0 < lam → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
fun (lam x1 x2 x3 : ℝ) (hlam : 0 < lam) =>
  KR21Monoculture.rumRankingPMFOfMeasure (KR21Monoculture.theorem7LaplacianDefinition2ScoreMeasure lam x1 x2 x3)
    (KR21Monoculture.rum3RankByScoreFns KR21Monoculture.theorem7LaplacianDefinition2Score1
      KR21Monoculture.theorem7LaplacianDefinition2Score2 KR21Monoculture.theorem7LaplacianDefinition2Score3)
    KR21Monoculture.theorem7LaplacianDefinition2RankingPMF._proof_1
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.Theorem7LaplacianDefinition2ScoreSpace
- KR21Monoculture.rum3RankByScoreFns
- KR21Monoculture.rumRankingPMFOfMeasure
- KR21Monoculture.theorem7LaplacianDefinition2Score1
- KR21Monoculture.theorem7LaplacianDefinition2Score2
- KR21Monoculture.theorem7LaplacianDefinition2Score3
- KR21Monoculture.theorem7LaplacianDefinition2ScoreMeasure

KR21Monoculture.laplaceThreeCandidateRankingLaw

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
```
fun (theta x1 x2 x3 : ℝ) =>
  if htheta : 0 < theta then KR21Monoculture.theorem7LaplacianDefinition2RankingPMF theta x1 x2 x3 htheta
  else PMF.pure KR21Monoculture.rum3Ranking012
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.rum3Ranking012
- KR21Monoculture.theorem7LaplacianDefinition2RankingPMF

KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateRankingLaw

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
```
fun (theta x1 x2 x3 : ℝ) =>
  KR21Monoculture.laplaceThreeCandidateRankingLaw (KR21Monoculture.sourceUnitVarianceLaplaceRate theta) x1 x2 x3
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.laplaceThreeCandidateRankingLaw
- KR21Monoculture.sourceUnitVarianceLaplaceRate

KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.DistributionalAccuracyFamily 1

value:
```
{
  dist := fun (theta : ℝ) (value : KR21Monoculture.ValueProfile 1) =>
    KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateRankingLaw theta (value 0) (value 1) (value 2) }
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.ValueProfile
- KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateRankingLaw

KR21Monoculture.theorem8GaussianDefinition2Score1

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace → ℝ

value:
```
fun (ω : KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace) => ω.1
```

Direct retained dependencies

- KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

KR21Monoculture.theorem8GaussianDefinition2Score2

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace → ℝ

value:
fun (ω : KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace) => ω.2.1

Direct retained dependencies

- KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

KR21Monoculture.theorem8GaussianDefinition2Score3

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace → ℝ

value:
fun (ω : KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace) => ω.2.2

Direct retained dependencies

- KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

KR21Monoculture.theorem8GaussianPairMeasureStd

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → MeasureTheory.Measure (ℝ × ℝ)

value:
fun (σ xi xj : ℝ) => AppliedModelingLib.Probability.independentGaussianPairMeasureWithStd σ xi xj

Direct retained dependencies

- AppliedModelingLib.Probability.independentGaussianPairMeasureWithStd

KR21Monoculture.theorem8GaussianVarianceFromStd

Declaration location: paper-local

Lean declaration body

type:
ℝ → NNReal

value:
fun (σ : ℝ) => AppliedModelingLib.Probability.gaussianVarianceFromStd σ

Direct retained dependencies

- AppliedModelingLib.Probability.gaussianVarianceFromStd

KR21Monoculture.theorem8GaussianDefinition2ScoreMeasureStd

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → ℝ → MeasureTheory.Measure KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace

value:
```
fun (σ x1 x2 x3 : ℝ) =>
  (ProbabilityTheory.gaussianReal x1 (KR21Monoculture.theorem8GaussianVarianceFromStd σ)).prod
    (KR21Monoculture.theorem8GaussianPairMeasureStd σ x2 x3)
```

Direct retained dependencies

- KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace
- KR21Monoculture.theorem8GaussianPairMeasureStd
- KR21Monoculture.theorem8GaussianVarianceFromStd

KR21Monoculture.gaussianThreeCandidateRankingLaw

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)

value:
```
fun (theta x1 x2 x3 : ℝ) =>
  KR21Monoculture.rumRankingPMFOfMeasure
    (KR21Monoculture.theorem8GaussianDefinition2ScoreMeasureStd (1 / theta) x1 x2 x3)
    (KR21Monoculture.rum3RankByScoreFns KR21Monoculture.theorem8GaussianDefinition2Score1
      KR21Monoculture.theorem8GaussianDefinition2Score2 KR21Monoculture.theorem8GaussianDefinition2Score3)
    KR21Monoculture.gaussianThreeCandidateRankingLaw._proof_2
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.Theorem8GaussianDefinition2ScoreSpace
- KR21Monoculture.rum3RankByScoreFns
- KR21Monoculture.rumRankingPMFOfMeasure
- KR21Monoculture.theorem8GaussianDefinition2Score1
- KR21Monoculture.theorem8GaussianDefinition2Score2
- KR21Monoculture.theorem8GaussianDefinition2Score3
- KR21Monoculture.theorem8GaussianDefinition2ScoreMeasureStd

KR21Monoculture.gaussianThreeCandidateDistributionalFamily

Declaration location: paper-local

Lean declaration body

type:
KR21Monoculture.DistributionalAccuracyFamily 1

value:
```
{
  dist := fun (theta : ℝ) (value : KR21Monoculture.ValueProfile 1) =>
    KR21Monoculture.gaussianThreeCandidateRankingLaw theta (value 0) (value 1) (value 2) }
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.ValueProfile
- KR21Monoculture.gaussianThreeCandidateRankingLaw

KR21Monoculture.threeCandidateValueProfile

Declaration location: paper-local

Lean declaration body

type:
ℝ → ℝ → ℝ → AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ

value:
fun (x1 x2 x3 : ℝ) (a : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
  if a = 0 then x1 else if a = 1 then x2 else x3

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.tripleToCandidateMeasurableEquiv

Declaration location: paper-local

Lean declaration body

type:
(α : Type u_1) → [inst : MeasurableSpace α] → (α × α) × α ≃ᵐ (AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → α)

value:
fun (α : Type u_1) [MeasurableSpace α] =>
  MeasurableEquiv.prodAssoc.trans
    (((MeasurableEquiv.refl α).prodCongr (MeasurableEquiv.piFinTwo fun (x : Fin 2) => α).symm).trans
      (MeasurableEquiv.piFinSuccAbove (fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) => α) 0).symm)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

KR21Monoculture.rightTripleToCandidateMeasurableEquiv

Declaration location: paper-local

Lean declaration body

type:
(α : Type u_1) → [inst : MeasurableSpace α] → α × α × α ≃ᵐ (AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → α)

value:
fun (α : Type u_1) [MeasurableSpace α] =>
  MeasurableEquiv.prodAssoc.symm.trans (KR21Monoculture.tripleToCandidateMeasurableEquiv α)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.tripleToCandidateMeasurableEquiv

KR21Monoculture.unitVarianceGumbelScale

Declaration location: paper-local

Lean declaration body

type:
ℝ

value:
√6 / Real.pi

KR21Monoculture.sourceUnitVarianceGumbelMeasure

Declaration location: paper-local

Lean declaration body

type:
ℝ → MeasureTheory.Measure ℝ

value:
fun (location : ℝ) =>
  KR21Monoculture.commonLocationPositiveScaleGumbelMeasure location KR21Monoculture.unitVarianceGumbelScale

Direct retained dependencies

- KR21Monoculture.commonLocationPositiveScaleGumbelMeasure
- KR21Monoculture.unitVarianceGumbelScale

KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → ℝ → MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)

value:
fun {n : ℕ} (location : ℝ) =>
  MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    KR21Monoculture.sourceUnitVarianceGumbelMeasure location

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.sourceUnitVarianceGumbelMeasure

KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  ℝ →
    ℝ →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
        PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

value:
fun {n : ℕ} (location theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure
    (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location)
    (KR21Monoculture.sourceUnitVarianceGumbelRank theta value)
    (KR21Monoculture.measurable_sourceUnitVarianceGumbelRank theta value)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankingPMFOfMeasure
- KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw
- KR21Monoculture.sourceUnitVarianceGumbelRank

KR21Monoculture.DistributionalAccuracyFamily.sourceUnitVarianceGumbelRUMDistributionalFamily

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → ℝ → KR21Monoculture.DistributionalAccuracyFamily n

value:
fun {n : ℕ} (location : ℝ) =>
  {
    dist := fun (theta : ℝ) (value : KR21Monoculture.ValueProfile n) =>
      KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta value }

Direct retained dependencies

- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.ValueProfile
- KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF

KR21Monoculture.w11BaseNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
(ℝ → ℝ) → MeasureTheory.Measure ℝ

value:
fun (f : ℝ → ℝ) => MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (f x)

KR21Monoculture.w11CandidateNoiseLaw

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} → (ℝ → ℝ) → MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)

value:
fun {n : ℕ} (f : ℝ → ℝ) =>
  MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
    KR21Monoculture.w11BaseNoiseLaw f

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.w11BaseNoiseLaw

KR21Monoculture.w11CorrectedScaledNoiseFamily

Declaration location: paper-local

Lean declaration body

type:
{n : ℕ} →
  (f : ℝ → ℝ) →
    ∫⁻ (x : ℝ), ENNReal.ofReal (f x) = 1 →
      (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → KR21Monoculture.AccuracyFamily n

value:
fun {n : ℕ} (f : ℝ → ℝ) (hnormalized : ∫⁻ (x : ℝ), ENNReal.ofReal (f x) = 1)
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  {
    dist := fun (theta : ℝ) =>
      KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value theta,
    value := value }

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
- KR21Monoculture.w11CandidateNoiseLaw

KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → KR21Monoculture.AccuracyFamily 1

value:
fun (x1 x2 x3 : ℝ) =>
  KR21Monoculture.w11CorrectedScaledNoiseFamily KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity
    KR21Monoculture.sourceUnitVarianceLaplace_w11Regularity.normalized
    (KR21Monoculture.threeCandidateValueProfile x1 x2 x3)
```

Direct retained dependencies

- KR21Monoculture.AccuracyFamily
- KR21Monoculture.SourceUnitVarianceLaplaceW11Regularity
- KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity
- KR21Monoculture.threeCandidateValueProfile
- KR21Monoculture.w11CorrectedScaledNoiseFamily

KR21Monoculture.theorem2GaussianScaledNoiseFamily

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → KR21Monoculture.AccuracyFamily 1

value:
fun (x1 x2 x3 : ℝ) =>
  KR21Monoculture.w11CorrectedScaledNoiseFamily KR21Monoculture.theorem2GaussianBaseDensity
    KR21Monoculture.theorem2GaussianScaledNoiseFamily._proof_1 (KR21Monoculture.threeCandidateValueProfile x1 x2 x3)
```

Direct retained dependencies

- KR21Monoculture.AccuracyFamily
- KR21Monoculture.finiteGaussianMixtureDensity
- KR21Monoculture.theorem2GaussianBaseDensity
- KR21Monoculture.threeCandidateValueProfile
- KR21Monoculture.w11CorrectedScaledNoiseFamily

Material library prerequisites

AppliedModelingLib.Probability.StrictlyWellOrderedNoise

Source locator: cited publication, lines 1151-1162

Verbatim paper-source connection

Definition 4. A noise model with density f (·) is well-ordered if for any a > b and c > d,
f (a − c)f (b − d) > f (a − d)f (b − c).

Lean-expanded library semantic target

type:
(ℝ → ℝ) → Prop

value:
fun (f : ℝ → ℝ) => ∀ {{a b c d : ℝ}}, b < a → d < c → f (a - c) * f (b - d) > f (a - d) * f (b - c)

Exact Lean library declaration

```
/--
Strict well-ordering condition for one-dimensional additive noise kernels.

For `a > b` and `c > d`, assigning the larger realized value to the larger true
value is strictly more likely than the crossed assignment.
-/
def StrictlyWellOrderedNoise (f : ℝ → ℝ) : Prop :=
  ∀ {{a b c d : ℝ}}, b < a → d < c →
    f (a - c) * f (b - d) > f (a - d) * f (b - c)
```

Recorded source-to-library screening **Verdict:** matches

Reason: Definition 4 gives the identical strict four-point cross-product inequality for a one-dimensional noise density.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.SocialChoice.Ranking.Candidate

Source locator: cited publication, lines 133-138

Verbatim paper-source connection

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

Lean-expanded library semantic target

type:
ℕ → Type

value:
fun (n : ℕ) => Fin (n + 2)

Exact Lean library declaration

```
/--
Candidate set with at least two candidates. The parameter `n` means there are
`n + 2` candidates.
-/
abbrev Candidate (n : ℕ) := Fin (n + 2)
```

Recorded source-to-library screening

Verdict: matches

Reason: The source’s finite candidate set is represented by the Lean finite candidate type, with the index parameter recording its cardinality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.SocialChoice.Ranking.Ranking

Source locator: cited publication, lines 133-138

Verbatim paper-source connection

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

Lean-expanded library semantic target

type:
ℕ → Type

value:
fun (n : ℕ) => Equiv.Perm (AppliedModelingLib.SocialChoice.Ranking.Candidate n)

Exact Lean library declaration

/-- A ranking is a permutation of the candidate set. -/
abbrev Ranking (n : ℕ) := Equiv.Perm (Candidate n)

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

Recorded source-to-library screening **Verdict:** matches

Reason: The source mechanism outputs a permutation of the candidates, which is exactly the Lean ranking declaration.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy

Source locator: cited publication, lines 133-138

Verbatim paper-source connection

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

Lean-expanded library semantic target

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop

value:
fun {n : ℕ} (ρ : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  ∀ {a b : AppliedModelingLib.SocialChoice.Ranking.Candidate n},
    AppliedModelingLib.SocialChoice.Ranking.rankOf ρ a < AppliedModelingLib.SocialChoice.Ranking.rankOf ρ b →
      value b < value a
```

Exact Lean library declaration

```
/-- A value vector is strictly decreasing down the reference ranking. -/
def StrictlyOrderedBy {n : ℕ} (ρ : Ranking n) (value : Candidate n → ℝ) : Prop :=
  ∀ {a b : Candidate n}, rankOf ρ a < rankOf ρ b → value b < value a
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankOf

Recorded source-to-library screening **Verdict:** matches

Reason: The source's x_1 > x_2 > ... ordering is the same strict ordering of values along the Lean reference ranking.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility

Source locator: cited publication, lines 176-192

Verbatim paper-source connection

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.
As described earlier, each firm can choose to use either a private “human evaluator” or an algorithmically generated ranking as its randomized mechanism R. We assume that both candidate mechanisms come from a noisy permutation family Fθ , with differing values of the accuracy parameter θ: human evaluators all have the same accuracy θH , and the algorithm has accuracy θA . However, while the human evaluator produces a ranking independent of any other firm, the algorithmically generated ranking is identical for all firms who choose to use it. In other words, if two firms choose to use the algorithmically generated ranking, they will both receive the same permutation π.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

Additional assumptions exactly two labelled firms; each firm is first with probability one half; arrival order is independent of values, rankings, and strategy randomization

Lean-expanded library semantic target

```
type:
{n : ℕ} →
  (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
    AppliedModelingLib.SocialChoice.Ranking.Ranking n → AppliedModelingLib.SocialChoice.Ranking.Ranking n → ℝ

value:
fun {n : ℕ} (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (π σ : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
  value
    (AppliedModelingLib.SocialChoice.Ranking.bestRemainingAfter π
      (AppliedModelingLib.SocialChoice.Ranking.firstChoice σ))
```

Exact Lean library declaration

```
/-- Pointwise utility to the second mover when the first mover uses `σ`
and the second mover uses `π`. -/
def secondMoverUtility {n : ℕ} (value : Candidate n → ℝ)
    (π σ : Ranking n) : ℝ :=
  value (bestRemainingAfter π (firstChoice σ))
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.bestRemainingAfter
- AppliedModelingLib.SocialChoice.Ranking.firstChoice

Recorded source-to-library screening **Verdict:** matches

Reason: After the first firm hires its highest-ranked candidate, the other firm selects its own highest-ranked remaining candidate; the Lean utility declaration is exactly that value.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Paper-specific semantic prerequisites

KR21Monoculture.AccuracyFamily

Paper-local declaration:
KR21Monoculture.AccuracyFamily
Source locator: cited publication, lines 133-138

Verbatim paper-source connection

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

Lean-elaborated paper signature

field dist:
{n : ℕ} → KR21Monoculture.AccuracyFamily n → ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

field value:
{n : ℕ} → KR21Monoculture.AccuracyFamily n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

Recorded source-to-declaration screening **Verdict:** matches

Reason: The source’s noisy ranking family sends each candidate-value profile to a distribution on rankings, matching the Lean family interface used by Definition 1.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.ValueProfile

Paper-local declaration:
KR21Monoculture.ValueProfile
Source locator: cited publication, lines 190-198

Verbatim paper-source connection

rational behavior depends not only on the accuracy of the ranking mechanism, but also on the underlying candidate values x1 , . . . , xn . Thus, to fully specify a firm's behavior, we assume that x1 , . . . , xn are drawn from a known joint distribution D. Our main results will hold for any D, meaning they apply even when the candidate values (but not their identities) are deterministically known.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

[Next verbatim source excerpt]

The choice of which ranking mechanism to use leads to a game-theoretic setting: both firms know the accuracy parameters of the human evaluators (θH) and the algorithm (θA), and they must decide whether to
3

[form-feed in source extraction]
use a human evaluator or the algorithm. This choice introduces a subtlety: for many ranking models, a firm's rational behavior depends not only on the accuracy of the ranking mechanism, but also on the underlying candidate values x1 , . . . , xn . Thus, to fully specify a firm's behavior, we assume that x1 , . . . , xn are drawn from a known joint distribution D. Our main results will hold for any D, meaning they apply even when the candidate values (but not their identities) are deterministically known.

Additional assumptions rank-labelled almost-everywhere strict source order; coordinatewise finite first moments; measurable finite ranking atoms; positive Definition 2 disagreement mass

Lean-expanded paper semantic target

type:
ℕ → Type

value:
fun (n : ℕ) => AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate

Recorded source-to-declaration screening **Verdict:** matches

Reason: The source's x_i are the intrinsic utilities from hiring candidates i, and the Lean value profile is the corresponding candidate-to-real map.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.DistributionalAccuracyFamily

Paper-local declaration:
KR21Monoculture.DistributionalAccuracyFamily
Source locator: cited publication, lines 190-198

Verbatim paper-source connection

rational behavior depends not only on the accuracy of the ranking mechanism, but also on the underlying candidate values x1 , . . . , xn . Thus, to fully specify a firm's behavior, we assume that x1 , . . . , xn are drawn from a known joint distribution D. Our main results will hold for any D, meaning they apply even when the candidate values (but not their identities) are deterministically known.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

[Next verbatim source excerpt]

The choice of which ranking mechanism to use leads to a game-theoretic setting: both firms know the accuracy parameters of the human evaluators (θH) and the algorithm (θA), and they must decide whether to
3

[form-feed in source extraction]
use a human evaluator or the algorithm. This choice introduces a subtlety: for many ranking models, a firm's rational behavior depends not only on the accuracy of the ranking mechanism, but also on the underlying candidate values x1 , . . . , xn . Thus, to fully specify a firm's behavior, we assume that x1 , . . . , xn are drawn from a known joint distribution D. Our main results will hold for any D, meaning they apply even when the candidate values (but not their identities) are deterministically known.

Additional assumptions rank-labelled almost-everywhere strict source order; coordinatewise finite first moments; measurable finite ranking atoms; positive Definition 2 disagreement mass

Lean-elaborated paper signature

```
field dist:
{n : ℕ} →
  KR21Monoculture.DistributionalAccuracyFamily n →
    ℝ → KR21Monoculture.ValueProfile n → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.ValueProfile

Recorded source-to-declaration screening **Verdict:** matches

Reason: The source separately samples a candidate-value profile from D and independently draws rankings from the declared family; the Lean declaration records that outer distributional experiment.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.Model

Paper-local declaration:
KR21Monoculture.Model
Source locator: cited publication, lines 176-192

Verbatim paper-source connection

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.
As described earlier, each firm can choose to use either a private "human evaluator" or an algorithmically generated ranking as its randomized mechanism R. We assume that both candidate mechanisms come from a noisy permutation family Fθ , with differing values of the accuracy parameter θ: human evaluators all have the same accuracy θH , and the algorithm has accuracy θA . However, while the human evaluator produces a ranking independent of any other firm, the algorithmically generated ranking is identical for all firms who choose to use it. In other words, if two firms choose to use the algorithmically generated ranking, they will both receive the same permutation π.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

Additional assumptions exactly two labelled firms; each firm is first with probability one half; arrival order is independent of values, rankings, and strategy randomization

Lean-elaborated paper signature

field algorithmRanking:
{n : ℕ} → KR21Monoculture.Model n → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

field humanRanking:
{n : ℕ} → KR21Monoculture.Model n → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)

field value:
{n : ℕ} → KR21Monoculture.Model n → AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

Recorded source-to-declaration screening **Verdict:** matches
Reason: The Lean model has the source's two labelled firms, common candidates, rankings, sequential selection, and the recorded approved arrival-order convention.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.PaperInterface.section31_iidFinitePositiveVariance_normalizationSpec

Paper-local declaration:
KR21Monoculture.PaperInterface.section31_iidFinitePositiveVariance_normalizationSpec
Source locator: cited publication, lines 493-504

Verbatim paper-source connection

First, we must define a family of RUMs that satisfies the conditions of Definition 1. Assume without loss of generality that the noise distribution E has unit variance. Then, consider the family of RUMs parameterized by θ in which candidates are ranked according to xi + εθi . By this definition, the standard deviation of the noise for a particular value of θ is simply 1/θ. Intuitively, larger values of θ reduce the effect of the noise,

[Next verbatim source excerpt]

In Random Utility Models, the underlying candidate values xi are perturbed by independent and identically distributed noise εi ∼ E, and the perturbed values are ranked to produce π. Originally conceived in the psychology literature [23], this model has been well-studied over nearly a century, [7, 4, 9, 24, 16, 22], including more recently in the computer science and machine learning literature [2, 1, 19, 25, 13].
First, we must define a family of RUMs that satisfies the conditions of Definition 1. Assume without loss of generality that the noise distribution E has unit variance. Then, consider the family of RUMs parameterized by θ in which candidates are ranked according to xi + εθi . By this definition, the standard deviation of the noise for a particular value of θ is simply 1/θ. Intuitively, larger values of θ reduce the effect of the noise, making the ranking more accurate. In Theorem 5 in Appendix A, we show as long as the noise distribution E has positive support on (−∞, ∞), this definition of Fθ meets the differentiability, asymptotic optimality, and monotonicity conditions in Definition 1. For distributions with finite support, many of our results can be generalized by relaxing strict inequalities in Definition 1 and Theorem 1 to weak inequalities.

Lean-expanded paper semantic target

type:
```
{n : ℕ} →
  (E : MeasureTheory.Measure ℝ) →
    [MeasureTheory.IsProbabilityMeasure E] →
      MeasureTheory.MemLp id 2 E →
        0 < ProbabilityTheory.variance id E →
          (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → (theta : ℝ) → 0 < theta → Prop
```

value:
```
fun {n : ℕ} (E : MeasureTheory.Measure ℝ) [MeasureTheory.IsProbabilityMeasure E] (hsecond : MeasureTheory.MemLp id 2 E)
    (hvariance_pos : 0 < ProbabilityTheory.variance id E)
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) (theta : ℝ) (htheta : 0 < theta) =>
  ProbabilityTheory.iIndepFun
      (fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n)
         (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
        epsilon i)
      (KR21Monoculture.sourceRUMNormalizedIIDNoiseLaw E √(ProbabilityTheory.variance id E)) ∧
    MeasureTheory.MemLp id 2 (KR21Monoculture.sourceRUMNormalizedScalarNoiseLaw E √(ProbabilityTheory.variance id E)) ∧
      (∀ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
         ProbabilityTheory.variance
             (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) => epsilon i)
             (KR21Monoculture.sourceRUMNormalizedIIDNoiseLaw E √(ProbabilityTheory.variance id E)) =
           1) ∧
        KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
          (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => E) value theta =
        KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
          (KR21Monoculture.sourceRUMNormalizedIIDNoiseLaw E √(ProbabilityTheory.variance id E)) value
          (theta / √(ProbabilityTheory.variance id E))
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
- KR21Monoculture.sourceRUMNormalizedIIDNoiseLaw
- KR21Monoculture.sourceRUMNormalizedScalarNoiseLaw

Recorded source-to-declaration screening **Verdict:** matches

Reason: Section 3.1 specifies independent, identically distributed positive-variance noise scaled by positive accuracy; the Lean target states that complete normalization and induced ranking law.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.expectedBestInSet

Paper-local declaration:
KR21Monoculture.expectedBestInSet
Source locator: cited publication, lines 139-165

Verbatim paper-source connection

Definition 1 (Noisy permutation family). A noisy permutation family Fθ is a family of distributions over permutations that satisfies the following conditions for any θ > 0 and set of candidates x:
1. (Differentiability) For any permutation π, PrFθ [π] is continuous and differentiable in θ.
2. (Asymptotic optimality) For the true ranking π ∗ , limθ→∞ PrFθ [π ∗] = 1.
3. (Monotonicity) For any (possibly empty) S ⊂ x, let π (−S) be the partial ranking produced by removing
(−S)
the items in S from π. Let π1
denote the value of the top-ranked candidate according to π (−S) . For
0
any θ > θ,
h
i
h
i
(−S)
(−S)
EFθ0 π1
≥ EFθ π1
.
(1)
Moreover, for S = ∅, (1) holds with strict inequality.

Lean-expanded paper semantic target

```
type:
{n : ℕ} →
  PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
    (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) →
      Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n) → ℝ

value:
fun {n : ℕ} (μ : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n))
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
    (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)) =>
  AppliedModelingLib.pmfExp μ fun (π : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
    value (KR21Monoculture.bestInSet π remaining)
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.pmfExp
- KR21Monoculture.bestInSet

Recorded source-to-declaration screening **Verdict:** matches

Reason: The source payoff is the expected value of the highest-ranked available candidate; the Lean declaration takes the same best element under the given ranking and averages its value.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.PaperInterface.source_definition1_iffSpec

Paper-local declaration:
KR21Monoculture.PaperInterface.source_definition1_iffSpec
Source locator: cited publication, lines 139-165

Verbatim paper-source connection

Definition 1 (Noisy permutation family). A noisy permutation family Fθ is a family of distributions over
permutations that satisfies the following conditions for any θ > 0 and set of candidates x:
1. (Differentiability) For any permutation π, PrFθ [π] is continuous and differentiable in θ.
2. (Asymptotic optimality) For the true ranking π ∗ , limθ→∞ PrFθ [π ∗] = 1.
3. (Monotonicity) For any (possibly empty) S ⊂ x, let π (−S) be the partial ranking produced by removing
(−S)
the items in S from π. Let π1
denote the value of the top-ranked candidate according to π (−S) . For
0
any θ > θ,
h
i
h
i
(−S)
(−S)
EFθ0 π1
≥ EFθ π1
.
(1)
Moreover, for S = ∅, (1) holds with strict inequality.

Lean-expanded paper semantic target

```
type:
{n : ℕ} →
  (F : KR21Monoculture.AccuracyFamily n) →
    (trueRanking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) →
      AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy trueRanking F.value → Prop

value:
fun {n : ℕ} (F : KR21Monoculture.AccuracyFamily n) (trueRanking : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (hvalueOrder : AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy trueRanking F.value) =>
  KR21Monoculture.SourceDefinition1 F trueRanking ↔
    (∀ (theta : ℝ),
       0 < theta →
         ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
           ContinuousAt (fun (theta' : ℝ) => ((F.dist theta') pi).toReal) theta ∧
             DifferentiableAt ℝ (fun (theta' : ℝ) => ((F.dist theta') pi).toReal) theta) ∧
      Filter.Tendsto (fun (theta : ℝ) => ((F.dist theta) trueRanking).toReal) Filter.atTop (nhds 1) ∧
        ∀ (thetaA thetaH : ℝ),
          0 < thetaH →
            thetaH < thetaA →
              (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                 remaining.Nonempty →
                   KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value remaining ≤
                     KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value remaining) ∧
                KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value Finset.univ <
                  KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value Finset.univ
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.SourceDefinition1
- KR21Monoculture.expectedBestInSet

Recorded source-to-declaration screening **Verdict:** matches

Reason: Definition 1’s ranking distribution, shared-value ordering, and removal-monotonicity condition are the components exposed by the transparent Lean specification.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.PaperInterface.source_definition2_iffSpec

Paper-local declaration:
KR21Monoculture.PaperInterface.source_definition2_iffSpec
Source locator: cited publication, lines 210-216

Verbatim paper-source connection

Definition 2 (Preference for the first position.). A candidate distribution D and noisy permutation family
Fθ exhibits a preference for the first position if for all θ > 0, if π, σ ∼ Fθ ,
E [π1 − π2 | π1 6= σ1] > 0.

[Next verbatim source excerpt]

Definition 2 (Preference for the first position.). A candidate distribution D and noisy permutation family
Fθ exhibits a preference for the first position if for all θ > 0, if π, σ ∼ Fθ ,
E [π1 − π2 | π1 6= σ1] > 0.

In other words, for any θ > 0, suppose we draw two permutations π and σ independently from Fθ , and
suppose that the first-ranked candidates differ in π and σ. Then the expected value of the first-ranked
candidate in π is strictly greater than the expected value of the second-ranked candidate in π.

Lean-expanded paper semantic target

type:
```
{n : ℕ} → KR21Monoculture.DistributionalAccuracyFamily n → MeasureTheory.Measure (KR21Monoculture.ValueProfile n) → Prop
```

value:
```
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) =>
  KR21Monoculture.SourceDefinition2 F D ↔
    ∀ (theta : ℝ),
      0 < theta →
        ∀
          (hatom :
            ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
              Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking),
          have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
            F.outerIndependentPairJointLaw D theta hatom;
          have numerator : ℝ :=
            ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
              if
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                    AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                  x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
              else 0 ∂J;
          have denominator : ℝ :=
            ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
              if
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                    AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                1
              else 0 ∂J;
          0 < denominator → 0 < numerator / denominator
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.SourceDefinition2
- KR21Monoculture.ValueProfile

Recorded source-to-declaration screening **Verdict:** matches

Reason: Definition 2 quantifies over positive accuracy and admissible ranking laws, then compares the literal conditional first-minus-second payoff under the induced independent experiment; the Lean target does the same.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.PaperInterface.source_definition3_iffSpec

Paper-local declaration:
KR21Monoculture.PaperInterface.source_definition3_iffSpec
Source locator: cited publication, lines 217-242

Verbatim paper-source connection

Definition 3 (Preference for weaker competition.). A candidate distribution D and noisy permutation family
Fθ , exhibits a preference for weaker competition if the following holds: for all θ1 > θ2 , σ ∼ Fθ1 and
π, τ ∼ Fθ2 ,
h
i
h
i
(−{σ1 })

E π1

(−{τ1 })

< E π1

.

Lean-expanded paper semantic target

type:
{n : ℕ} → KR21Monoculture.DistributionalAccuracyFamily n → MeasureTheory.Measure (KR21Monoculture.ValueProfile n) → Prop

value:
```
fun {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
    (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) =>
  KR21Monoculture.SourceDefinition3 F D ↔
    ∀ (thetaA thetaH : ℝ),
      0 < thetaH →
        thetaH < thetaA →
          ∫ (value : KR21Monoculture.ValueProfile n),
              AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaA value)
                fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D <
            ∫ (value : KR21Monoculture.ValueProfile n),
              AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaH value)
                fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfPairExp
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.SourceDefinition3
- KR21Monoculture.ValueProfile

Recorded source-to-declaration screening **Verdict:** matches

Reason: Definition 3 compares the second mover's expected remaining-candidate value at every ordered pair of positive accuracies; the Lean target states the two source experiments and strict inequality directly.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.SourceCandidateValueOrder

Paper-local declaration:
KR21Monoculture.SourceCandidateValueOrder
Source locator: cited publication, lines 133-138

Verbatim paper-source connection

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

Lean-expanded paper semantic target

```
type:
{n : ℕ} →
  AppliedModelingLib.SocialChoice.Ranking.Ranking n → (AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) → Prop

value:
fun {n : ℕ} (trueRanking : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
    (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy trueRanking value
```

Direct retained dependencies

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy

Recorded source-to-declaration screening **Verdict:** matches

Reason: The source assumes intrinsically valued candidates ordered from best to worst; the Lean declaration is precisely that strict value-order relation.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.Strategy

Paper-local declaration:
KR21Monoculture.Strategy
Source locator: cited publication, lines 176-192

Verbatim paper-source connection

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.
As described earlier, each firm can choose to use either a private "human evaluator" or an algorithmically generated ranking as its randomized mechanism R. We assume that both candidate mechanisms come from a noisy permutation family Fθ , with differing values of the accuracy parameter θ: human evaluators all have the same accuracy θH , and the algorithm has accuracy θA . However, while the human evaluator produces a ranking independent of any other firm, the algorithmically generated ranking is identical for all firms who choose to use it. In other words, if two firms choose to use the algorithmically generated ranking, they will both receive the same permutation π.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

Additional assumptions exactly two labelled firms; each firm is first with probability one half; arrival order is independent of values, rankings, and strategy randomization

Lean-elaborated paper signature

type:
Type

constructor KR21Monoculture.Strategy.algorithm:
KR21Monoculture.Strategy

constructor KR21Monoculture.Strategy.human:
KR21Monoculture.Strategy

Recorded source-to-declaration screening **Verdict:** matches
Reason: The source permits each firm to choose a human evaluator or the shared algorithmic ranking; the Lean strategy type represents exactly those choices.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

KR21Monoculture.theorem8Erf

Paper-local declaration:
KR21Monoculture.theorem8Erf
Source locator: cited publication, lines 1815-1836

Verbatim paper-source connection

Ra
√
exp(−(x − xi)2)/ π · (1 + erf(x − xj))/2 dx
−∞
=
(1 + erf(a − xi))/2 · (1 + erf(a − xj))/2
Ra
exp(−(x − xi)2)(1 + erf(x − xj)) dx
2
= √ −∞
(1 + erf(a − xi)) · (1 + erf(a − xj))
π
The derivative with respect to a is positive if and only if
(1 + erf(a − xi))(1 + erf(a − xj)) exp(−(a − xi)2)(1 + erf(a − xj))
Z a
>
exp(−(x − xi)2)(1 + erf(x − xj)) dx
−∞

[U+0001 control character]
2
· √ (1 + erf(a − xi)) exp(−(a − xj)2) + (1 + erf(a − xj)) exp(−(a − xi)2)

Lean-expanded paper semantic target

type:
ℝ → ℝ

value:
fun (t : ℝ) => 2 / √Real.pi * ∫ (x : ℝ) in 0..t, Real.exp (-x ^ 2)

Recorded source-to-declaration screening **Verdict:** matches

Reason: Appendix C uses unqualified standard ‘erf‘ in normalized Gaussian CDF and derivative formulas; its standard normalization is exactly the Lean integral definition.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 143-159

Verbatim source input

0
any θ > θ,
h
i
h
i
(−S)
(−S)
EFθ0 π1
≥ EFθ π1
.
(1)

Expanded PaperInterface specification

```
∀ {n : ℕ} {F : KR21Monoculture.AccuracyFamily n} {trueRanking : AppliedModelingLib.SocialChoice.Ranking.Ranking n},
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy trueRanking F.value →
    KR21Monoculture.SourceDefinition1 F trueRanking →
      ∀ (thetaA thetaH : ℝ),
        0 < thetaH →
          thetaH < thetaA →
            (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                remaining.Nonempty →
                  KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value remaining ≤
                    KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value remaining) ∧
              KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value Finset.univ <
                KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value Finset.univ
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.SourceDefinition1
- KR21Monoculture.expectedBestInSet

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: KR21Monoculture.AccuracyFamily n
3. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
4. assumption: AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy trueRanking F.value
5. assumption: KR21Monoculture.SourceDefinition1 F trueRanking
6. parameter: ℝ
7. parameter: ℝ
8. assumption: 0 < thetaH
9. assumption: thetaH < thetaA
10. conclusion: (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
    remaining.Nonempty →
      KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value remaining ≤
        KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value remaining) ∧
  KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value Finset.univ <
    KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value Finset.univ

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.source_equation1_removal_monotonicity

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target is Equation (1): weak monotonicity for every nonempty remaining set and strict inequality for the full candidate set.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 244-246

Verbatim source input

Theorem 1. Suppose that a given candidate distribution D and noisy permutation family Fθ satisfy Definition 2 (preference for the first position) and
↪ Definition 3 (preference for weaker competition).
Then, for any θH , there exists θA > θH such that using the algorithmic ranking is a strictly dominant strategy for both firms, but social welfare would be higher if both firms used human evaluators.

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

Proving Theorem 1

With this intuition, we are ready to prove Theorem 1.
Proof of Theorem 1. For given values of θA and θH , using the algorithmic ranking is a strictly dominant strategy as long as
UA (θA , θH) + UAA (θA , θH) > UH (θA , θH) + UAH (θA , θH)

(4)

UA (θA , θH) + UHA (θA , θH) > UH (θA , θH) + UHH (θA , θH)

(5)

Note that (5) is always true for θA > θH by the monotonicity assumption on Fθ : UA (θA , θH) ≥ UH (θA , θH) because a more accurate ranking produces a top-ranked candidate with higher expected value, and UHA (θA , θH) ≥ UHH (θA , θH) because this holds even conditioned on removing any candidate from the pool (in this case, the
5

[form-feed in source extraction]
candidate randomly selected by the firm that hires first). Crucially, in (5), the first firm's random selection is independent from the second firm's selection; the same logic could not be used to argue that (4) always holds for θA ≥ θH . Moreover, when θA > θH , UA (θA , θH) > UH (θA , θH) by the monotonicity assumption, meaning (5) holds.
Let Ws1 s2 (θA , θH) denote social welfare when the two firms employ strategies s1 , s2 ∈ {A, H}. Then, when both firms use the algorithmic ranking, social welfare is
WAA (θA , θH) = UA (θA , θH) + UAA (θA , θH).
By (2), Definition 2 implies that for any θ, UAA (θ, θ) < UAH (θ, θ), implying
UA (θH , θH) + UAA (θH , θH) < UH (θH , θH) + UAH (θH , θH).
However, by the optimality assumption on Fθ in Definition 1, for sufficiently large θ̂A ,
UA (θ̂A , θH) + UAA (θ̂A , θH) > UH (θ̂A , θH) + UAH (θ̂A , θH).
Note that Us1 (θA , θH) and Us1 s2 (θA , θH) are continuous with respect to θA for any s1 , s2 ∈ {A, H} since they are expectations over discrete distributions with probabilities that are by assumption differentiable with
∗
> θH
respect to θA . Therefore, by the Differentiability assumption on Fθ from Definition 1, there is some θA such that
∗
∗
∗
∗
UA (θA
, θH) + UAA (θA
, θH) = UH (θA
, θH) + UAH (θA
, θH),
(6)
i.e., given that its competitor uses the algorithmic ranking, a firm is indifferent between the two strategies.
∗
, using the algorithmic ranking is still a weakly dominant strategy. By definition of WAA ,
For such θA
∗
∗
∗
WAA (θA
, θH) = UH (θA
, θH) + UAH (θA
, θH).

If both firms had instead used human evaluators, social welfare would be
∗
∗
∗
WHH (θA
, θH) = UH (θA
, θH) + UHH (θA
, θH).

By Definition 3, for σ ∼ FθA∗ and π, τ ∼ FθH ,
h
i
h
i
(−{σ })

(−{τ })
E π1 1
< E π1 1 .
Note that
h
i
(−{σ })
∗
UAH (θA
, θH) = E π1 1
h
i
(−{τ })
∗
UHH (θA
, θH) = E π1 1
∗
∗
, θH). As a result for θA∗ > θH , using
Thus, Definition 3 implies that for θA∗ > θH , UHH (θA
, θH) > UAH (θA
the algorithmic ranking is a weakly dominant strategy, but
∗
∗
∗
WHH (θA
, θH) = UH (θA
, θH) + UHH (θA
, θH)
∗
∗
> UH (θA
, θH) + UAH (θA
, θH)
∗
∗
= UA (θA
, θH) + UAA (θA
, θH)
∗
= WAA (θA
, θH),

meaning social welfare would have been higher had both firms used human evaluators.
0
We can show that this effect persists for a value θA
such that using the algorithmic ranking is a strictly
∗
dominant strategy. Intuitively, this is simply by slightly increasing θA
so the algorithmic ranking is strictly
dominant. For fixed θH , define
f (θA) = UA (θA , θH) + UAA (θA , θH)
g(θA) = UH (θA , θH) + UAH (θA , θH)
h(θA) = UH (θA , θH) + UHH (θA , θH)
6

[form-feed in source extraction]
0
0
0
Because (5) always holds for θA > θH , it suffices to show that there exists θA
such that g(θA
) < f (θA
)<
0
0
0
0
0
0
h(θA). This is because g(θA) < f (θA) is equivalent to (4) and f (θA) < h(θA) is equivalent to WAA (θA , θH) <
0
WHH (θA
, θH).
First, note that h(θA) is a constant, and by Definition 3, g(θA) < h(θA) for all θA > θH . By the
optimality assumption of Definition 1, there exists sufficiently large θ̂A such that f (θ̂A) > g(θ̂A). Recall
∗
∗
∗
that by definition of θA
, f (θA
) = g(θA
). Both f and g are continuous by the Differentiability assumption in
0
∗
0
0
0
Definition 1. Thus, there must exist some θA
> θA
such that g(θA
) < f (θA
) < h(θA
). This means that for
0
θA , using the algorithmic ranking is a strictly dominant strategy, but social welfare would still be larger if
both firms used human evaluators.

Expanded PaperInterface specification

```
∀ {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
  (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)),
  MeasureTheory.IsProbabilityMeasure D →
    ∀ (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (thetaH : ℝ),
      0 < thetaH →
        (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
           MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D) →
         (∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
            MeasureTheory.AEStronglyMeasurable
              (fun (value : KR21Monoculture.ValueProfile n) => ((F.dist theta value) pi).toReal) D) →
          (∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
             0 < theta →
               ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                 ContinuousAt (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta) →
           (∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
              0 < theta →
                ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                  DifferentiableAt ℝ (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta) →
            (∀ (value : KR21Monoculture.ValueProfile n),
               Filter.Tendsto (fun (theta : ℝ) => ((F.dist theta value) center).toReal) Filter.atTop (nhds 1)) →
             (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
                AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value) →
               ∀
                 (hatom_measurable :
                    ∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                      Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) pi),
                 (∀ (theta : ℝ),
                    0 < theta →
                      MeasureTheory.Integrable
                        (fun (value : KR21Monoculture.ValueProfile n) =>
                          AppliedModelingLib.pmfPairExp (F.dist theta value) (F.dist theta value)
                            fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                            if
                                AppliedModelingLib.SocialChoice.Ranking.firstChoice pi ≠
                                  AppliedModelingLib.SocialChoice.Ranking.firstChoice sigma then
                              1
                            else 0)
                        D) →
                   (∀ (theta : ℝ),
                      0 < theta →
                        MeasureTheory.Integrable
                          (fun (value : KR21Monoculture.ValueProfile n) =>
                            AppliedModelingLib.pmfExp (F.dist theta value)
                              fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                              value (AppliedModelingLib.SocialChoice.Ranking.secondChoice pi))
                          D) →
                     (∀ (theta : ℝ),
                        0 < theta →
                          MeasureTheory.Integrable
                            (fun (value : KR21Monoculture.ValueProfile n) =>
                              AppliedModelingLib.pmfPairExp (F.dist theta value) (F.dist theta value)
                                fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                                AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma)
                            D) →
                       (∀ (theta : ℝ),
                          0 < theta →
                            MeasureTheory.Integrable
                              (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
                                AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1)
                              (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))) →
                         (∀ (theta : ℝ),
                            0 < theta →
                              MeasureTheory.Integrable
                                (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
                                  AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2)
                                (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))) →
                           (∀ (theta : ℝ),
                              0 < theta →
                                0 <
                                  ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                                    if
                                        AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                                          AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                                      1
                                    else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)) →
                             (∀ (theta : ℝ),
                                0 < theta →
                                  0 <
                                    (∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                                        if
                                            AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                                              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                                          x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                                            x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
                                        else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)) /
                                      ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                                        if
                                            AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                                              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                                          1
                                        else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)) →
                               (∀ (thetaA thetaH : ℝ),
                                  0 < thetaH →
                                    thetaH < thetaA →
                                      MeasureTheory.Integrable
                                        (fun (value : KR21Monoculture.ValueProfile n) =>
                                          AppliedModelingLib.pmfPairExp (F.dist thetaH value)
                                            (F.dist thetaA value)
```

```
          fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi
            sigma)
        D) →
(∀ (thetaA thetaH : ℝ),
   0 < thetaH →
    thetaH < thetaA →
     MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
       AppliedModelingLib.pmfPairExp (F.dist thetaH value)
        (F.dist thetaH value)
        fun
          (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
        AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi
          sigma)
      D) →
 (∀ (thetaA thetaH : ℝ),
    0 < thetaH →
     thetaH < thetaA →
      ∫ (value : KR21Monoculture.ValueProfile n),
         AppliedModelingLib.pmfPairExp (F.dist thetaH value)
          (F.dist thetaA value)
          fun
            (pi sigma :
              AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value
            pi sigma ∂D <
       ∫ (value : KR21Monoculture.ValueProfile n),
         AppliedModelingLib.pmfPairExp (F.dist thetaH value)
          (F.dist thetaH value)
          fun
            (pi sigma :
              AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value
            pi sigma ∂D) →
  (∀ (thetaA thetaH : ℝ),
     0 < thetaH →
      thetaH < thetaA →
       ∀ (value : KR21Monoculture.ValueProfile n)
        (remaining :
          Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
       remaining.Nonempty →
        KR21Monoculture.expectedBestInSet (F.dist thetaH value) value
          remaining ≤
         KR21Monoculture.expectedBestInSet (F.dist thetaA value) value
          remaining) →
   (∀ (thetaA thetaH : ℝ),
      0 < thetaH →
       thetaH < thetaA →
        ∀ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
         AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center
           value →
          KR21Monoculture.expectedBestInSet (F.dist thetaH value) value
            Finset.univ <
           KR21Monoculture.expectedBestInSet (F.dist thetaA value) value
            Finset.univ) →
      ∃ (thetaA : ℝ),
       thetaH < thetaA ∧
        ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
             KR21Monoculture.Strategy.human ∂D +
           ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
             KR21Monoculture.Strategy.algorithm
             KR21Monoculture.Strategy.human ∂D <
          ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
             KR21Monoculture.Strategy.algorithm ∂D +
           ∫ (value : KR21Monoculture.ValueProfile n),
            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
             KR21Monoculture.Strategy.algorithm
             KR21Monoculture.Strategy.algorithm ∂D ∧
         ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
              KR21Monoculture.Strategy.human ∂D +
            ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
              KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.human ∂D <
           ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
              KR21Monoculture.Strategy.algorithm ∂D +
            ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
              KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.algorithm ∂D ∧
          ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
              KR21Monoculture.Strategy.algorithm ∂D +
            ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
              KR21Monoculture.Strategy.algorithm
              KR21Monoculture.Strategy.algorithm ∂D <
           ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
              KR21Monoculture.Strategy.human ∂D +
            ∫ (value : KR21Monoculture.ValueProfile n),
             ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
              KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.human ∂D
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfExp
- AppliedModelingLib.pmfPairExp
- KR21Monoculture.AccuracyFamily.modelAt
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.DistributionalAccuracyFamily.pointFamily
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.RankingPair
- KR21Monoculture.Strategy
- KR21Monoculture.ValueProfile
- KR21Monoculture.expectedBestInSet

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: KR21Monoculture.DistributionalAccuracyFamily n
3. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
4. assumption: MeasureTheory.IsProbabilityMeasure D
5. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
6. parameter: ℝ
7. assumption: 0 < thetaH
8. assumption: ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D
9. assumption: ∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  MeasureTheory.AEStronglyMeasurable (fun (value : KR21Monoculture.ValueProfile n) => ((F.dist theta value) pi).toReal)
    D
10. assumption: ∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
  0 < theta →
    ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
      ContinuousAt (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta
11. assumption: ∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
  0 < theta →
    ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
      DifferentiableAt ℝ (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta
12. assumption: ∀ (value : KR21Monoculture.ValueProfile n),
  Filter.Tendsto (fun (theta : ℝ) => ((F.dist theta value) center).toReal) Filter.atTop (nhds 1)
13. assumption: ∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
14. assumption: ∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) pi
15. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.pmfPairExp (F.dist theta value) (F.dist theta value)
          fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
          if
              AppliedModelingLib.SocialChoice.Ranking.firstChoice pi ≠
                AppliedModelingLib.SocialChoice.Ranking.firstChoice sigma then
            1
          else 0)
      D
16. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.pmfExp (F.dist theta value) fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
          value (AppliedModelingLib.SocialChoice.Ranking.secondChoice pi))
      D
17. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.pmfPairExp (F.dist theta value) (F.dist theta value)
```

```
      fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
      AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma)
    D
18. assumption: ∀ (theta : ℝ),
  0 < theta →
   MeasureTheory.Integrable
    (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
     AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1)
    (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))
19. assumption: ∀ (theta : ℝ),
  0 < theta →
   MeasureTheory.Integrable
    (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
     AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2)
    (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))
20. assumption: ∀ (theta : ℝ),
  0 < theta →
   0 <
    ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
     if
       AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
         AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
       1
     else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)
21. assumption: ∀ (theta : ℝ),
  0 < theta →
   0 <
    (∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
       if
         AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
           AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
         x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
           x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
       else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)) /
     ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
       if
         AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
           AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
         1
       else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)
22. assumption: ∀ (thetaA thetaH : ℝ),
  0 < thetaH →
   thetaH < thetaA →
    MeasureTheory.Integrable
     (fun (value : KR21Monoculture.ValueProfile n) =>
      AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaA value)
       fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
       AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma)
     D
23. assumption: ∀ (thetaA thetaH : ℝ),
  0 < thetaH →
   thetaH < thetaA →
    MeasureTheory.Integrable
     (fun (value : KR21Monoculture.ValueProfile n) =>
      AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaH value)
       fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
       AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma)
     D
24. assumption: ∀ (thetaA thetaH : ℝ),
  0 < thetaH →
   thetaH < thetaA →
    ∫ (value : KR21Monoculture.ValueProfile n),
      AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaA value)
       fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
       AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D <
     ∫ (value : KR21Monoculture.ValueProfile n),
      AppliedModelingLib.pmfPairExp (F.dist thetaH value) (F.dist thetaH value)
       fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
       AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D
25. assumption: ∀ (thetaA thetaH : ℝ),
  0 < thetaH →
   thetaH < thetaA →
    ∀ (value : KR21Monoculture.ValueProfile n)
     (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
     remaining.Nonempty →
      KR21Monoculture.expectedBestInSet (F.dist thetaH value) value remaining ≤
       KR21Monoculture.expectedBestInSet (F.dist thetaA value) value remaining
26. assumption: ∀ (thetaA thetaH : ℝ),
  0 < thetaH →
   thetaH < thetaA →
    ∀ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
     AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value →
      KR21Monoculture.expectedBestInSet (F.dist thetaH value) value Finset.univ <
       KR21Monoculture.expectedBestInSet (F.dist thetaA value) value Finset.univ
27. conclusion: ∃ (thetaA : ℝ),
  thetaH < thetaA ∧
   ∫ (value : KR21Monoculture.ValueProfile n),
       ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
     ∫ (value : KR21Monoculture.ValueProfile n),
       ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
         KR21Monoculture.Strategy.human ∂D <
    ∫ (value : KR21Monoculture.ValueProfile n),
       ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
     ∫ (value : KR21Monoculture.ValueProfile n),
       ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
         KR21Monoculture.Strategy.algorithm ∂D ∧
   ∫ (value : KR21Monoculture.ValueProfile n),
       ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
     ∫ (value : KR21Monoculture.ValueProfile n),
       ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
```

```
      KR21Monoculture.Strategy.human ∂D <
  ∫ (value : KR21Monoculture.ValueProfile n),
      ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
        KR21Monoculture.Strategy.algorithm ∂D ∧
∫ (value : KR21Monoculture.ValueProfile n),
      ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
        KR21Monoculture.Strategy.algorithm ∂D <
  ∫ (value : KR21Monoculture.ValueProfile n),
      ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
        KR21Monoculture.Strategy.human ∂D
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem1_raw_outer_literal_payoff_terminal_conclusion

Recorded source-to-Spec screening **Verdict:** matches

Reason: Compared source Theorem 1 and proof inequalities (4)-(5) with the expanded target and displayed payoff declarations. The target chooses one thetaA > thetaH and states exactly UA + UAA > UH + UAH, UA + UHA > UH + UHH, and UA + UAA < UH + UHH after expectation over D. modelAt assigns the algorithm and human laws at thetaA and thetaH; firstMoverEU and secondMoverEU encode the source first-mover and second-mover utilities, with the AA branch sharing one algorithmic ranking and every other branch using the independent pair law. Thus the first two conclusions are strict dominance against an algorithmic and a human opponent, and the third is the source all-human welfare comparison. The explicit positive-accuracy, probability, measurability, and integrability premises express the source noisy-family domain and make its stated expectations/conditional expectations well-defined without changing those comparisons.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 263-276

Verbatim source input

In what follows, we will show that for any θ,
E [π1 − π2 | π1 6= σ1] > 0 ⇐⇒ UAH (θ, θ) > UAA (θ, θ).

(2)

Expanded PaperInterface specification

```
∀ {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
  (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) [MeasureTheory.IsProbabilityMeasure D] (theta : ℝ),
  0 < theta →
    ∀
      (hatom :
        ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
          Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking),
      MeasureTheory.Integrable
          (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
            AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1)
          (F.outerIndependentPairJointLaw D theta hatom) →
        MeasureTheory.Integrable
            (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
              AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2)
            (F.outerIndependentPairJointLaw D theta hatom) →
          0 <
            ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
              if
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                    AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                1
              else 0 ∂F.outerIndependentPairJointLaw D theta hatom →
        have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
          F.outerIndependentPairJointLaw D theta hatom;
        have numerator : ℝ :=
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            if
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
              x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
            else 0 ∂J;
        have denominator : ℝ :=
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            if
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
              1
            else 0 ∂J;
        have UAA : ℝ :=
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1 ∂J;
        have UAH : ℝ :=
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2 ∂J;
        J = D.compProd (F.independentPairKernel theta hatom) ∧
          (∀ (value : KR21Monoculture.ValueProfile n),
              (F.independentPairKernel theta hatom) value =
                (AppliedModelingLib.pmfProd (F.dist theta value) (F.dist theta value)).toMeasure) ∧
            (0 < numerator / denominator ↔ UAA < UAH)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfProd
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.independentPairKernel
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: KR21Monoculture.DistributionalAccuracyFamily n
3. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
4. assumption: MeasureTheory.IsProbabilityMeasure D
5. parameter: ℝ
6. assumption: 0 < theta
7. assumption: ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking
8. assumption: MeasureTheory.Integrable
  (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
    AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1)
  (F.outerIndependentPairJointLaw D theta hatom)
9. assumption: MeasureTheory.Integrable
  (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
    AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2)
  (F.outerIndependentPairJointLaw D theta hatom)
10. assumption: 0 <
  ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
    if
        AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
          AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
      1
    else 0 ∂F.outerIndependentPairJointLaw D theta hatom
11. conclusion: F.outerIndependentPairJointLaw D theta hatom = D.compProd (F.independentPairKernel theta hatom) ∧
  (∀ (value : KR21Monoculture.ValueProfile n),
      (F.independentPairKernel theta hatom) value =
        (AppliedModelingLib.pmfProd (F.dist theta value) (F.dist theta value)).toMeasure) ∧
    (0 <
        (∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            if
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
              x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
            else 0 ∂F.outerIndependentPairJointLaw D theta hatom) /
          ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
            if
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
              1
            else 0 ∂F.outerIndependentPairJointLaw D theta hatom ↔
      ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1
            x.2.1 ∂F.outerIndependentPairJointLaw D theta hatom <
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1
            x.2.2 ∂F.outerIndependentPairJointLaw D theta hatom)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation2_outer_joint_payoff_equivalence_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: On the displayed conditionally iid outer joint law, the target identifies the source disagreement conditional gain as positive exactly when UAA<UAH.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 260-288

Verbatim source input

second-ranked candidate according to either permutation. In what follows, we'll demonstrate that this condition implies that firms in our model rationally prefer to make decisions using independent (but equally accurate) rankings.
To do so, we need to introduce some notation. Recall that the two firms hire in a random order. Given a candidate distribution D, let Us (θA , θH) denote the expected utility of the first firm to hire a candidate when using ranking s, where s ∈ {A, H} is either the algorithmic ranking or the ranking generated by a human evaluator respectively. Similarly, let Us1 s2 (θA , θH) be the expected utility of the second firm to hire given that the first firm used strategy s1 and the second firm uses strategy s2 , where again s1 , s2 ∈ {A, H}. Finally, let π, σ ∼ Fθ .
In what follows, we will show that for any θ,
E [π1 − π2 | π1 6= σ1] > 0 ⇐⇒ UAH (θ, θ) > UAA (θ, θ).

(2)

In other words, whenever a ranking model meets Definition 2, the firm chosen to select second will prefer to use an independent ranking mechanism from it's competitor, given that the ranking mechanisms are equally accurate.
First, we can write
UAH (θA , θH) = E [π1 · 1π1 6=σ1 + π2 · 1π1 =σ1]
UAA (θA , θH) = E [σ2]
= E [σ2 · 1π1 6=σ1 + σ2 · 1π1 =σ1]
Thus,
UAH (θA , θH) − UAA (θA , θH)
= E [(π1 − σ2) · 1π1 6=σ1 + (π2 − σ2) · 1π1 =σ1] .
Conditioned on either π1 = σ1 or π1 6= σ1 , π2 and σ2 are identically distributed and therefore have equal expectations. As a result,
UAH (θA , θH) − UAA (θA , θH) = E [(π1 − π2) · 1π1 6=σ1] ,

(3)

Expanded PaperInterface specification

```
∀ {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
  (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) [MeasureTheory.IsProbabilityMeasure D]
  (thetaA thetaH : ℝ),
  thetaA = thetaH →
    ∀
      (hatom :
        ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
          Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist thetaH value) ranking),
      MeasureTheory.Integrable
        (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1)
        (F.outerIndependentPairJointLaw D thetaH hatom) →
      MeasureTheory.Integrable
        (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2)
        (F.outerIndependentPairJointLaw D thetaH hatom) →
      have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
        F.outerIndependentPairJointLaw D thetaH hatom;
      have numerator : ℝ :=
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          if
              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
            x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
              x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
          else 0 ∂J;
      have UAA : ℝ :=
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1 ∂J;
      have UAH : ℝ :=
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2 ∂J;
      J = D.compProd (F.independentPairKernel thetaH hatom) ∧
        (∀ (value : KR21Monoculture.ValueProfile n),
            (F.independentPairKernel thetaH hatom) value =
              (AppliedModelingLib.pmfProd (F.dist thetaH value) (F.dist thetaH value)).toMeasure) ∧
          UAH - UAA = numerator
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfProd

- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.independentPairKernel
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: KR21Monoculture.DistributionalAccuracyFamily n
3. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
4. assumption: MeasureTheory.IsProbabilityMeasure D
5. parameter: ℝ
6. parameter: ℝ
7. assumption: thetaA = thetaH
8. assumption: ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist thetaH value) ranking
9. assumption: MeasureTheory.Integrable
  (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
    AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.1)
  (F.outerIndependentPairJointLaw D thetaH hatom)
10. assumption: MeasureTheory.Integrable
  (fun (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) =>
    AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1 x.2.2)
  (F.outerIndependentPairJointLaw D thetaH hatom)
11. conclusion: F.outerIndependentPairJointLaw D thetaH hatom = D.compProd (F.independentPairKernel thetaH hatom) ∧
  (∀ (value : KR21Monoculture.ValueProfile n),
      (F.independentPairKernel thetaH hatom) value =
        (AppliedModelingLib.pmfProd (F.dist thetaH value) (F.dist thetaH value)).toMeasure) ∧
    ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
        AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1
          x.2.2 ∂F.outerIndependentPairJointLaw D thetaH hatom -
      ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
        AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility x.1 x.2.1
          x.2.1 ∂F.outerIndependentPairJointLaw D thetaH hatom =
    ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
      if
          AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
            AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
        x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
          x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
      else 0 ∂F.outerIndependentPairJointLaw D thetaH hatom
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation3_outer_joint_payoff_identity_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: At equal accuracy, the target expands UAH-UAA to the source disagreement-event integral of the first-minus-second ranked value.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 176-182

Verbatim source input

Proof of Theorem 1. For given values of θA and θH , using the algorithmic ranking is a strictly dominant strategy as long as
UA (θA , θH) + UAA (θA , θH) > UH (θA , θH) + UAH (θA , θH)

(4)

Expanded PaperInterface specification

```
∀ {n : ℕ} (M : KR21Monoculture.Model n),
  M.firstMoverEU KR21Monoculture.Strategy.algorithm +
      M.secondMoverEU KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm >
    M.firstMoverEU KR21Monoculture.Strategy.human +
      M.secondMoverEU KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ↔
   M.payoffAgainst KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm >
    M.payoffAgainst KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm
```

Retained governing declarations

- KR21Monoculture.Model
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.payoffAgainst
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.Strategy

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: KR21Monoculture.Model n
3. conclusion: M.firstMoverEU KR21Monoculture.Strategy.algorithm +
     M.secondMoverEU KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm >
   M.firstMoverEU KR21Monoculture.Strategy.human +
     M.secondMoverEU KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ↔
  M.payoffAgainst KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm >
   M.payoffAgainst KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation4_algorithm_best_response_against_algorithm_iff

Recorded source-to-Spec screening **Verdict:** matches

Reason: The source U_A+U_AA>U_H+U_AH inequality is exactly the target payoffAgainst comparison against an algorithmic opponent.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 176-182

Verbatim source input

UA (θA , θH) + UHA (θA , θH) > UH (θA , θH) + UHH (θA , θH)

(5)

Expanded PaperInterface specification

```
∀ {n : ℕ} (F : KR21Monoculture.AccuracyFamily n) (thetaA thetaH : ℝ),
  0 < thetaH →
    thetaH < thetaA →
      (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
         remaining.Nonempty →
           KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value remaining ≤
             KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value remaining) →
        KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value Finset.univ <
          KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value Finset.univ →
        (F.modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm +
            (F.modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
              KR21Monoculture.Strategy.algorithm >
          (F.modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human +
            (F.modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.AccuracyFamily.modelAt
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.Strategy
- KR21Monoculture.expectedBestInSet

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: KR21Monoculture.AccuracyFamily n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 0 < thetaH
6. assumption: thetaH < thetaA
7. assumption: ∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
  remaining.Nonempty →
    KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value remaining ≤
      KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value remaining
8. assumption: KR21Monoculture.expectedBestInSet (F.dist thetaH) F.value Finset.univ <
  KR21Monoculture.expectedBestInSet (F.dist thetaA) F.value Finset.univ
9. conclusion: Real.lt✝
  ((F.modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human +
    (F.modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human)
  ((F.modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm +
    (F.modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation5_from_literal_definition1_removal

Recorded source-to-Spec screening **Verdict:** matches

Reason: The Definition 1 nonempty-set weak and full-set strict source inequalities yield the displayed U_A+U_HA>U_H+U_HH comparison.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 176-182

Verbatim source input

respect to θA . Therefore, by the Differentiability assumption on Fθ from Definition 1, there is some θA
such that
∗
∗
∗
∗
UA (θA
, θH) + UAA (θA
, θH) = UH (θA
, θH) + UAH (θA
, θH),
(6)

Expanded PaperInterface specification

```
∀ {n : ℕ} (F : KR21Monoculture.DistributionalAccuracyFamily n)
  (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)),
  MeasureTheory.IsProbabilityMeasure D →
    ∀ (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (thetaH : ℝ),
      0 < thetaH →
        (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
           MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D) →
          (∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
             MeasureTheory.AEStronglyMeasurable
               (fun (value : KR21Monoculture.ValueProfile n) => ((F.dist theta value) pi).toReal) D) →
            (∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
               0 < theta →
                 ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                   ContinuousAt (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta) →
              (∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
                 0 < theta →
                   ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                     DifferentiableAt ℝ (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta) →
                (∀ (value : KR21Monoculture.ValueProfile n),
                   Filter.Tendsto (fun (theta : ℝ) => ((F.dist theta value) center).toReal) Filter.atTop (nhds 1)) →
                  (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
                     AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value) →
                    ∀
                      (hatom_measurable :
                         ∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                           Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) pi),
                      (∀ (theta : ℝ),
                         0 < theta →
                           MeasureTheory.Integrable
                             (fun (value : KR21Monoculture.ValueProfile n) =>
                               KR21Monoculture.disagreementProb (F.dist theta value))
                             D) →
                        (∀ (theta : ℝ),
                           0 < theta →
                             MeasureTheory.Integrable
                               (fun (value : KR21Monoculture.ValueProfile n) =>
                                 AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value)
                                   value)
                               D) →
                          (∀ (theta : ℝ),
                             0 < theta →
                               MeasureTheory.Integrable
                                 (fun (value : KR21Monoculture.ValueProfile n) =>
                                   AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                                     (F.dist theta value) (F.dist theta value) value)
                                 D) →
                            (∀ (theta : ℝ),
                               0 < theta →
                                 MeasureTheory.Integrable
                                   KR21Monoculture.DistributionalAccuracyFamily.jointSharedSecondMoverPayoff
                                   (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))) →
                              (∀ (theta : ℝ),
                                 0 < theta →
                                   MeasureTheory.Integrable
                                     KR21Monoculture.DistributionalAccuracyFamily.jointIndependentSecondMoverPayoff
                                     (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))) →
                                (∀ (theta : ℝ),
                                   0 < theta →
                                     0 <
                                       ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                                         if KR21Monoculture.disagreementEvent x.2 then 1
                                         else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)) →
                                  (∀ (theta : ℝ),
                                     0 < theta →
                                       0 < F.jointLawDisagreementConditionalGain D theta (hatom_measurable theta)) →
                                    ∃ (thetaA : ℝ),
                                      thetaH < thetaA ∧
                                        ∫ (value : KR21Monoculture.ValueProfile n),
                                            ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                                              KR21Monoculture.Strategy.algorithm ∂D +
                                          ∫ (value : KR21Monoculture.ValueProfile n),
                                            ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                                              KR21Monoculture.Strategy.algorithm
                                              KR21Monoculture.Strategy.algorithm ∂D =
```

```
∫ (value : KR21Monoculture.ValueProfile n),
    ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
      KR21Monoculture.Strategy.human ∂D +
  ∫ (value : KR21Monoculture.ValueProfile n),
    ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
      KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ∂D
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- KR21Monoculture.AccuracyFamily.modelAt
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.jointIndependentSecondMoverPayoff
- KR21Monoculture.DistributionalAccuracyFamily.jointLawDisagreementConditionalGain
- KR21Monoculture.DistributionalAccuracyFamily.jointSharedSecondMoverPayoff
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.DistributionalAccuracyFamily.pointFamily
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.RankingPair
- KR21Monoculture.Strategy
- KR21Monoculture.ValueProfile
- KR21Monoculture.decidableDisagreementEvent
- KR21Monoculture.disagreementEvent
- KR21Monoculture.disagreementProb

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: KR21Monoculture.DistributionalAccuracyFamily n
3. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
4. assumption: MeasureTheory.IsProbabilityMeasure D
5. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
6. parameter: ℝ
7. assumption: 0 < thetaH
8. assumption: ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D
9. assumption: ∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  MeasureTheory.AEStronglyMeasurable (fun (value : KR21Monoculture.ValueProfile n) => ((F.dist theta value) pi).toReal)
    D
10. assumption: ∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
  0 < theta →
    ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
      ContinuousAt (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta
11. assumption: ∀ (value : KR21Monoculture.ValueProfile n) (theta : ℝ),
  0 < theta →
    ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
      DifferentiableAt ℝ (fun (theta' : ℝ) => ((F.dist theta' value) pi).toReal) theta
12. assumption: ∀ (value : KR21Monoculture.ValueProfile n),
  Filter.Tendsto (fun (theta : ℝ) => ((F.dist theta value) center).toReal) Filter.atTop (nhds 1)
13. assumption: ∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
14. assumption: ∀ (theta : ℝ) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) pi
15. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) => KR21Monoculture.disagreementProb (F.dist theta value)) D
16. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value) value)
      D
```

```
17. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable
      (fun (value : KR21Monoculture.ValueProfile n) =>
        AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (F.dist theta value) (F.dist theta value)
          value)
      D
18. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable KR21Monoculture.DistributionalAccuracyFamily.jointSharedSecondMoverPayoff
      (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))
19. assumption: ∀ (theta : ℝ),
  0 < theta →
    MeasureTheory.Integrable KR21Monoculture.DistributionalAccuracyFamily.jointIndependentSecondMoverPayoff
      (F.outerIndependentPairJointLaw D theta (hatom_measurable theta))
20. assumption: ∀ (theta : ℝ),
  0 < theta →
    0 <
      ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
        if KR21Monoculture.disagreementEvent x.2 then 1
        else 0 ∂F.outerIndependentPairJointLaw D theta (hatom_measurable theta)
21. assumption: ∀ (theta : ℝ), 0 < theta → 0 < F.jointLawDisagreementConditionalGain D theta (hatom_measurable theta)
22. conclusion: ∃ (thetaA : ℝ),
  thetaH < thetaA ∧
    ∫ (value : KR21Monoculture.ValueProfile n),
        ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.algorithm ∂D +
      ∫ (value : KR21Monoculture.ValueProfile n),
        ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
          KR21Monoculture.Strategy.algorithm ∂D =
    ∫ (value : KR21Monoculture.ValueProfile n),
        ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
      ∫ (value : KR21Monoculture.ValueProfile n),
        ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
          KR21Monoculture.Strategy.human ∂D
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation6_outer_payoff_indifference_from_source_conditions

Recorded source-to-Spec screening **Verdict:** matches

Reason: Under the displayed Definition 1, Definition 2, measurability, and integrability conditions, the target is the source existence of thetaA>thetaH with the literal outer payoff equality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 176-182

Verbatim source input

Theorem 2. Let Fθ be the family of RUMs with either Gaussian or Laplacian noise with standard deviation 1/θ. Then, for any candidate distribution D over 3 candidates, the conditions of Theorem 1 are satisfied.

Additional assumptions rank-labelled almost-everywhere strict source order; coordinatewise finite first moments and the induced payoff integrability; measurable finite ranking atoms; positive Definition 2 disagreement mass before conditional interpretation

Expanded PaperInterface specification

```
(∀ {theta x1 x2 x3 : ℝ},
   0 < theta →
    MeasureTheory.MeasurePreserving (⇑(KR21Monoculture.rightTripleToCandidateMeasurableEquiv ℝ))
       ((ProbabilityTheory.gaussianReal 0 1).prod
         ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1)))
       (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
         ProbabilityTheory.gaussianReal 0 1) ∧
      ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta).toMeasure =
         MeasureTheory.Measure.map
           (fun (epsilon : ℝ × ℝ × ℝ) =>
             AppliedModelingLib.SocialChoice.Ranking.rankByScore
               fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
               KR21Monoculture.threeCandidateValueProfile x1 x2 x3 i +
                 KR21Monoculture.rightTripleToCandidateFunction epsilon i / theta)
           ((ProbabilityTheory.gaussianReal 0 1).prod
             ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))) ∧
       (KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta =
         KR21Monoculture.gaussianThreeCandidateRankingLaw theta x1 x2 x3) ∧
  ProbabilityTheory.variance id (ProbabilityTheory.gaussianReal 0 1) = 1 ∧
    ProbabilityTheory.variance id
       (KR21Monoculture.w11BaseNoiseLaw KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity) =
      1 ∧
     (∀ {theta x1 x2 x3 : ℝ},
        0 < theta →
         MeasureTheory.MeasurePreserving (⇑(KR21Monoculture.rightTripleToCandidateMeasurableEquiv ℝ))
            ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
              ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
                (KR21Monoculture.theorem7LaplaceMeasure (√2) 0)))
            (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
              KR21Monoculture.theorem7LaplaceMeasure (√2) 0) ∧
           ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta).toMeasure =
              MeasureTheory.Measure.map
                (fun (epsilon : ℝ × ℝ × ℝ) =>
                  AppliedModelingLib.SocialChoice.Ranking.rankByScore
                    fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                    KR21Monoculture.threeCandidateValueProfile x1 x2 x3 i +
                      KR21Monoculture.rightTripleToCandidateFunction epsilon i / theta)
                ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
                  ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
                    (KR21Monoculture.theorem7LaplaceMeasure (√2) 0))) ∧
            (KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta =
              KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateRankingLaw theta x1 x2 x3) ∧
       (∀ {x1 x2 x3 : ℝ},
          x2 < x1 →
           x3 < x2 →
            (∀ (theta : ℝ),
               0 < theta →
                ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
                  ContinuousAt
                      (fun (theta' : ℝ) =>
                        (((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta') pi).toReal)
                      theta ∧
                    DifferentiableAt ℝ
                      (fun (theta' : ℝ) =>
                        (((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta') pi).toReal)
                      theta) ∧
              Filter.Tendsto
                 (fun (theta : ℝ) =>
                   (((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta)
                      KR21Monoculture.rum3Ranking012).toReal)
                 Filter.atTop (nhds 1) ∧
               ∀ (thetaA thetaH : ℝ),
                 0 < thetaH →
                  thetaH < thetaA →
                   (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)),
                      remaining.Nonempty →
                        KR21Monoculture.expectedBestInSet
                            ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaH)
                            (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining ≤
                          KR21Monoculture.expectedBestInSet
                            ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaA)
                            (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining) ∧
                     KR21Monoculture.expectedBestInSet
                        ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaH)
                        (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ <
                      KR21Monoculture.expectedBestInSet
                        ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaA)
```

```
                (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ) ∧
(∀ {x1 x2 x3 : ℝ},
   x2 < x1 →
    x3 < x2 →
     (∀ (theta : ℝ),
        0 < theta →
         ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
           ContinuousAt
              (fun (theta' : ℝ) =>
                (((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta')
                   pi).toReal)
              theta ∧
            DifferentiableAt ℝ
              (fun (theta' : ℝ) =>
                (((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta')
                   pi).toReal)
              theta) ∧
       Filter.Tendsto
          (fun (theta : ℝ) =>
            (((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta)
               KR21Monoculture.rum3Ranking012).toReal)
          Filter.atTop (nhds 1) ∧
        ∀ (thetaA thetaH : ℝ),
          0 < thetaH →
            thetaH < thetaA →
              (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)),
                 remaining.Nonempty →
                   KR21Monoculture.expectedBestInSet
                      ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist
                        thetaH)
                      (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining ≤
                     KR21Monoculture.expectedBestInSet
                      ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist
                        thetaA)
                      (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining) ∧
                KR21Monoculture.expectedBestInSet
                   ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist thetaH)
                   (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ <
                 KR21Monoculture.expectedBestInSet
                   ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist thetaA)
                   (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ) ∧
 (∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D],
    (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
       MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
     (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
      (∀ (theta : ℝ),
         0 < theta →
          ∀
            (regularity :
              KR21Monoculture.gaussianThreeCandidateDistributionalFamily.OuterIndependentRerankingJointRegularity
                D theta),
            0 <
               ∫ (x : KR21Monoculture.ValueProfile 1 × KR21Monoculture.RankingPair 1),
                 if KR21Monoculture.disagreementEvent x.2 then 1
                 else
                   0 ∂KR21Monoculture.gaussianThreeCandidateDistributionalFamily.outerIndependentPairJointLaw
                   D theta
                   (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_2
                     D theta regularity) ∧
              0 <
                KR21Monoculture.gaussianThreeCandidateDistributionalFamily.jointLawDisagreementConditionalGain
                  D theta
                  (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_2
                    D theta regularity)) ∧
        ∀ (thetaA thetaH : ℝ),
          0 < thetaH →
            thetaH < thetaA →
              ∫ (value : KR21Monoculture.ValueProfile 1),
                 AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                   (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaH value)
                   (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaA value)
                   value ∂D <
                ∫ (value : KR21Monoculture.ValueProfile 1),
                  AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                    (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaH value)
                    (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaH value)
                    value ∂D) ∧
  (∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D],
     (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
        MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
      (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
       (∀ (theta : ℝ),
          0 < theta →
           ∀
             (regularity :
               KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.OuterIndependentRerankingJointRegularity
                 D theta),
             0 <
                ∫ (x : KR21Monoculture.ValueProfile 1 × KR21Monoculture.RankingPair 1),
                  if KR21Monoculture.disagreementEvent x.2 then 1
                  else
                    0 ∂KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.outerIndependentPairJointLaw
                    D theta
                    (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_3
                      D theta regularity) ∧
               0 <
                 KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.jointLawDisagreementConditionalGain
                   D theta
                   (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_3
                     D theta regularity)) ∧
```

```
    ∀ (thetaA thetaH : ℝ),
      0 < thetaH →
        thetaH < thetaA →
          ∫ (value : KR21Monoculture.ValueProfile 1),
              AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                  thetaH value)
                (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                  thetaA value)
                value ∂D <
            ∫ (value : KR21Monoculture.ValueProfile 1),
              AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                  thetaH value)
                (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                  thetaH value)
                value ∂D) ∧
(∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D]
    (thetaH : ℝ),
    0 < thetaH →
      (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
          MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
        (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
          KR21Monoculture.PaperInterface.literal_outer_payoff_paradox
            KR21Monoculture.gaussianThreeCandidateDistributionalFamily D thetaH) ∧
  ∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D]
    (thetaH : ℝ),
    0 < thetaH →
      (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
          MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
        (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
          KR21Monoculture.PaperInterface.literal_outer_payoff_paradox
            KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily D thetaH
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.OuterIndependentRerankingJointRegularity
- KR21Monoculture.DistributionalAccuracyFamily.jointLawDisagreementConditionalGain
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.PaperInterface.literal_outer_payoff_paradox
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile
- KR21Monoculture.decidableDisagreementEvent
- KR21Monoculture.disagreementEvent
- KR21Monoculture.expectedBestInSet
- KR21Monoculture.gaussianThreeCandidateDistributionalFamily
- KR21Monoculture.gaussianThreeCandidateRankingLaw
- KR21Monoculture.rightTripleToCandidateFunction
- KR21Monoculture.rightTripleToCandidateMeasurableEquiv
- KR21Monoculture.rum3Ranking012
- KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity
- KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily
- KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily
- KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateRankingLaw
- KR21Monoculture.theorem2GaussianScaledNoiseFamily
- KR21Monoculture.theorem7LaplaceMeasure
- KR21Monoculture.threeCandidateValueProfile
- KR21Monoculture.w11BaseNoiseLaw

Lean-elaborated claim roles

```
1. conclusion: (∀ {theta x1 x2 x3 : ℝ},
    0 < theta →
      MeasureTheory.MeasurePreserving (⇑(KR21Monoculture.rightTripleToCandidateMeasurableEquiv ℝ))
          ((ProbabilityTheory.gaussianReal 0 1).prod
            ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1)))
          (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
            ProbabilityTheory.gaussianReal 0 1) ∧
        ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta).toMeasure =
          MeasureTheory.Measure.map
            (fun (epsilon : ℝ × ℝ × ℝ) =>
              AppliedModelingLib.SocialChoice.Ranking.rankByScore
                fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                KR21Monoculture.threeCandidateValueProfile x1 x2 x3 i +
                  KR21Monoculture.rightTripleToCandidateFunction epsilon i / theta)
            ((ProbabilityTheory.gaussianReal 0 1).prod
              ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))) ∧
        (KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta =
          KR21Monoculture.gaussianThreeCandidateRankingLaw theta x1 x2 x3) ∧
  ProbabilityTheory.variance id (ProbabilityTheory.gaussianReal 0 1) = 1 ∧
    ProbabilityTheory.variance id
        (KR21Monoculture.w11BaseNoiseLaw KR21Monoculture.sourceUnitVarianceLaplaceBaseDensity) =
      1 ∧
    (∀ {theta x1 x2 x3 : ℝ},
      0 < theta →
        MeasureTheory.MeasurePreserving (⇑(KR21Monoculture.rightTripleToCandidateMeasurableEquiv ℝ))
            ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
              ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
                (KR21Monoculture.theorem7LaplaceMeasure (√2) 0)))
            (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
              KR21Monoculture.theorem7LaplaceMeasure (√2) 0) ∧
          ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta).toMeasure =
            MeasureTheory.Measure.map
              (fun (epsilon : ℝ × ℝ × ℝ) =>
                AppliedModelingLib.SocialChoice.Ranking.rankByScore
                  fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                  KR21Monoculture.threeCandidateValueProfile x1 x2 x3 i +
                    KR21Monoculture.rightTripleToCandidateFunction epsilon i / theta)
              ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
                ((KR21Monoculture.theorem7LaplaceMeasure (√2) 0).prod
                  (KR21Monoculture.theorem7LaplaceMeasure (√2) 0))) ∧
            (KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta =
              KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateRankingLaw theta x1 x2 x3) ∧
      (∀ {x1 x2 x3 : ℝ},
        x2 < x1 →
          x3 < x2 →
            (∀ (theta : ℝ),
              0 < theta →
                ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
                  ContinuousAt
                      (fun (theta' : ℝ) =>
                        (((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta') pi).toReal)
                      theta ∧
                    DifferentiableAt ℝ
                      (fun (theta' : ℝ) =>
                        (((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta') pi).toReal)
                      theta) ∧
              Filter.Tendsto
                  (fun (theta : ℝ) =>
                    (((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist theta)
                        KR21Monoculture.rum3Ranking012).toReal)
                  Filter.atTop (nhds 1) ∧
                ∀ (thetaA thetaH : ℝ),
                  0 < thetaH →
                    thetaH < thetaA →
                      (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)),
                          remaining.Nonempty →
                            KR21Monoculture.expectedBestInSet
                                ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaH)
                                (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining ≤
                              KR21Monoculture.expectedBestInSet
                                ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaA)
                                (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining) ∧
                        KR21Monoculture.expectedBestInSet
                            ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaH)
                            (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ <
                          KR21Monoculture.expectedBestInSet
                            ((KR21Monoculture.theorem2GaussianScaledNoiseFamily x1 x2 x3).dist thetaA)
                            (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ) ∧
        (∀ {x1 x2 x3 : ℝ},
          x2 < x1 →
            x3 < x2 →
              (∀ (theta : ℝ),
                0 < theta →
                  ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
                    ContinuousAt
                        (fun (theta' : ℝ) =>
                          (((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta')
                              pi).toReal)
                        theta ∧
                      DifferentiableAt ℝ
                        (fun (theta' : ℝ) =>
                          (((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta')
                              pi).toReal)
                        theta) ∧
                Filter.Tendsto
                    (fun (theta : ℝ) =>
                      (((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist theta)
                          KR21Monoculture.rum3Ranking012).toReal)
```

```
          Filter.atTop (nhds 1) ∧
        ∀ (thetaA thetaH : ℝ),
          0 < thetaH →
            thetaH < thetaA →
              (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)),
                  remaining.Nonempty →
                    KR21Monoculture.expectedBestInSet
                        ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist
                          thetaH)
                        (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining ≤
                      KR21Monoculture.expectedBestInSet
                        ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist
                          thetaA)
                        (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) remaining) ∧
                KR21Monoculture.expectedBestInSet
                    ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist thetaH)
                    (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ <
                  KR21Monoculture.expectedBestInSet
                    ((KR21Monoculture.sourceUnitVarianceLaplaceScaledNoiseFamily x1 x2 x3).dist thetaA)
                    (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) Finset.univ) ∧
(∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D],
    (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
        MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
      (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
        (∀ (theta : ℝ),
            0 < theta →
              ∀
                (regularity :
                  KR21Monoculture.gaussianThreeCandidateDistributionalFamily.OuterIndependentRerankingJointRegularity
                    D theta),
                0 <
                    ∫ (x : KR21Monoculture.ValueProfile 1 × KR21Monoculture.RankingPair 1),
                      if KR21Monoculture.disagreementEvent x.2 then 1
                      else
                        0 ∂KR21Monoculture.gaussianThreeCandidateDistributionalFamily.outerIndependentPairJointLaw
                        D theta
                        (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_2
                          D theta regularity) ∧
                  0 <
                    KR21Monoculture.gaussianThreeCandidateDistributionalFamily.jointLawDisagreementConditionalGain
                      D theta
                      (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_2
                        D theta regularity)) ∧
          ∀ (thetaA thetaH : ℝ),
            0 < thetaH →
              thetaH < thetaA →
                ∫ (value : KR21Monoculture.ValueProfile 1),
                    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                      (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaH value)
                      (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaA value)
                      value ∂D <
                  ∫ (value : KR21Monoculture.ValueProfile 1),
                    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                      (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaH value)
                      (KR21Monoculture.gaussianThreeCandidateDistributionalFamily.dist thetaH value)
                      value ∂D) ∧
  (∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D],
      (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
          MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
        (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
          (∀ (theta : ℝ),
              0 < theta →
                ∀
                  (regularity :
                    KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.OuterIndependentRerankingJointRegularity
                      D theta),
                  0 <
                      ∫ (x : KR21Monoculture.ValueProfile 1 × KR21Monoculture.RankingPair 1),
                        if KR21Monoculture.disagreementEvent x.2 then 1
                        else
                          0 ∂KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.outerIndependentPairJointLaw
                          D theta
                          (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_3
                            D theta regularity) ∧
                    0 <
                      KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.jointLawDisagreementConditionalGain
                        D theta
                        (KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_completeSpec._proof_3
                          D theta regularity)) ∧
            ∀ (thetaA thetaH : ℝ),
              0 < thetaH →
                thetaH < thetaA →
                  ∫ (value : KR21Monoculture.ValueProfile 1),
                      AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                        (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                          thetaH value)
                        (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                          thetaA value)
                        value ∂D <
                    ∫ (value : KR21Monoculture.ValueProfile 1),
                      AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                        (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                          thetaH value)
                        (KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily.dist
                          thetaH value)
                        value ∂D) ∧
    (∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D]
        (thetaH : ℝ),
        0 < thetaH →
          (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
```

```
      MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
    (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
      KR21Monoculture.PaperInterface.literal_outer_payoff_paradox
        KR21Monoculture.gaussianThreeCandidateDistributionalFamily D thetaH) ∧
∀ (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile 1)) [MeasureTheory.IsProbabilityMeasure D]
  (thetaH : ℝ),
  0 < thetaH →
    (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1),
      MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile 1) => value c) D) →
    (∀ᵐ (value : KR21Monoculture.ValueProfile 1) ∂D, value 1 < value 0 ∧ value 2 < value 1) →
      KR21Monoculture.PaperInterface.literal_outer_payoff_paradox
        KR21Monoculture.sourceUnitVarianceLaplaceThreeCandidateDistributionalFamily D thetaH
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem2_gaussian_laplace_source_semantic_complete_literal_payoff

Recorded source-to-Spec screening **Verdict:** matches

Reason: For the source three-candidate unit-variance Gaussian and Laplace RUMs, the target supplies the exact iid laws and all Definition 1-3 and Theorem 1 conclusions under the displayed approved outer-D conditions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 530-540

Verbatim source input

Finally, there exist noise distributions that violate Definition 2 for any candidate distribution D. In
particular, the RUM family defined by the Gumbel distribution is well-known to be equivalent to the PlackettLuce model of ranking, which is generated by
↪ sequentially selecting candidate i with probability
exp(θxi)
,
j∈S exp(θxj)

P

(7)

where S is the set of remaining candidates [12, 4]. Under the Plackett-Luce model, for any θ, UAH (θ, θ) =

Expanded PaperInterface specification

```
∀ {n : ℕ} (theta : ℝ) (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
  0 < theta →
    ∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n))
      {i : AppliedModelingLib.SocialChoice.Ranking.Candidate n},
      i ∈ remaining →
        KR21Monoculture.plackettLuceChoiceProb theta value remaining i =
          Real.exp (theta * value i) / ∑ j ∈ remaining, Real.exp (theta * value j)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- KR21Monoculture.plackettLuceChoiceProb

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: ℝ
3. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
4. assumption: 0 < theta
5. parameter: Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)
6. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
7. assumption: i ∈ remaining
8. conclusion: KR21Monoculture.plackettLuceChoiceProb theta value remaining i =
   Real.exp (theta * value i) / ∑ j ∈ remaining, Real.exp (theta * value j)

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation7_plackettLuce_choice_probability

Recorded source-to-Spec screening **Verdict:** matches

Reason: For a remaining candidate i, the target is the source exp(theta*x_i) share divided by the sum over the remaining set.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 530-548

Verbatim source input

where S is the set of remaining candidates [12, 4]. Under the Plackett-Luce model, for any θ, UAH (θ, θ) =
UAA (θ, θ). To see this, suppose the firm that hires first selects candidate i∗ . Then, the firm that hires second
8

[form-feed in source extraction]
gets each candidate i with probability given by (7) with S = {1, . . . , n}\i∗ . As a result, by (3), if π, σ ∼ Fθ ,
E [π1 − π2 | π1 6= σ1] = 0
for any candidate distribution D, meaning the Plackett-Luce model never meets Definition 2. Thus, under

Expanded PaperInterface specification

```
∀ {n : ℕ} (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) [MeasureTheory.IsProbabilityMeasure D]
  {location theta : ℝ},
  0 < theta →
    (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
       MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D) →
     (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location =
        MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
          MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -(√6 / Real.pi) * Real.log arrival)
            (ProbabilityTheory.expMeasure 1)) ∧
       (∀ (profile epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
          (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
          KR21Monoculture.sourceUnitVarianceGumbelScores theta profile epsilon i = profile i + epsilon i / theta) ∧
         (∀ (profile : KR21Monoculture.ValueProfile n),
            (KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta profile).toMeasure =
              MeasureTheory.Measure.map
                (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                  AppliedModelingLib.SocialChoice.Ranking.rankByScore
                    fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => profile i + epsilon i / theta)
                (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                  MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -(√6 / Real.pi) * Real.log arrival)
                    (ProbabilityTheory.expMeasure 1))) ∧
           (∀ (profile : KR21Monoculture.ValueProfile n),
              KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta profile =
                KR21Monoculture.plackettLuceRankingPMF (theta / KR21Monoculture.unitVarianceGumbelScale) profile) ∧
             have F : KR21Monoculture.DistributionalAccuracyFamily n :=
               KR21Monoculture.DistributionalAccuracyFamily.sourceUnitVarianceGumbelRUMDistributionalFamily location;
             ∃ (hatom :
               ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                 Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking),
               have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
                 F.outerIndependentPairJointLaw D theta hatom;
               have numerator : ℝ :=
                 ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                   if
                       AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                         AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                     x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                       x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
                   else 0 ∂J;
               have denominator : ℝ :=
                 ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                   if
                       AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                         AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                     1
                   else 0 ∂J;
               J = D.compProd (F.independentPairKernel theta hatom) ∧
                 (∀ (profile : KR21Monoculture.ValueProfile n),
                    (F.independentPairKernel theta hatom) profile =
                      (AppliedModelingLib.pmfProd (F.dist theta profile) (F.dist theta profile)).toMeasure) ∧
                   MeasureTheory.Integrable
                     (fun (value : KR21Monoculture.ValueProfile n) =>
                       AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (F.dist theta value)
                         (F.dist theta value) value)
                     D ∧
                   MeasureTheory.Integrable
                     (fun (value : KR21Monoculture.ValueProfile n) =>
                       AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value)
                         value)
                     D ∧
                   0 < denominator ∧
                     numerator / denominator = 0 ∧
                       ∫ (value : KR21Monoculture.ValueProfile n),
                           AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                             (F.dist theta value) (F.dist theta value) value ∂D =
                         ∫ (value : KR21Monoculture.ValueProfile n),
                           AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value)
                             value ∂D
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking

- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.pmfProd
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.independentPairKernel
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.DistributionalAccuracyFamily.sourceUnitVarianceGumbelRUMDistributionalFamily
- KR21Monoculture.RankingPair
- KR21Monoculture.ValueProfile
- KR21Monoculture.plackettLuceRankingPMF
- KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw
- KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF
- KR21Monoculture.sourceUnitVarianceGumbelScores
- KR21Monoculture.unitVarianceGumbelScale

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
3. assumption: MeasureTheory.IsProbabilityMeasure D
4. parameter: ℝ
5. parameter: ℝ
6. assumption: 0 < theta
7. assumption: ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D
8. conclusion: (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location =
    MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
      MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -(√6 / Real.pi) * Real.log arrival)
        (ProbabilityTheory.expMeasure 1)) ∧
  (∀ (profile epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
      (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
      KR21Monoculture.sourceUnitVarianceGumbelScores theta profile epsilon i = profile i + epsilon i / theta) ∧
    (∀ (profile : KR21Monoculture.ValueProfile n),
      (KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta profile).toMeasure =
        MeasureTheory.Measure.map
          (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
            AppliedModelingLib.SocialChoice.Ranking.rankByScore
              fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => profile i + epsilon i / theta)
          (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
            MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -(√6 / Real.pi) * Real.log arrival)
              (ProbabilityTheory.expMeasure 1))) ∧
    (∀ (profile : KR21Monoculture.ValueProfile n),
      KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta profile =
        KR21Monoculture.plackettLuceRankingPMF (theta / KR21Monoculture.unitVarianceGumbelScale) profile) ∧
    have F : KR21Monoculture.DistributionalAccuracyFamily n :=
      KR21Monoculture.DistributionalAccuracyFamily.sourceUnitVarianceGumbelRUMDistributionalFamily location;
    ∃ (hatom :
      ∀ (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
        Measurable fun (value : KR21Monoculture.ValueProfile n) => (F.dist theta value) ranking),
      have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
        F.outerIndependentPairJointLaw D theta hatom;
      have numerator : ℝ :=
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          if
              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
            x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
              x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
          else 0 ∂J;
      have denominator : ℝ :=
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          if
              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
            1
          else 0 ∂J;
      J = D.compProd (F.independentPairKernel theta hatom) ∧
        (∀ (profile : KR21Monoculture.ValueProfile n),
          (F.independentPairKernel theta hatom) profile =
            (AppliedModelingLib.pmfProd (F.dist theta profile) (F.dist theta profile)).toMeasure) ∧
        MeasureTheory.Integrable
          (fun (value : KR21Monoculture.ValueProfile n) =>
```

```
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (F.dist theta value)
      (F.dist theta value) value)
  D ∧
MeasureTheory.Integrable
    (fun (value : KR21Monoculture.ValueProfile n) =>
      AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value) value)
    D ∧
  0 < denominator ∧
    numerator / denominator = 0 ∧
      ∫ (value : KR21Monoculture.ValueProfile n),
          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (F.dist theta value)
            (F.dist theta value) value ∂D =
        ∫ (value : KR21Monoculture.ValueProfile n),
          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared (F.dist theta value)
            value ∂D
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.section31_literalUnitVarianceGumbel_outer_zero_effect_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For the source unit-variance Gumbel RUM, the target gives its corrected Plackett-Luce temperature and the literal outer equality UAH(theta,theta)=UAA(theta,theta), equivalently zero conditional gain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 574-587

Verbatim source input

The Mallows Model also appears frequently in the ranking literature [8, 11], and is much more analytically tractable than RUMs. Under the Mallows Model, the likelihood of a permutation is related to its distance from the true ranking π ∗ :
∗
1
(8)
Pr[π] = φ−d(π,π) ,
Z
where Z is a normalizing constant. In this model, φ > 1 is the accuracy parameter: the larger φ is, the

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
        have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
        M.q = phi⁻¹ ∧
          (M.law pi).toReal =
            phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
              ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
8. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
   ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
     phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
       ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
         phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equation8_source_concrete_mallows_probability

Recorded source-to-Spec screening **Verdict:** matches

Reason: With theta=phi-1, the target gives the source normalized phi^(-Kendall-tau) Mallows mass and q=phi^-1.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 133-138

Verbatim source input

Theorem 3. Let Fθ be the family of Mallows Model distributions with parameter θ = φ − 1. Then, for any candidate distribution D, the conditions of Theorem 1 are satisfied.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For Candidate n = Fin(n+2) with n > 0, a fixed rank-labelled Mallows center, and a probability law D on value profiles with coordinatewise finite first moments and D-a.e. strict center order: for every phi > 1 and theta = phi - 1, the fixed-center source Mallows law has q = phi^(-1) and, for every value profile v and ranking pi, mass phi^(-KendallTau(center, pi)) / sum_tau phi^(-KendallTau(center, tau)). For every theta > 0 the actual outer-law conditionally-iid-ranking top-disagreement gain is positive, and for every thetaA > thetaH > 0 the Definition 3 weaker-competition conclusion holds.

Archival source anchor: cited publication, lines 590-591

Expanded PaperInterface specification

```
∀ {n : ℕ} (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) [MeasureTheory.IsProbabilityMeasure D]
  (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
  0 < n →
    (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
       MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D) →
      (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
         AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value) →
        (∀ (phi theta : ℝ),
           1 < phi →
             theta = phi - 1 →
               (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
                 ∀ (value : KR21Monoculture.ValueProfile n) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                   (((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist theta value) pi).toReal =
                     phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                       ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                         phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
          (∀ (value : KR21Monoculture.ValueProfile n),
             AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value →
               (∀ (theta : ℝ),
                  0 < theta →
                    ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                      ContinuousAt
                          (fun (theta' : ℝ) =>
                            (((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist theta') pi).toReal)
                          theta ∧
                        DifferentiableAt ℝ
                          (fun (theta' : ℝ) =>
                            (((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist theta') pi).toReal)
                          theta) ∧
                 Filter.Tendsto
                     (fun (theta : ℝ) =>
                       (((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist theta) center).toReal)
                     Filter.atTop (nhds 1) ∧
                   ∀ (thetaA thetaH : ℝ),
                     0 < thetaH →
                       thetaH < thetaA →
                         (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                            remaining.Nonempty →
                              KR21Monoculture.expectedBestInSet
                                  ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaH) value
                                  remaining ≤
                                KR21Monoculture.expectedBestInSet
                                  ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaA) value
                                  remaining) ∧
                           KR21Monoculture.expectedBestInSet
                               ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaH) value
                               Finset.univ <
                             KR21Monoculture.expectedBestInSet
                               ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaA) value
                               Finset.univ) ∧
            (∀ (theta : ℝ),
               0 < theta →
                 have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
                   (KR21Monoculture.fixedCenterMallowsDistributionalFamily center).outerIndependentPairJointLaw D theta
                     fun (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                     KR21Monoculture.DistributionalAccuracyFamily.fixedCenterMallows_ranking_atom_measurable center
                       theta ranking;
                 0 <
                   ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                     if
                         AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                           AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                       1
                     else 0 ∂J ∧
                 0 <
                   (∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                      if
                          AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                            AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                        x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                          x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
                      else 0 ∂J) /
```

```
        ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
          if
              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
            1
          else 0 ∂J) ∧
  (∀ (thetaA thetaH : ℝ),
     0 < thetaH →
       thetaH < thetaA →
         ∫ (value : KR21Monoculture.ValueProfile n),
           AppliedModelingLib.pmfPairExp
             ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaH value)
             ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaA value)
             fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
             AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D <
           ∫ (value : KR21Monoculture.ValueProfile n),
             AppliedModelingLib.pmfPairExp
               ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaH value)
               ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaH value)
               fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
               AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D) ∧
    ∀ (thetaH : ℝ),
      0 < thetaH →
        have F : KR21Monoculture.DistributionalAccuracyFamily n :=
          KR21Monoculture.fixedCenterMallowsDistributionalFamily center;
        ∃ (thetaA : ℝ),
          thetaH < thetaA ∧
            ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                  KR21Monoculture.Strategy.human ∂D +
              ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                  KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ∂D <
            ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                  KR21Monoculture.Strategy.algorithm ∂D +
              ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                  KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm ∂D ∧
            ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                  KR21Monoculture.Strategy.human ∂D +
              ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                  KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human ∂D <
            ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                  KR21Monoculture.Strategy.algorithm ∂D +
              ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                  KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm ∂D ∧
            ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                  KR21Monoculture.Strategy.algorithm ∂D +
              ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                  KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm ∂D <
            ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                  KR21Monoculture.Strategy.human ∂D +
              ∫ (value : KR21Monoculture.ValueProfile n),
                ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU
                  KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human ∂D
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility
- AppliedModelingLib.pmfPairExp
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.AccuracyFamily.modelAt
- KR21Monoculture.DistributionalAccuracyFamily
- KR21Monoculture.DistributionalAccuracyFamily.outerIndependentPairJointLaw
- KR21Monoculture.DistributionalAccuracyFamily.pointFamily

- KR21Monoculture.MallowsSpec
- KR21Monoculture.Model.firstMoverEU
- KR21Monoculture.Model.secondMoverEU
- KR21Monoculture.RankingPair
- KR21Monoculture.Strategy
- KR21Monoculture.ValueProfile
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.expectedBestInSet
- KR21Monoculture.fixedCenterMallowsDistributionalFamily
- KR21Monoculture.fixedCenterMallowsPointFamily

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
3. assumption: MeasureTheory.IsProbabilityMeasure D
4. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
5. assumption: 0 < n
6. assumption: ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D
7. assumption: ∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
8. conclusion: (∀ (phi theta : ℝ),
    1 < phi →
      theta = phi - 1 →
        (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
          ∀ (value : KR21Monoculture.ValueProfile n) (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
            (((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist theta value) pi).toReal =
              phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                  phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
  (∀ (value : KR21Monoculture.ValueProfile n),
      AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value →
        (∀ (theta : ℝ),
            0 < theta →
              ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                ContinuousAt
                    (fun (theta' : ℝ) =>
                      (((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist theta') pi).toReal)
                    theta ∧
                  DifferentiableAt ℝ
                    (fun (theta' : ℝ) =>
                      (((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist theta') pi).toReal)
                    theta) ∧
          Filter.Tendsto
              (fun (theta : ℝ) =>
                (((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist theta) center).toReal)
              Filter.atTop (nhds 1) ∧
            ∀ (thetaA thetaH : ℝ),
              0 < thetaH →
                thetaH < thetaA →
                  (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                      remaining.Nonempty →
                        KR21Monoculture.expectedBestInSet
                            ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaH) value remaining ≤
                          KR21Monoculture.expectedBestInSet
                            ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaA) value
                            remaining) ∧
                    KR21Monoculture.expectedBestInSet
                        ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaH) value Finset.univ <
                      KR21Monoculture.expectedBestInSet
                        ((KR21Monoculture.fixedCenterMallowsPointFamily center value).dist thetaA) value Finset.univ) ∧
    (∀ (theta : ℝ),
      0 < theta →
        have J : MeasureTheory.Measure (KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n) :=
          (KR21Monoculture.fixedCenterMallowsDistributionalFamily center).outerIndependentPairJointLaw D theta
            fun (ranking : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
            KR21Monoculture.DistributionalAccuracyFamily.fixedCenterMallows_ranking_atom_measurable center theta
              ranking;
        0 <
            ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
              if
                  AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                    AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                1
              else 0 ∂J ∧
          0 <
            (∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                if
                    AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                      AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
                  x.1 (AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1) -
                    x.1 (AppliedModelingLib.SocialChoice.Ranking.secondChoice x.2.1)
                else 0 ∂J) /
              ∫ (x : KR21Monoculture.ValueProfile n × KR21Monoculture.RankingPair n),
                if
```

```
              AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.1 ≠
                AppliedModelingLib.SocialChoice.Ranking.firstChoice x.2.2 then
            1
          else 0 ∂J) ∧
(∀ (thetaA thetaH : ℝ),
    0 < thetaH →
      thetaH < thetaA →
        ∫ (value : KR21Monoculture.ValueProfile n),
            AppliedModelingLib.pmfPairExp
              ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaH value)
              ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaA value)
              fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
              AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D <
          ∫ (value : KR21Monoculture.ValueProfile n),
            AppliedModelingLib.pmfPairExp
              ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaH value)
              ((KR21Monoculture.fixedCenterMallowsDistributionalFamily center).dist thetaH value)
              fun (pi sigma : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
              AppliedModelingLib.SocialChoice.Ranking.secondMoverUtility value pi sigma ∂D) ∧
 ∀ (thetaH : ℝ),
   0 < thetaH →
     have F : KR21Monoculture.DistributionalAccuracyFamily n :=
       KR21Monoculture.fixedCenterMallowsDistributionalFamily center;
     ∃ (thetaA : ℝ),
       thetaH < thetaA ∧
         ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
             ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
                 KR21Monoculture.Strategy.human ∂D <
           ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                 KR21Monoculture.Strategy.algorithm ∂D +
             ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
                 KR21Monoculture.Strategy.algorithm ∂D ∧
           ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
             ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
                 KR21Monoculture.Strategy.human ∂D <
           ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                 KR21Monoculture.Strategy.algorithm ∂D +
             ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
                 KR21Monoculture.Strategy.algorithm ∂D ∧
           ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU
                 KR21Monoculture.Strategy.algorithm ∂D +
             ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.algorithm
                 KR21Monoculture.Strategy.algorithm ∂D <
           ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).firstMoverEU KR21Monoculture.Strategy.human ∂D +
             ∫ (value : KR21Monoculture.ValueProfile n),
               ((F.pointFamily value).modelAt thetaA thetaH).secondMoverEU KR21Monoculture.Strategy.human
                 KR21Monoculture.Strategy.human ∂D
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem3_source_complete_semantic_complete_literal_payoff

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The target is the approved corrected Mallows theorem: N>=3, fixed center order, finite first moments, actual outer iid rankings, Definitions 1-3, and the literal outer Theorem 1 endpoint.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 133-138

Verbatim source input

Theorem 4. When φH ≥ φA , one optimal sequence is for all firms to choose H. When φH > φA , the unique optimal sequence is for all firms to choose H.

[Next verbatim source excerpt]

[form-feed in source extraction]
order. In this context, consider the rational behavior of each firm: given a distribution over candidate values, which ranking should each firm use? Clearly, the first firm to make a selection should use the more accurate ranking mechanism; however, as shown previously, subsequent firms' decisions are less clear-cut. For a fixed number of firms, number of candidates, and distribution over candidate values, we can explore the firms' optimal strategies over the possible space of (φH , φA) values.
An optimal choice of strategies for the k firms moving sequentially can be written as a sequence of length k made up of the symbols A and H; the ith term in the sequence is equal to A if the ith firm to move sequentially uses the algorithm as its optimal strategy (given the choices of the previous i − 1 firms), and it is equal to H if the ith firm uses an independent human evaluation. We can therefore represent the choice of optimal strategies, as the parameters (φH , φA) vary, by a labeling of the (φH , φA)-plane: we label each point (φH , φA) with the length-k sequence that specifies the optimal sequence of strategies.
We can make the following initial formal observation about these optimal sequences:
Theorem 4. When φH ≥ φA , one optimal sequence is for all firms to choose H. When φH > φA , the unique optimal sequence is for all firms to choose H.
We prove this formally in Appendix F.1, but we provide a sketch here. When φH ≥ φA , the first firm to move in sequence will simply use the more accurate strategy, and hence will choose H. Now, proceeding by induction, suppose that the first i firms have all chosen H, and consider the (i + 1)st firm to move in sequence. Regardless of whether this firm chooses A or H, it will be making a selection that is independent of the previous i selections, and hence it is optimal for it to choose H as well. Hence, by induction, it is an optimal solution for all firms to choose H when φH ≥ φA . (This argument, slightly adapted, also directly establishes that it is uniquely optimal for all firms to choose H when φH > φA .)

[Next verbatim source excerpt]

Proof of Theorem 4

To prove this theorem, we make use of the following lemma.
Lemma 8. Under the Mallows model, the probability that any two items i < j are correctly ranked increases monotonically with the accuracy parameter φ.
1 Alternatively, we could prove this by showing that for any permutation with i in front, the permutation in which i and i − 1 are swapped is φ times more likely, and thus, i − 1 is φ times more likely to be in front than i.

33

[form-feed in source extraction]
Proof. Let inv(π) be the number of inversions in a permutation π. Under the Mallows model, the probability of observing π is proportional to φ− inv(π) . Let Si[U+001F control character]j (resp. Sj[U+001F control character]i) be the set of permutations where i is ↪ ranked before j (resp. j is ranked before i). Then, the probability i is ranked before j is

P
− inv(π)
π∈Si[U+001F control character]j φ
P
.
Pr[i [U+001F control character] j] = P
− inv(π) +
− inv(π)
π∈Si[U+001F control character]j φ
π∈Sj[U+001F control character]i φ
d
d Pr[i[U+001F control character]j]
We will show that dφ
Pr[i [U+001F control character] j] > 0. Note that this is equivalent to showing dφ
Pr[j[U+001F control character]i] > 0. Note that

P
− inv(π)
Pr[i [U+001F control character] j]
π∈Si[U+001F control character]j φ
.
=P
− inv(π)
Pr[j [U+001F control character] i]
π∈Sj[U+001F control character]i φ

Let πi:j be the subsequence of π containing elements i through j. Then, we have

P
− inv(π)
Pr[i [U+001F control character] j]
π∈Si[U+001F control character]j φ
=P
− inv(π)
Pr[j [U+001F control character] i]
π∈Sj[U+001F control character]i φ
P
P
0
− inv(πi:j)
φinv(πi:j)−inv(π)
0 =π
πi:j :π∈Si[U+001F control character]j φ
π 0 :πi:j
i:j
P

=P
0
− inv(πi:j)
φinv(πi:j)−inv(π)
0 =π
πi:j :π∈Sj[U+001F control character]i φ
π 0 :πi:j
i:j
P
− inv(πi:j)
πi:j :π∈Si[U+001F control character]j φ
=P
− inv(πi:j)
πi:j :π∈Sj[U+001F control character]i φ
P
0
Intuitively, the term
φinv(πi:j)−inv(π) does not depend on πi:j because for any πi:j , if we fix
0 =π
π 0 :πi:j
i:j
the order and positions of the remaining elements, the number of inversions involving at least one element outside of i : j (i.e., inv(π 0)−inv(πi:j)) is a constant. For fixed πi:j , there is a bijection between permutations
0
π 0 : πi:j
= πi:j and a fixed order and position of the remaining elements (excluding i : j), meaning this sum does not depend on πi:j . Thus, for the remainder of this proof, we can assume without loss of generality that i = 1 and j = n. The quantity of interest becomes
P
− inv(π1:n)
Pr[1 [U+001F control character] n]
π :π∈S1[U+001F control character]n φ
= P 1:n
− inv(π1:n)
Pr[n [U+001F control character] 1]
πi:j :π∈Sn[U+001F control character]1 φ
P
− inv(π)
π∈S1[U+001F control character]n φ
=P
− inv(π)
π∈Sn[U+001F control character]1 φ
Next, we observe that we can similarly ignore inversions between two elements that are neither 1 nor n. To see this, let inv1,n (π) be the number of inversions involving at least one of 1 and n. Then, if we fix the order and positions of 1 and n, all possible permutations of the remaining elements 2 through n − 1 produce the same number of inversions inv1,n (π). More formally, let π(1) and π(n) be the respective positions of elements 1 and n. Then, this we have
X
X
X
φ− inv(π) =
φ− inv(π)
π∈S1[U+001F control character]n

k<` π:π(1) =k,π(n) =`

=

X

=

X

X

φ− inv1,n (φ) · φinv1,n (φ)−inv(π)

k<` π:π(1) =k,π(n) =`

φ−(k−1)−(n−`)

k<`

X

φinv1,n (φ)−inv(π)

π:π(1) =k,π(n) =`

inv1,n (φ)−inv(π)
As noted above,
does not depend on k or `, since every permutation of
π:π(1) =k,π(n) =` φ
the remaining elements yields the same number of inversions among them regardless of k and `. A similar argument yields
X
X
X
φ− inv(π) =
φ−(k−1)−(n−`)+1
φinv1,n (φ)−inv(π)

P

π∈Sn[U+001F control character]1

k>`

π:π(1) =k,π(n) =`

[form-feed in source extraction]
Putting these together, we have
P
P
−(k−1)−(n−`)
inv1,n (φ)−inv(π)
Pr[1 [U+001F control character] n]
k<` φ
π:π(1) =k,π(n) =` φ
P
=P
inv1,n (φ)−inv(π)
−(k−1)−(n−`)+1
Pr[n [U+001F control character] 1]
π:π(1) =k,π(n) =` φ
k>` φ
P
φ−(k−1)−(n−`)
= P k<` −(k−1)−(n−`)+1
k>` φ
P
φ−(k−1)−(n−`)
φn−1
= P k<` −(k−1)−(n−`)+1 · n−1
φ
k>` φ
P
φ`−k
= P k<` `−k+1
k>` φ
Note that each term in the numerator is strictly increasing in φ, while each term in the denominator is
d
d Pr[1[U+001F control character]n]
weakly decreasing in φ. As a result, dφ
Pr[n[U+001F control character]1] > 0, meaning for any i < j, dφ Pr[i [U+001F control character] j] > 0.
With this, we proceed inductively, showing that when φH ≥ φA , each firm rationally chooses to use H.
For the first firm, by Lemma 8, H is more likely than A to correctly order any pair of candidates, meaning it produces higher expected utility. Similarly, for any subsequent firm, conditioned on the remaining candidates, H is still more likely to correctly order any pair of remaining candidates, meaning it leads to higher expected utility. A similar argument shows that all firms strictly prefer H when φH > φA .

Additional assumptions fixed rank-labelled Mallows center almost surely orders the source value profiles; coordinatewise finite first moments under D; independent fresh rankings and ex-ante comparison at each feasible remaining set; finite hiring horizon K < N so every reviewed history leaves at least one candidate; no unprovided posterior kernel is inferred from prior hiring histories; no strategy payoff is defined after every candidate has been hired

Expanded PaperInterface specification

```
∀ {n k : ℕ} (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) [MeasureTheory.IsProbabilityMeasure D]
  (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phiA thetaA phiH thetaH : ℝ),
  1 < phiA →
    1 < phiH →
      thetaA = phiA - 1 →
        thetaH = phiH - 1 →
          k < Fintype.card (AppliedModelingLib.SocialChoice.Ranking.Candidate n) →
            (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
               MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D) →
              have algorithm : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center thetaA;
              have human : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center thetaH;
              algorithm.q = phiA⁻¹ ∧
                human.q = phiH⁻¹ ∧
                  (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                     (algorithm.law pi).toReal =
                       phiA⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                         ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                           phiA⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                    (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                       (human.law pi).toReal =
                         phiH⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                           ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                             phiH⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                      (phiA ≤ phiH →
                         (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
                            AppliedModelingLib.SocialChoice.Ranking.WeaklyOrderedBy center value) →
                           ∀ (i : Fin k) (hired : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                             hired.card = ↑i →
                               ∀ (strategy : KR21Monoculture.Strategy),
                                 ∫ (value : KR21Monoculture.ValueProfile n),
                                     ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                                       ((match strategy with
                                           | KR21Monoculture.Strategy.algorithm => algorithm.law
                                           | KR21Monoculture.Strategy.human => human.law)
                                         pi).toReal *
                                         value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D ≤
                                   ∫ (value : KR21Monoculture.ValueProfile n),
                                     ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                                       (human.law pi).toReal *
                                         value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D) ∧
                        (phiA < phiH →
                           (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
                              AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value) →
```

```
∀ (i : Fin k) (hired : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
  hired.card = ↑i →
    ∀ (strategy : KR21Monoculture.Strategy),
      (∀ (alternative : KR21Monoculture.Strategy),
         ∫ (value : KR21Monoculture.ValueProfile n),
           ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
             ((match alternative with
                 | KR21Monoculture.Strategy.algorithm => algorithm.law
                 | KR21Monoculture.Strategy.human => human.law)
               pi).toReal *
             value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D ≤
       ∫ (value : KR21Monoculture.ValueProfile n),
         ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
           ((match strategy with
               | KR21Monoculture.Strategy.algorithm => algorithm.law
               | KR21Monoculture.Strategy.human => human.law)
             pi).toReal *
           value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D) →
    strategy = KR21Monoculture.Strategy.human)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.WeaklyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- KR21Monoculture.MallowsSpec
- KR21Monoculture.Strategy
- KR21Monoculture.ValueProfile
- KR21Monoculture.bestInSet
- KR21Monoculture.concreteMallowsSpec

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: ℕ
3. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
4. assumption: MeasureTheory.IsProbabilityMeasure D
5. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
6. parameter: ℝ
7. parameter: ℝ
8. parameter: ℝ
9. parameter: ℝ
10. assumption: 1 < phiA
11. assumption: 1 < phiH
12. assumption: thetaA = phiA - 1
13. assumption: thetaH = phiH - 1
14. assumption: k < Fintype.card (AppliedModelingLib.SocialChoice.Ranking.Candidate n)
15. assumption: ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
  MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D
16. conclusion: (KR21Monoculture.concreteMallowsSpec center thetaA).q = phiA⁻¹ ∧
  (KR21Monoculture.concreteMallowsSpec center thetaH).q = phiH⁻¹ ∧
    (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
       ((KR21Monoculture.concreteMallowsSpec center thetaA).law pi).toReal =
         phiA⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
           ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
             phiA⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
      (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
         ((KR21Monoculture.concreteMallowsSpec center thetaH).law pi).toReal =
           phiH⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
             ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
               phiH⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
        (phiA ≤ phiH →
           (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
              AppliedModelingLib.SocialChoice.Ranking.WeaklyOrderedBy center value) →
             ∀ (i : Fin k) (hired : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
               hired.card = ↑i →
                 ∀ (strategy : KR21Monoculture.Strategy),
                   ∫ (value : KR21Monoculture.ValueProfile n),
                     ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                       ((match strategy with
                           | KR21Monoculture.Strategy.algorithm =>
                             (KR21Monoculture.concreteMallowsSpec center thetaA).law
                           | KR21Monoculture.Strategy.human =>
                             (KR21Monoculture.concreteMallowsSpec center thetaH).law)
                         pi).toReal *
                       value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D ≤
                     ∫ (value : KR21Monoculture.ValueProfile n),
                       ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                         ((KR21Monoculture.concreteMallowsSpec center thetaH).law pi).toReal *
                           value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D) ∧
          (phiA < phiH →
             (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
```

```
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value) →
∀ (i : Fin k) (hired : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
  hired.card = ↑i →
    ∀ (strategy : KR21Monoculture.Strategy),
      (∀ (alternative : KR21Monoculture.Strategy),
          ∫ (value : KR21Monoculture.ValueProfile n),
              ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                ((match alternative with
                      | KR21Monoculture.Strategy.algorithm =>
                        (KR21Monoculture.concreteMallowsSpec center thetaA).law
                      | KR21Monoculture.Strategy.human =>
                        (KR21Monoculture.concreteMallowsSpec center thetaH).law)
                      pi).toReal *
                  value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D ≤
            ∫ (value : KR21Monoculture.ValueProfile n),
              ∑ pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                ((match strategy with
                      | KR21Monoculture.Strategy.algorithm =>
                        (KR21Monoculture.concreteMallowsSpec center thetaA).law
                      | KR21Monoculture.Strategy.human =>
                        (KR21Monoculture.concreteMallowsSpec center thetaH).law)
                      pi).toReal *
                  value (KR21Monoculture.bestInSet pi (Finset.univ \ hired)) ∂D) →
        strategy = KR21Monoculture.Strategy.human)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem4_source_mallows_outer_horizon_lt_literal_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: With the approved finite horizon k<N and fresh rankings, the target is Theorem 4: all-human is optimal when phiH>=phiA and uniquely optimal when phiH>phiA.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 14

Source locator: cited publication, lines 782-951

Verbatim source input

Theorem 5. Let f be the pdf of E. The family of RUMs Fθ given by ranking xi + εθi with εi ∼ E satisfies
the conditions of Definition 1 if:
• f is differentiable
• f has positive support on (−∞, ∞)

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

Let Candidate n = Fin(n+2). Let f : R -> R be measurable, integrable, everywhere strictly positive, normalized by integral ENNReal.ofReal(f) = 1, and
↪ absolutely continuous on every real interval; let f_prime be integrable and agree almost everywhere with deriv f. For a value profile strictly descending
↪ along a center ranking, the iid product density law is a probability law, and there is a ranking law F_theta such that for every theta > 0 it is exactly the
↪ pushforward of that product law under epsilon |-> rank(value_i + epsilon_i/theta). Every ranking atom of F_theta is continuous and differentiable at
↪ every positive theta, every atom tends to the pure center-ranking mass as theta tends to infinity, expected value of the best candidate in every
↪ nonempty remaining set is weakly increasing with theta, and full-set expected value is strictly increasing when theta_A > theta_H > 0.

Archival source anchor: cited publication, lines 784-787

Expanded PaperInterface specification

```
∀ {n : ℕ} (f derivative : ℝ → ℝ),
  MeasureTheory.Integrable f MeasureTheory.volume →
    MeasureTheory.Integrable derivative MeasureTheory.volume →
      Measurable f →
        (∀ (x : ℝ), 0 < f x) →
          (∀ (a b : ℝ), AbsolutelyContinuousOnInterval f a b) →
            derivative =ᵐ[MeasureTheory.volume] deriv f →
              ∫⁻ (x : ℝ), ENNReal.ofReal (f x) = 1 →
                ∀ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
                  (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                  (∀ (i : Fin (n + 1)), value (center i.succ) < value (center i.castSucc)) →
                    Fintype.card (AppliedModelingLib.SocialChoice.Ranking.Candidate n) = n + 2 ∧
                      MeasureTheory.IsProbabilityMeasure
                          (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                            MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (f x)) ∧
                        ∃ (rankingLaw : ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)),
                          (∀ (theta : ℝ),
                              0 < theta →
                                (rankingLaw theta).toMeasure =
                                  MeasureTheory.Measure.map
                                    (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                                      AppliedModelingLib.SocialChoice.Ranking.rankByScore
                                        fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                                        value i + epsilon i / theta)
                                    (MeasureTheory.Measure.pi
                                      fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                                      MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (f x))) ∧
                            (∀ (theta : ℝ),
                              0 < theta →
                                ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                                  ContinuousAt (fun (theta' : ℝ) => ((rankingLaw theta') pi).toReal) theta ∧
                                    DifferentiableAt ℝ (fun (theta' : ℝ) => ((rankingLaw theta') pi).toReal) theta) ∧
                              (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                                Filter.Tendsto (fun (theta : ℝ) => ((rankingLaw theta) pi).toReal) Filter.atTop
                                  (nhds ((PMF.pure center) pi).toReal)) ∧
                              (∀ (thetaA thetaH : ℝ),
                                0 < thetaH →
                                  thetaH < thetaA →
                                    ∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                                      remaining.Nonempty →
                                        KR21Monoculture.expectedBestInSet (rankingLaw thetaH) value remaining ≤
                                          KR21Monoculture.expectedBestInSet (rankingLaw thetaA) value remaining) ∧
                                ∀ (thetaA thetaH : ℝ),
                                  0 < thetaH →
                                    thetaH < thetaA →
                                      KR21Monoculture.expectedBestInSet (rankingLaw thetaH) value Finset.univ <
                                        KR21Monoculture.expectedBestInSet (rankingLaw thetaA) value Finset.univ
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.expectedBestInSet

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: ℝ → ℝ
3. parameter: ℝ → ℝ
4. assumption: MeasureTheory.Integrable f MeasureTheory.volume
5. assumption: MeasureTheory.Integrable derivative MeasureTheory.volume
6. assumption: Measurable f
7. assumption: ∀ (x : ℝ), 0 < f x
8. assumption: ∀ (a b : ℝ), AbsolutelyContinuousOnInterval f a b
9. assumption: derivative =ᵐ[MeasureTheory.volume] deriv f
10. assumption: ∫⁻ (x : ℝ), ENNReal.ofReal (f x) = 1
11. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
12. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
13. assumption: ∀ (i : Fin (n + 1)), value (center i.succ) < value (center i.castSucc)
14. conclusion: Fintype.card (AppliedModelingLib.SocialChoice.Ranking.Candidate n) = n + 2 ∧
  MeasureTheory.IsProbabilityMeasure
      (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
        MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (f x)) ∧
    ∃ (rankingLaw : ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking n)),
      (∀ (theta : ℝ),
          0 < theta →
            (rankingLaw theta).toMeasure =
              MeasureTheory.Measure.map
                (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                  AppliedModelingLib.SocialChoice.Ranking.rankByScore
                    fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => value i + epsilon i / theta)
                (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                  MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (f x))) ∧
        (∀ (theta : ℝ),
            0 < theta →
              ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                ContinuousAt (fun (theta' : ℝ) => ((rankingLaw theta') pi).toReal) theta ∧
                  DifferentiableAt ℝ (fun (theta' : ℝ) => ((rankingLaw theta') pi).toReal) theta) ∧
          (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
              Filter.Tendsto (fun (theta : ℝ) => ((rankingLaw theta) pi).toReal) Filter.atTop
                (nhds ((PMF.pure center) pi).toReal)) ∧
            (∀ (thetaA thetaH : ℝ),
                0 < thetaH →
                  thetaH < thetaA →
                    ∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                      remaining.Nonempty →
                        KR21Monoculture.expectedBestInSet (rankingLaw thetaH) value remaining ≤
                          KR21Monoculture.expectedBestInSet (rankingLaw thetaA) value remaining) ∧
              ∀ (thetaA thetaH : ℝ),
                0 < thetaH →
                  thetaH < thetaA →
                    KR21Monoculture.expectedBestInSet (rankingLaw thetaH) value Finset.univ <
                      KR21Monoculture.expectedBestInSet (rankingLaw thetaA) value Finset.univ
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.appendixA_theorem5_corrected_source_complete

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The expanded W1,1 density assumptions and iid scaled-ranking law, atom regularity, limit, and weak/strict monotonicity are the queue's approved corrected Theorem 5 target.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 15

Source locator: cited publication, lines 493-504

Verbatim source input

Recall that by definition, the candidates are ranked according to xi + εθi . The probability that x1 is
ranked first is
[U+0014 control character]
[U+0015 control character]
[U+0014 control character]
[U+0015 control character]
εi
εi
ε1
ε1
Pr x1 +
> max xi +
= Pr
> max xi − x1 +
2≤i≤n
2≤i≤n
θ
θ
θ
θ
[U+0014 control character]
[U+0015 control character]
= Pr ε1 > max θ(xi − x1) + εi
2≤i≤n
[U+0014 control character]
[U+0015 control character]
= Eε2 ,...,εn Pr ε1 > max θ(xi − x1) + εi | ε2 , . . . , εn
(A.1)
2≤i≤n

Expanded PaperInterface specification

```
∀ {n : ℕ} (f : ℝ → ℝ),
  Measurable f →
    (∀ (x : ℝ), 0 ≤ f x) →
      ∫⁻ (x : ℝ), ENNReal.ofReal (f x) = 1 →
        ∀ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
          (∀ (d : Fin (n + 1)), value d.succ < value 0) →
            ∀ {theta : ℝ},
              0 < theta →
                (AppliedModelingLib.measureProb
                   ((KR21Monoculture.sourceAppendixARestNoiseLaw n (KR21Monoculture.w11BaseNoiseLaw f)).prod
                     (KR21Monoculture.w11BaseNoiseLaw f))
                   fun (z : (Fin (n + 1) → ℝ) × ℝ) =>
                   AppliedModelingLib.SocialChoice.Ranking.firstChoice
                     (AppliedModelingLib.SocialChoice.Ranking.rankByScore
                       fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                       value i + KR21Monoculture.sourceAppendixAProductNoise z i / theta) =
                     0) =
                  ∫ (rest : Fin (n + 1) → ℝ),
                    AppliedModelingLib.measureProb (KR21Monoculture.w11BaseNoiseLaw f) fun (epsilon : ℝ) =>
                      ∀ (d : Fin (n + 1)),
                        theta * (value d.succ - value 0) + rest d <
                          epsilon ∂KR21Monoculture.sourceAppendixARestNoiseLaw n (KR21Monoculture.w11BaseNoiseLaw f)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- AppliedModelingLib.measureProb
- KR21Monoculture.sourceAppendixAProductNoise
- KR21Monoculture.sourceAppendixARestNoiseLaw
- KR21Monoculture.w11BaseNoiseLaw

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: ℝ → ℝ
3. assumption: Measurable f
4. assumption: ∀ (x : ℝ), 0 ≤ f x
5. assumption: ∫⁻ (x : ℝ), ENNReal.ofReal (f x) = 1
6. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
7. assumption: ∀ (d : Fin (n + 1)), value d.succ < value 0
8. parameter: ℝ
9. assumption: 0 < theta
10. conclusion: (AppliedModelingLib.measureProb

```
  ((KR21Monoculture.sourceAppendixARestNoiseLaw n (KR21Monoculture.w11BaseNoiseLaw f)).prod
    (KR21Monoculture.w11BaseNoiseLaw f))
  fun (z : (Fin (n + 1) → ℝ) × ℝ) =>
  AppliedModelingLib.SocialChoice.Ranking.firstChoice
      (AppliedModelingLib.SocialChoice.Ranking.rankByScore
        fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
        value i + KR21Monoculture.sourceAppendixAProductNoise z i / theta) =
    0) =
∫ (rest : Fin (n + 1) → ℝ),
  AppliedModelingLib.measureProb (KR21Monoculture.w11BaseNoiseLaw f) fun (epsilon : ℝ) =>
    ∀ (d : Fin (n + 1)),
      theta * (value d.succ - value 0) + rest d <
        epsilon ∂KR21Monoculture.sourceAppendixARestNoiseLaw n (KR21Monoculture.w11BaseNoiseLaw f)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationA1_source_w11_iid_literal_event_conditionalTail_integral

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target expands Pr[x1 is first] under iid scaled noise to the source conditional tail event epsilon1>max_i theta(x_i-x1)+epsiloni.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 16

Source locator: cited publication, lines 190-198

Verbatim source input

Here, we provide a noise mode E, accuracy parameter θ, and candidate distribution D such that UAH < UAA .
Choose the noise distribution E and accuracy parameter θ such that
[U+F8F1 delimiter glyph]
[U+F8F4 delimiter glyph]1
w.p. 2δ
ε [U+F8F2 delimiter glyph]
= 0
w.p.1 − δ
θ [U+F8F4 delimiter glyph]
[U+F8F3 delimiter glyph]
−1 w.p. 2δ
Note that this distribution does not satisfy Definition 1 because it is neither differentiable nor supported on (−∞, ∞); however, we can provide a “smooth” approximation to this distribution by expressing it as the sum of arbitrarily tightly concentrated Gaussians with the same results.

15

[form-feed in source extraction]
We choose the candidate distribution D such that x1 − 1 > x2 > x3 > x1 − 2. For example,
7
4
1
x2 =
2
x3 = 0
x1 =

Under this condition, assuming x3 = 0 without loss of generality,
UAH (θ, θ) − UAA (θ, θ) =

[U+0001 control character]
δ2 3
δ x1 − 4δ 2 x1 + 4δx1 + 2δ 3 x2 − 14δ 2 x2 + 20δx2 − 8x2
32
2

Notice that the lowest-power δ term is − δ 4x2 . Therefore, for sufficiently small δ, this is negative. For example, plugging in the values given above with δ = .1, UAH (θ, θ) − UAA (θ, θ) ≈ −0.00076.

Expanded PaperInterface specification

```
have componentLaw : PMF KR21Monoculture.AppendixB1NoiseAtom := KR21Monoculture.appendixB1NoisePMF;
have rawNoiseLaw : PMF KR21Monoculture.AppendixB1NoiseTriple :=
  AppliedModelingLib.pmfProd (AppliedModelingLib.pmfProd componentLaw componentLaw) componentLaw;
have rawRank : KR21Monoculture.AppendixB1NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB1Value c +
      KR21Monoculture.appendixB1NoiseValue (KR21Monoculture.appendixB1NoiseTripleFunction noise c);
have rawLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map rawRank rawNoiseLaw;
(KR21Monoculture.appendixB1Value 0 = 7 / 4 ∧
    KR21Monoculture.appendixB1Value 1 = 1 / 2 ∧ KR21Monoculture.appendixB1Value 2 = 0) ∧
  (KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.plusOne = 1 ∧
      KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.zero = 0 ∧
        KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.minusOne = -1) ∧
    (componentLaw KR21Monoculture.AppendixB1NoiseAtom.plusOne).toReal = 1 / 20 ∧
      (componentLaw KR21Monoculture.AppendixB1NoiseAtom.zero).toReal = 9 / 10 ∧
        (componentLaw KR21Monoculture.AppendixB1NoiseAtom.minusOne).toReal = 1 / 20 ∧
          (∀ (noise : KR21Monoculture.AppendixB1NoiseTriple),
              rawRank noise = KR21Monoculture.appendixB1DiscreteTripleRank noise) ∧
            rawLaw = KR21Monoculture.appendixB1RankingPMF ∧
              AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent rawLaw rawLaw
                    KR21Monoculture.appendixB1Value -
                  AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared rawLaw
                    KR21Monoculture.appendixB1Value =
                -(9749 / 12800000) ∧
              ¬KR21Monoculture.Model.PrefersIndependentReranking rawLaw KR21Monoculture.appendixB1Value
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- AppliedModelingLib.pmfProd
- KR21Monoculture.AppendixB1NoiseAtom

- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.Model.PrefersIndependentReranking
- KR21Monoculture.appendixB1DiscreteTripleRank
- KR21Monoculture.appendixB1NoisePMF
- KR21Monoculture.appendixB1NoiseTripleFunction
- KR21Monoculture.appendixB1NoiseValue
- KR21Monoculture.appendixB1RankingPMF
- KR21Monoculture.appendixB1Value

Lean-elaborated claim roles

```
1. conclusion: (KR21Monoculture.appendixB1Value 0 = 7 / 4 ∧
   KR21Monoculture.appendixB1Value 1 = 1 / 2 ∧ KR21Monoculture.appendixB1Value 2 = 0) ∧
  (KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.plusOne = 1 ∧
     KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.zero = 0 ∧
       KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.minusOne = -1) ∧
    (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.plusOne).toReal = 1 / 20 ∧
     (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.zero).toReal = 9 / 10 ∧
      (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.minusOne).toReal = 1 / 20 ∧
       (∀ (noise : KR21Monoculture.AppendixB1NoiseTriple),
          (fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
               AppliedModelingLib.SocialChoice.Ranking.rankByScore
                fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                KR21Monoculture.appendixB1Value c +
                  KR21Monoculture.appendixB1NoiseValue (KR21Monoculture.appendixB1NoiseTripleFunction noise c))
            noise =
          KR21Monoculture.appendixB1DiscreteTripleRank noise) ∧
        PMF.map
            (fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
              AppliedModelingLib.SocialChoice.Ranking.rankByScore
                fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                KR21Monoculture.appendixB1Value c +
                  KR21Monoculture.appendixB1NoiseValue (KR21Monoculture.appendixB1NoiseTripleFunction noise c))
            (AppliedModelingLib.pmfProd
              (AppliedModelingLib.pmfProd KR21Monoculture.appendixB1NoisePMF KR21Monoculture.appendixB1NoisePMF)
              KR21Monoculture.appendixB1NoisePMF) =
          KR21Monoculture.appendixB1RankingPMF ∧
         AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
            (PMF.map
              (fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
                AppliedModelingLib.SocialChoice.Ranking.rankByScore
                  fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                  KR21Monoculture.appendixB1Value c +
                    KR21Monoculture.appendixB1NoiseValue
                      (KR21Monoculture.appendixB1NoiseTripleFunction noise c))
              (AppliedModelingLib.pmfProd
                (AppliedModelingLib.pmfProd KR21Monoculture.appendixB1NoisePMF
                  KR21Monoculture.appendixB1NoisePMF)
                KR21Monoculture.appendixB1NoisePMF))
            (PMF.map
              (fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
                AppliedModelingLib.SocialChoice.Ranking.rankByScore
                  fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                  KR21Monoculture.appendixB1Value c +
                    KR21Monoculture.appendixB1NoiseValue
                      (KR21Monoculture.appendixB1NoiseTripleFunction noise c))
              (AppliedModelingLib.pmfProd
                (AppliedModelingLib.pmfProd KR21Monoculture.appendixB1NoisePMF
                  KR21Monoculture.appendixB1NoisePMF)
                KR21Monoculture.appendixB1NoisePMF))
            KR21Monoculture.appendixB1Value -
          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
            (PMF.map
              (fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
                AppliedModelingLib.SocialChoice.Ranking.rankByScore
                  fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                  KR21Monoculture.appendixB1Value c +
                    KR21Monoculture.appendixB1NoiseValue
                      (KR21Monoculture.appendixB1NoiseTripleFunction noise c))
              (AppliedModelingLib.pmfProd
                (AppliedModelingLib.pmfProd KR21Monoculture.appendixB1NoisePMF
                  KR21Monoculture.appendixB1NoisePMF)
                KR21Monoculture.appendixB1NoisePMF))
            KR21Monoculture.appendixB1Value =
           -(9749 / 12800000) ∧
          ¬KR21Monoculture.Model.PrefersIndependentReranking
            (PMF.map
              (fun (noise : KR21Monoculture.AppendixB1NoiseTriple) =>
                AppliedModelingLib.SocialChoice.Ranking.rankByScore
                  fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                  KR21Monoculture.appendixB1Value c +
                    KR21Monoculture.appendixB1NoiseValue
                      (KR21Monoculture.appendixB1NoiseTripleFunction noise c))
              (AppliedModelingLib.pmfProd
                (AppliedModelingLib.pmfProd KR21Monoculture.appendixB1NoisePMF
                  KR21Monoculture.appendixB1NoisePMF)
                KR21Monoculture.appendixB1NoisePMF))
            KR21Monoculture.appendixB1Value
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.appendixB1_source_discrete_definition2_counterexample_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target fixes the source values 7/4, 1/2, 0 and delta=.1 three-point iid noise, then states the displayed negative UAH-UAA counterexample.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 17

Source locator: cited publication, lines 190-198

Verbatim source input

Next, we’ll give a 3-candidate RUM for which UAH < UHH does not hold in general. Consider the following 3-candidate example.
x1 = 3
x2 = 2
x3 = 0
Choose E and θ such that

[U+F8F1 delimiter glyph]
1
[U+F8F4 delimiter glyph]
[U+F8F4 delimiter glyph]
[U+F8F4 delimiter glyph]
[U+F8F2 delimiter glyph]
−1
ε
=
θ [U+F8F4 delimiter glyph]
10
[U+F8F4 delimiter glyph]
[U+F8F4 delimiter glyph]
[U+F8F3 delimiter glyph]
−10

w.p. 1−δ
2
w.p. 1−δ
2
w.p. 2δ
w.p. 2δ

Again, while this noise model doesn’t satisfy Definition 1, we can approximate it arbitrarily closely with the sum of tightly concentrated Gaussians. Let the θA = 1.1θ and θH = 0.9θ.
We will show that for these parameters, UAH (θA , θH) > UHH (θA , θH), i.e., it is somehow better to choose

Expanded PaperInterface specification

```
have componentLaw : PMF KR21Monoculture.AppendixB2NoiseAtom := KR21Monoculture.appendixB2NoisePMF;
have rawNoiseLaw : PMF KR21Monoculture.AppendixB2NoiseTriple :=
  AppliedModelingLib.pmfProd (AppliedModelingLib.pmfProd componentLaw componentLaw) componentLaw;
have algorithmRank : KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c +
      10 / 11 * KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c);
have humanRank : KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c +
      10 / 9 * KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c);
have algorithmLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map algorithmRank rawNoiseLaw;
have humanLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map humanRank rawNoiseLaw;
(KR21Monoculture.appendixB2Value 0 = 3 ∧
    KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
  (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
      KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
        KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
          KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
    (componentLaw KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
      (componentLaw KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
        (componentLaw KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
          (componentLaw KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal = 1 / 20 ∧
            (11 / 10 > 9 / 10 ∧ 10 / 11 * (11 / 10) = 1 ∧ 10 / 9 * (9 / 10) = 1) ∧
              (∀ (noise : KR21Monoculture.AppendixB2NoiseTriple),
                  algorithmRank noise = KR21Monoculture.appendixB2AlgorithmDiscreteTripleRank noise) ∧
                (∀ (noise : KR21Monoculture.AppendixB2NoiseTriple),
                    humanRank noise = KR21Monoculture.appendixB2HumanDiscreteTripleRank noise) ∧
                  algorithmLaw = KR21Monoculture.appendixB2AlgorithmRankingPMF ∧
                    humanLaw = KR21Monoculture.appendixB2HumanRankingPMF ∧
                      AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent humanLaw algorithmLaw
                            KR21Monoculture.appendixB2Value -
                          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent humanLaw humanLaw
                            KR21Monoculture.appendixB2Value =
                        567 / 3200000 ∧
                      ¬KR21Monoculture.Model.PrefersWeakerCompetition algorithmLaw humanLaw
                          KR21Monoculture.appendixB2Value
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.rankByScore

- AppliedModelingLib.pmfProd
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.Model.PrefersWeakerCompetition
- KR21Monoculture.appendixB2AlgorithmDiscreteTripleRank
- KR21Monoculture.appendixB2AlgorithmRankingPMF
- KR21Monoculture.appendixB2HumanDiscreteTripleRank
- KR21Monoculture.appendixB2HumanRankingPMF
- KR21Monoculture.appendixB2NoisePMF
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixB2Value

Lean-elaborated claim roles

```
1. conclusion: (KR21Monoculture.appendixB2Value 0 = 3 ∧
    KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
  (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
     KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
      KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
       KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
    (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
     (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
      (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
       (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal = 1 / 20 ∧
        (11 / 10 > 9 / 10 ∧ 10 / 11 * (11 / 10) = 1 ∧ 10 / 9 * (9 / 10) = 1) ∧
         (∀ (noise : KR21Monoculture.AppendixB2NoiseTriple),
            (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                 AppliedModelingLib.SocialChoice.Ranking.rankByScore
                   fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                   KR21Monoculture.appendixB2Value c +
                     10 / 11 *
                       KR21Monoculture.appendixB2NoiseValue
                         (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
               noise =
             KR21Monoculture.appendixB2AlgorithmDiscreteTripleRank noise) ∧
           (∀ (noise : KR21Monoculture.AppendixB2NoiseTriple),
              (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                   AppliedModelingLib.SocialChoice.Ranking.rankByScore
                     fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                     KR21Monoculture.appendixB2Value c +
                       10 / 9 *
                         KR21Monoculture.appendixB2NoiseValue
                           (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
                 noise =
               KR21Monoculture.appendixB2HumanDiscreteTripleRank noise) ∧
             PMF.map
                 (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                   AppliedModelingLib.SocialChoice.Ranking.rankByScore
                     fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                     KR21Monoculture.appendixB2Value c +
                       10 / 11 *
                         KR21Monoculture.appendixB2NoiseValue
                           (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
                 (AppliedModelingLib.pmfProd
                   (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
                     KR21Monoculture.appendixB2NoisePMF)
                   KR21Monoculture.appendixB2NoisePMF) =
               KR21Monoculture.appendixB2AlgorithmRankingPMF ∧
              PMF.map
                 (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                   AppliedModelingLib.SocialChoice.Ranking.rankByScore
                     fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                     KR21Monoculture.appendixB2Value c +
                       10 / 9 *
                         KR21Monoculture.appendixB2NoiseValue
                           (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
                 (AppliedModelingLib.pmfProd
                   (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
                     KR21Monoculture.appendixB2NoisePMF)
                   KR21Monoculture.appendixB2NoisePMF) =
               KR21Monoculture.appendixB2HumanRankingPMF ∧
             AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                 (PMF.map
                   (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                     AppliedModelingLib.SocialChoice.Ranking.rankByScore
                       fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                       KR21Monoculture.appendixB2Value c +
                         10 / 9 *
                           KR21Monoculture.appendixB2NoiseValue
                             (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
                   (AppliedModelingLib.pmfProd
                     (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
                       KR21Monoculture.appendixB2NoisePMF)
                     KR21Monoculture.appendixB2NoisePMF)
```

```
      KR21Monoculture.appendixB2NoisePMF))
    (PMF.map
     (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
       fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
       KR21Monoculture.appendixB2Value c +
        10 / 11 *
         KR21Monoculture.appendixB2NoiseValue
          (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
     (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
       KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF))
   KR21Monoculture.appendixB2Value -
  AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
    (PMF.map
     (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
       fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
       KR21Monoculture.appendixB2Value c +
        10 / 9 *
         KR21Monoculture.appendixB2NoiseValue
          (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
     (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
       KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF))
    (PMF.map
     (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
       fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
       KR21Monoculture.appendixB2Value c +
        10 / 9 *
         KR21Monoculture.appendixB2NoiseValue
          (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
     (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
       KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF))
    KR21Monoculture.appendixB2Value =
 567 / 3200000 ∧
¬KR21Monoculture.Model.PrefersWeakerCompetition
   (PMF.map
    (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
     AppliedModelingLib.SocialChoice.Ranking.rankByScore
      fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
      KR21Monoculture.appendixB2Value c +
       10 / 11 *
        KR21Monoculture.appendixB2NoiseValue
         (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
    (AppliedModelingLib.pmfProd
     (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
      KR21Monoculture.appendixB2NoisePMF)
     KR21Monoculture.appendixB2NoisePMF))
   (PMF.map
    (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
     AppliedModelingLib.SocialChoice.Ranking.rankByScore
      fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
      KR21Monoculture.appendixB2Value c +
       10 / 9 *
        KR21Monoculture.appendixB2NoiseValue
         (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
    (AppliedModelingLib.pmfProd
     (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
      KR21Monoculture.appendixB2NoisePMF)
     KR21Monoculture.appendixB2NoisePMF))
   KR21Monoculture.appendixB2Value
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.appendixB2_source_discrete_definition3_counterexample_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: The values 3,2,0, source four-atom probabilities, thetaA=1.1 theta and thetaH=.9 theta scaling, and positive UAH-UHH witness agree with the source example.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 18

Source locator: cited publication, lines 1040-1057

Verbatim source input

Let τ and π be rankings generated by the algorithm and human evaluator respectively. First, we will
show that
Pr[τ1 = x1] = Pr[π1 = x1]

(B.1)

Expanded PaperInterface specification

```
have componentLaw : PMF KR21Monoculture.AppendixB2NoiseAtom := KR21Monoculture.appendixB2NoisePMF;
have rawNoiseLaw : PMF KR21Monoculture.AppendixB2NoiseTriple :=
  AppliedModelingLib.pmfProd (AppliedModelingLib.pmfProd componentLaw componentLaw) componentLaw;
have algorithmRank : KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c +
      10 / 11 * KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c);
have humanRank : KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c +
      10 / 9 * KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c);
have algorithmLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map algorithmRank rawNoiseLaw;
have humanLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map humanRank rawNoiseLaw;
(KR21Monoculture.appendixB2Value 0 = 3 ∧
    KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
  (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
      KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
        KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
          KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
    (componentLaw KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
      (componentLaw KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
        (componentLaw KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
          (componentLaw KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal = 1 / 20 ∧
            algorithmLaw = KR21Monoculture.appendixB2AlgorithmRankingPMF ∧
              humanLaw = KR21Monoculture.appendixB2HumanRankingPMF ∧
                KR21Monoculture.firstChoiceProb algorithmLaw 0 = KR21Monoculture.firstChoiceProb humanLaw 0
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- AppliedModelingLib.pmfProd
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.appendixB2AlgorithmRankingPMF
- KR21Monoculture.appendixB2HumanRankingPMF
- KR21Monoculture.appendixB2NoisePMF
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixB2Value
- KR21Monoculture.firstChoiceProb

Lean-elaborated claim roles

1. conclusion:

```
(KR21Monoculture.appendixB2Value 0 = 3 ∧
    KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
  (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
      KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
        KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
          KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
    (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
      (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
        (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
          (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal = 1 / 20 ∧
            PMF.map
                (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                  AppliedModelingLib.SocialChoice.Ranking.rankByScore
                    fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
```

```
      KR21Monoculture.appendixB2Value c +
        10 / 11 *
          KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
    (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF) =
  KR21Monoculture.appendixB2AlgorithmRankingPMF ∧
PMF.map
    (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
        fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
        KR21Monoculture.appendixB2Value c +
          10 / 9 *
            KR21Monoculture.appendixB2NoiseValue
              (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
    (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF) =
  KR21Monoculture.appendixB2HumanRankingPMF ∧
 KR21Monoculture.firstChoiceProb
    (PMF.map
      (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore
          fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
          KR21Monoculture.appendixB2Value c +
            10 / 11 *
              KR21Monoculture.appendixB2NoiseValue
                (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
      (AppliedModelingLib.pmfProd
        (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
          KR21Monoculture.appendixB2NoisePMF)
        KR21Monoculture.appendixB2NoisePMF))
    0 =
  KR21Monoculture.firstChoiceProb
    (PMF.map
      (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore
          fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
          KR21Monoculture.appendixB2Value c +
            10 / 9 *
              KR21Monoculture.appendixB2NoiseValue
                (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
      (AppliedModelingLib.pmfProd
        (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
          KR21Monoculture.appendixB2NoisePMF)
        KR21Monoculture.appendixB2NoisePMF))
    0
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationB1_counterexample_first_choice_x1_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For the source Appendix B.2 four-atom experiment, the target states exactly Pr[tau_1=x1]=Pr[pi_1=x1].

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 19

Source locator: cited publication, lines 1044-1105

Verbatim source input

Pr[τ1 = x2] > Pr[π1 = x2]

(B.2)

Expanded PaperInterface specification

```
have componentLaw : PMF KR21Monoculture.AppendixB2NoiseAtom := KR21Monoculture.appendixB2NoisePMF;
have rawNoiseLaw : PMF KR21Monoculture.AppendixB2NoiseTriple :=
  AppliedModelingLib.pmfProd (AppliedModelingLib.pmfProd componentLaw componentLaw) componentLaw;
have algorithmRank : KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c +
      10 / 11 * KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c);
have humanRank : KR21Monoculture.AppendixB2NoiseTriple → AppliedModelingLib.SocialChoice.Ranking.Ranking 1 :=
  fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
  AppliedModelingLib.SocialChoice.Ranking.rankByScore fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
    KR21Monoculture.appendixB2Value c +
      10 / 9 * KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c);
have algorithmLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map algorithmRank rawNoiseLaw;
have humanLaw : PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1) := PMF.map humanRank rawNoiseLaw;
(KR21Monoculture.appendixB2Value 0 = 3 ∧
    KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
  (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
      KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
        KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
          KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
    (componentLaw KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
      (componentLaw KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
        (componentLaw KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
          (componentLaw KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal = 1 / 20 ∧
            algorithmLaw = KR21Monoculture.appendixB2AlgorithmRankingPMF ∧
              humanLaw = KR21Monoculture.appendixB2HumanRankingPMF ∧
                KR21Monoculture.firstChoiceProb humanLaw 1 < KR21Monoculture.firstChoiceProb algorithmLaw 1
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- AppliedModelingLib.pmfProd
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.appendixB2AlgorithmRankingPMF
- KR21Monoculture.appendixB2HumanRankingPMF
- KR21Monoculture.appendixB2NoisePMF
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixB2Value
- KR21Monoculture.firstChoiceProb

Lean-elaborated claim roles

1. conclusion:

```
(KR21Monoculture.appendixB2Value 0 = 3 ∧
    KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
  (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
      KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
        KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
          KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
    (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
      (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
        (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
          (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal = 1 / 20 ∧
            PMF.map
                (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
                  AppliedModelingLib.SocialChoice.Ranking.rankByScore
                    fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                    KR21Monoculture.appendixB2Value c +
                      10 / 11 *
```

```
          KR21Monoculture.appendixB2NoiseValue (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
    (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF) =
  KR21Monoculture.appendixB2AlgorithmRankingPMF ∧
PMF.map
    (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
        fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
        KR21Monoculture.appendixB2Value c +
          10 / 9 *
            KR21Monoculture.appendixB2NoiseValue
              (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
    (AppliedModelingLib.pmfProd
      (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF KR21Monoculture.appendixB2NoisePMF)
      KR21Monoculture.appendixB2NoisePMF) =
  KR21Monoculture.appendixB2HumanRankingPMF ∧
 KR21Monoculture.firstChoiceProb
    (PMF.map
      (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore
          fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
          KR21Monoculture.appendixB2Value c +
            10 / 9 *
              KR21Monoculture.appendixB2NoiseValue
                (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
      (AppliedModelingLib.pmfProd
        (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
          KR21Monoculture.appendixB2NoisePMF)
        KR21Monoculture.appendixB2NoisePMF))
    1 <
   KR21Monoculture.firstChoiceProb
    (PMF.map
      (fun (noise : KR21Monoculture.AppendixB2NoiseTriple) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore
          fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
          KR21Monoculture.appendixB2Value c +
            10 / 11 *
              KR21Monoculture.appendixB2NoiseValue
                (KR21Monoculture.appendixB2NoiseTripleFunction noise c))
      (AppliedModelingLib.pmfProd
        (AppliedModelingLib.pmfProd KR21Monoculture.appendixB2NoisePMF
          KR21Monoculture.appendixB2NoisePMF)
        KR21Monoculture.appendixB2NoisePMF))
    1
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationB2_counterexample_first_choice_x2_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For the same source experiment, the target states exactly Pr[tau_1=x2]>Pr[pi_1=x2].

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 20

Source locator: cited publication, lines 1111-1147

Verbatim source input

Thus, it suffices to show that for any a,
Pr[Xi > Xj] > Pr[Xi > Xj | Xi < a, Xj < a].

(C.1)

Expanded PaperInterface specification

```
∀ {theta xi xj a : ℝ},
  0 < theta →
    xj < xi →
      have laplace : MeasureTheory.Measure ℝ :=
        MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|));
      have gaussian : MeasureTheory.Measure ℝ := ProbabilityTheory.gaussianReal 0 1;
      have laplaceDenominator : ℝ :=
        ((laplace.prod laplace) {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal;
      have gaussianDenominator : ℝ :=
        ((gaussian.prod gaussian) {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal;
      (0 < laplaceDenominator ∧
          ((laplace.prod laplace)
                {epsilon : ℝ × ℝ |
                  xi + epsilon.1 / theta < a ∧
                    xj + epsilon.2 / theta < a ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
              laplaceDenominator <
            ((laplace.prod laplace) {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal) ∧
        0 < gaussianDenominator ∧
          ((gaussian.prod gaussian)
                {epsilon : ℝ × ℝ |
                  xi + epsilon.1 / theta < a ∧
                    xj + epsilon.2 / theta < a ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
              gaussianDenominator <
            ((gaussian.prod gaussian) {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal
```

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 0 < theta
6. assumption: xj < xi
7. conclusion: (0 <
      (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
            (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
          {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal ∧
    (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
            (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
          {epsilon : ℝ × ℝ |
            xi + epsilon.1 / theta < a ∧
              xj + epsilon.2 / theta < a ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
        (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
            (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
          {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal <
      (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
            (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
          {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal) ∧
  0 <
      (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
          {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal ∧
    (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
          {epsilon : ℝ × ℝ |
            xi + epsilon.1 / theta < a ∧
              xj + epsilon.2 / theta < a ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
        (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
          {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal <
      (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
          {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.appendixC1_source_pairwise_full_events

Recorded source-to-Spec screening **Verdict:** matches

Reason: For xj<xi and positive scale, the target writes the source full winner event and the jointly truncated conditional event, with the required strict conditional loss for Laplace and Gaussian noise.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 21

Source locator: cited publication, lines 1139-1147

Verbatim source input

Since Pr[Xi > Xj] = lima→∞ Pr[Xi > Xj | Xi < a, Xj < a], it suffices to show that for all a,
d
Pr[Xi > Xj | Xi < a, Xj < a] ≥ 0,
(C.2)
da

Expanded PaperInterface specification

```
∀ {theta xi xj : ℝ},
  0 < theta →
    xj < xi →
      have laplace : MeasureTheory.Measure ℝ :=
        MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|));
      have gaussian : MeasureTheory.Measure ℝ := ProbabilityTheory.gaussianReal 0 1;
      have laplaceRatio : ℝ → ℝ := fun (u : ℝ) =>
        ((laplace.prod laplace)
            {epsilon : ℝ × ℝ |
              xi + epsilon.1 / theta < u ∧
                xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
          ((laplace.prod laplace) {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal;
      have gaussianRatio : ℝ → ℝ := fun (u : ℝ) =>
        ((gaussian.prod gaussian)
            {epsilon : ℝ × ℝ |
              xi + epsilon.1 / theta < u ∧
                xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
          ((gaussian.prod gaussian) {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal;
      have laplaceWinner : ℝ :=
        ((laplace.prod laplace) {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal;
      have gaussianWinner : ℝ :=
        ((gaussian.prod gaussian) {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal;
      ((∀ (a : ℝ),
          0 <
            ((laplace.prod laplace)
                {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal ∧
              ∃ (d : ℝ), HasDerivAt laplaceRatio d a ∧ 0 ≤ d) ∧
        (∃ (a : ℝ) (d : ℝ), HasDerivAt laplaceRatio d a ∧ 0 < d) ∧
          Filter.Tendsto laplaceRatio Filter.atTop (nhds laplaceWinner)) ∧
        (∀ (a : ℝ),
          0 <
            ((gaussian.prod gaussian)
                {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal ∧
              ∃ (d : ℝ), HasDerivAt gaussianRatio d a ∧ 0 < d) ∧
          Filter.Tendsto gaussianRatio Filter.atTop (nhds gaussianWinner)
```

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. assumption: 0 < theta
5. assumption: xj < xi
6. conclusion: ((∀ (a : ℝ),
    0 <
      (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
          (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
        {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal ∧
      ∃ (d : ℝ),
        HasDerivAt
          (fun (u : ℝ) =>
            (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
                  (MeasureTheory.volume.withDensity fun (z : ℝ) =>
                    ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
                {epsilon : ℝ × ℝ |
                  xi + epsilon.1 / theta < u ∧
                    xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
              (((MeasureTheory.volume.withDensity fun (z : ℝ) =>
                      ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
                  (MeasureTheory.volume.withDensity fun (z : ℝ) =>
                    ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
                {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal)
          d a ∧
        0 ≤ d) ∧
  (∃ (a : ℝ) (d : ℝ),
    HasDerivAt
      (fun (u : ℝ) =>
        (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
              (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
            {epsilon : ℝ × ℝ |
              xi + epsilon.1 / theta < u ∧
                xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
          (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
              (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
            {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal)
      d a ∧
    0 < d) ∧
  Filter.Tendsto
```

```
      (fun (u : ℝ) =>
        (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
              (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
            {epsilon : ℝ × ℝ |
              xi + epsilon.1 / theta < u ∧
                xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
          (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
              (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
            {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal)
      Filter.atTop
      (nhds
        (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
              (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
            {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal)) ∧
  (∀ (a : ℝ),
    0 <
        (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
            {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < a ∧ xj + epsilon.2 / theta < a}).toReal ∧
      ∃ (d : ℝ),
        HasDerivAt
          (fun (u : ℝ) =>
            (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
                {epsilon : ℝ × ℝ |
                  xi + epsilon.1 / theta < u ∧
                    xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
              (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
                {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal)
          d a ∧
        0 < d) ∧
    Filter.Tendsto
      (fun (u : ℝ) =>
        (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
            {epsilon : ℝ × ℝ |
              xi + epsilon.1 / theta < u ∧
                xj + epsilon.2 / theta < u ∧ xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal /
          (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
            {epsilon : ℝ × ℝ | xi + epsilon.1 / theta < u ∧ xj + epsilon.2 / theta < u}).toReal)
      Filter.atTop
      (nhds
        (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
          {epsilon : ℝ × ℝ | xj + epsilon.2 / theta < xi + epsilon.1 / theta}).toReal)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.appendixC2_source_pairwise_full_events

Recorded source-to-Spec screening **Verdict:** matches

Reason: The expanded Laplace and Gaussian truncated winner ratios have positive denominators, the source derivative signs, strict witnesses where stated, and their untruncated limits.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 22

Source locator: cited publication, lines 217-242

Verbatim source input

Theorem 6. For 3 candidates with unique values x1 > x2 > x3 and well-ordered i.i.d. noise with support
(−∞, ∞), if θA > θH , then UAH (θA , θH) < UHH (θA , θH).

Expanded PaperInterface specification

```
∀ (f : ℝ → ℝ) {thetaA thetaH x1 x2 x3 : ℝ},
  Measurable f →
    (∀ {{a b c d : ℝ}}, b < a → d < c → f (a - c) * f (b - d) > f (a - d) * f (b - c)) →
      (∀ (z : ℝ), 0 ≤ f z) →
        (∀ (z : ℝ), 0 < f z) →
          ∫⁻ (z : ℝ), ENNReal.ofReal (f z) = 1 →
            0 < thetaH →
              thetaH < thetaA →
                x2 < x1 →
                  x3 < x2 →
                    MeasureTheory.IsProbabilityMeasure
                        (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                          MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
                      ∃ (rankingLaw : ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)),
                        (∀ (theta : ℝ),
                            0 < theta →
                              (rankingLaw theta).toMeasure =
                                MeasureTheory.Measure.map
                                  (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ) =>
                                    AppliedModelingLib.SocialChoice.Ranking.rankByScore
                                      fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                                      KR21Monoculture.threeCandidateValueProfile x1 x2 x3 i + epsilon i / theta)
                                  (MeasureTheory.Measure.pi
                                    fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                                    MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z))) ∧
                          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (rankingLaw thetaH)
                            (rankingLaw thetaA) (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) <
                          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (rankingLaw thetaH)
                            (rankingLaw thetaH) (KR21Monoculture.threeCandidateValueProfile x1 x2 x3)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.threeCandidateValueProfile

Lean-elaborated claim roles

1. parameter: ℝ → ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. parameter: ℝ
6. parameter: ℝ
7. assumption: Measurable f
8. assumption: ∀ {{a b c d : ℝ}}, b < a → d < c → f (a - c) * f (b - d) > f (a - d) * f (b - c)
9. assumption: ∀ (z : ℝ), 0 ≤ f z
10. assumption: ∀ (z : ℝ), 0 < f z
11. assumption: ∫⁻ (z : ℝ), ENNReal.ofReal (f z) = 1
12. assumption: 0 < thetaH
13. assumption: thetaH < thetaA
14. assumption: x2 < x1
15. assumption: x3 < x2
16. conclusion: MeasureTheory.IsProbabilityMeasure
    (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
      MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
  ∃ (rankingLaw : ℝ → PMF (AppliedModelingLib.SocialChoice.Ranking.Ranking 1)),
    (∀ (theta : ℝ),
      0 < theta →
        (rankingLaw theta).toMeasure =
          MeasureTheory.Measure.map
            (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate 1 → ℝ) =>
              AppliedModelingLib.SocialChoice.Ranking.rankByScore
                fun (i : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                KR21Monoculture.threeCandidateValueProfile x1 x2 x3 i + epsilon i / theta)
            (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
              MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z))) ∧
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (rankingLaw thetaH) (rankingLaw thetaA)
      (KR21Monoculture.threeCandidateValueProfile x1 x2 x3) <
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent (rankingLaw thetaH) (rankingLaw thetaH)
      (KR21Monoculture.threeCandidateValueProfile x1 x2 x3)


```
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem6_source_raw_iid_density_literal_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For three strictly ordered values, full-support strictly well-ordered iid density noise, and thetaA>thetaH, the target is source Theorem 6's UAH<UHH conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 23

Source locator: cited publication, lines 1293-1329

Verbatim source input

Lemma 1. Both Gaussian and Laplacian distributions are well-ordered.

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For every kappa > 0 and lambda > 0, the Gaussian kernel G_kappa(r)=exp(-kappa*r^2) satisfies G_kappa(a-c)G_kappa(b-d) > G_kappa(a-d)G_kappa(b-c) for all a>b and c>d. The Laplace kernel L_lambda(r)=exp(-lambda*|r|) satisfies the corresponding >= inequality globally, does not satisfy it strictly for every such quadruple, and satisfies the strict inequality on the explicit overlap domain b<a, d<c, b<c, and d<a.

Archival source anchor: cited publication, lines 1293-1329

Expanded PaperInterface specification

```
∀ {kappa lam : ℝ},
  0 < kappa →
    0 < lam →
      (∀ {{a b c d : ℝ}},
         b < a →
           d < c →
             Real.exp (-kappa * (a - c) ^ 2) * Real.exp (-kappa * (b - d) ^ 2) >
               Real.exp (-kappa * (a - d) ^ 2) * Real.exp (-kappa * (b - c) ^ 2)) ∧
       (∀ {{a b c d : ℝ}},
          b < a →
            d < c →
              Real.exp (-lam * |a - d|) * Real.exp (-lam * |b - c|) ≤
                Real.exp (-lam * |a - c|) * Real.exp (-lam * |b - d|)) ∧
        (¬∀ {{a b c d : ℝ}},
           b < a →
             d < c →
               Real.exp (-lam * |a - c|) * Real.exp (-lam * |b - d|) >
                 Real.exp (-lam * |a - d|) * Real.exp (-lam * |b - c|)) ∧
          ∀ {a b c d : ℝ},
            b < a →
              d < c →
                b < c →
                  d < a →
                    Real.exp (-lam * |a - c|) * Real.exp (-lam * |b - d|) >
                      Real.exp (-lam * |a - d|) * Real.exp (-lam * |b - c|)
```

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. assumption: 0 < kappa
4. assumption: 0 < lam
5. conclusion: (∀ {{a b c d : ℝ}},
    b < a →
      d < c →
        Real.exp (-kappa * (a - c) ^ 2) * Real.exp (-kappa * (b - d) ^ 2) >
          Real.exp (-kappa * (a - d) ^ 2) * Real.exp (-kappa * (b - c) ^ 2)) ∧
   (∀ {{a b c d : ℝ}},
      b < a →
        d < c →
          Real.exp (-lam * |a - d|) * Real.exp (-lam * |b - c|) ≤
            Real.exp (-lam * |a - c|) * Real.exp (-lam * |b - d|)) ∧
    (¬∀ {{a b c d : ℝ}},
       b < a →
         d < c →
           Real.exp (-lam * |a - c|) * Real.exp (-lam * |b - d|) >
             Real.exp (-lam * |a - d|) * Real.exp (-lam * |b - c|)) ∧
      ∀ {a b c d : ℝ},
        b < a →
          d < c →
            b < c →
              d < a →
                Real.exp (-lam * |a - c|) * Real.exp (-lam * |b - d|) >
                  Real.exp (-lam * |a - d|) * Real.exp (-lam * |b - c|)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma1_corrected_gaussian_laplace_kernel_target

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The target is the recorded approved correction: strict Gaussian ordering, global weak Laplace ordering, failure of global strict Laplace ordering, and strict Laplace ordering on the explicit overlap domain.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 24

Source locator: cited publication, lines 1340-1364

Verbatim source input

Lemma 2. If Fθ is a RUM family satisfying Definition 1, then for τ ∼ FθA and π ∼ FθH ,
Pr[τ1 = xn] ≤ Pr[π1 = xn]

[Next verbatim source excerpt]

Each firm in our model has access to the same underlying pool of n candidates, which they rank using a randomized mechanism R to get a permutation π as described above. Then, in a random order, each firm hires the highest-ranked remaining candidate according to their ranking. Thus, if two firms both rank candidate i first, only one of them can hire i; the other must hire the next available candidate according to their ranking. In our model, candidates automatically accept the offer they get from a firm. For the sake of simplicity, throughout this paper, we restrict ourselves to the case where there are two firms hiring one candidate each, although our model readily generalizes to more complex cases.

[Next verbatim source excerpt]

In Random Utility Models, the underlying candidate values xi are perturbed by independent and identically distributed noise εi ∼ E, and the perturbed values are ranked to produce π. Originally conceived in the psychology literature [23], this model has been well-studied over nearly a century, [7, 4, 9, 24, 16, 22], including more recently in the computer science and machine learning literature [2, 1, 19, 25, 13].
First, we must define a family of RUMs that satisfies the conditions of Definition 1. Assume without loss of generality that the noise distribution E has unit variance. Then, consider the family of RUMs parameterized by θ in which candidates are ranked according to xi + εθi . By this definition, the standard deviation of the noise for a particular value of θ is simply 1/θ. Intuitively, larger values of θ reduce the effect of the noise, making the ranking more accurate. In Theorem 5 in Appendix A, we show as long as the noise distribution E has positive support on (−∞, ∞), this definition of Fθ meets the differentiability, asymptotic optimality, and monotonicity conditions in Definition 1. For distributions with finite support, many of our results can

[Next verbatim source excerpt]

Next, we show a few basic facts. Let fA (r) be the density function of the joint realization R = [X1 , . . . , Xn] = [x1 +ε1 /θA , . . . , xn +εn /θA] under the algorithmic ranking and fH (r) be the similarly defined density function under the human-generated ranking. Consider the “contraction” operation r0 = cont(r) such that ri0 = xi + (ri − xi) · θθH
. Essentially, the contraction defines a coupling between fA (·) and fH (·), since
A
for r0 = cont(r), fA (r0) dr0 = fH (r) dr. Let π(r) be the ranking induced by r. Note that contraction cannot introduce any new inversions in π(r)—that is, if i is ranked above j in π(r) for i < j, then i is ranked above j in π(cont(r)). Intuitively, this is because contraction pulls values closer to their means, and can therefore only correct existing inversions, not introduce new ones. This fact will allow us to prove some useful lemmas.
Lemma 2. If Fθ is a RUM family satisfying Definition 1, then for τ ∼ FθA and π ∼ FθH ,
Pr[τ1 = xn] ≤ Pr[π1 = xn]

Proof. Consider any realization r. Because inversions can only be corrected, not generated, by contraction, if π1 (r0) = n, then π1 (r) = n where r0 = cont(r). Since r0 and r have equal measure under fA and fH respectively, we have
Z
Pr[π1 = xn] =
fH (r)1π1 (r)=xn dr
n
ZR
=
fA (cont(r))1π1 (r)=xn d cont(r)
n
ZR
≥
fA (cont(r))1π1 (cont(r))=xn d cont(r)
Rn
Z
=
fA (r)1π1 (r)=xn dr
Rn

= Pr[τ1 = xn]

Expanded PaperInterface specification

```
∀ {n : ℕ} {f : ℝ → ℝ},
  Measurable f →
    (∀ (z : ℝ), 0 ≤ f z) →
      ∀ (hnormalized : ∫⁻ (z : ℝ), ENNReal.ofReal (f z) = 1)
        (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
        StrictAnti value →
          ∀ {thetaA thetaH : ℝ},
            0 < thetaH →
              thetaH < thetaA →
                (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value
                      thetaA).toMeasure =
                  MeasureTheory.Measure.map
                    (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                      AppliedModelingLib.SocialChoice.Ranking.rankByScore
                        fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => value c + epsilon c / thetaA)
                    (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                      MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
                (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value
                      thetaH).toMeasure =
                  MeasureTheory.Measure.map
                    (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                      AppliedModelingLib.SocialChoice.Ranking.rankByScore
                        fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
```

```
      value c + epsilon c / thetaH)
    (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
     MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
  KR21Monoculture.firstChoiceProb
    (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f)
     value thetaA)
    (Fin.last (n + 1)) ≤
   KR21Monoculture.firstChoiceProb
    (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f)
     value thetaH)
    (Fin.last (n + 1))
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.firstChoiceProb
- KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
- KR21Monoculture.w11CandidateNoiseLaw

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: ℝ → ℝ
3. assumption: Measurable f
4. assumption: ∀ (z : ℝ), 0 ≤ f z
5. assumption: ∫⁻ (z : ℝ), ENNReal.ofReal (f z) = 1
6. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
7. assumption: StrictAnti value
8. parameter: ℝ
9. parameter: ℝ
10. assumption: 0 < thetaH
11. assumption: thetaH < thetaA
12. conclusion: (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value
      thetaA).toMeasure =
    MeasureTheory.Measure.map
     (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
       fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => value c + epsilon c / thetaA)
     (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
      MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
   (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value
      thetaH).toMeasure =
    MeasureTheory.Measure.map
     (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
       fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => value c + epsilon c / thetaH)
     (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
      MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
    KR21Monoculture.firstChoiceProb
     (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value thetaA)
     (Fin.last (n + 1)) ≤
    KR21Monoculture.firstChoiceProb
     (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value thetaH)
     (Fin.last (n + 1))
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma2_source_arbitraryFinite_iid_bottom_first_probability_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For the displayed iid scaled RUM and thetaA>thetaH, the target is Lemma 2’s source inequality Pr[tau_1=x_n] <= Pr[pi_1=x_n].

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 25

Source locator: cited publication, lines 1366-1464

Verbatim source input

Lemma 3. For any i > 1, if the noise model E is well-ordered, for θA ≥ θH , τ ∼ FθA , and π ∼ FθH ,
Pr[τ1 = x1] − Pr[π1 = x1] ≥ Pr[τ1 = xi] − Pr[π1 = xi]

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:
Definition 1 (Noisy permutation family). A noisy permutation family Fθ is a family of distributions over permutations that satisfies the following conditions for any θ > 0 and set of candidates x:

[Next verbatim source excerpt]

In Random Utility Models, the underlying candidate values xi are perturbed by independent and identically distributed noise εi ∼ E, and the perturbed values are ranked to produce π. Originally conceived in the psychology literature [23], this model has been well-studied over nearly a century, [7, 4, 9, 24, 16, 22], including more recently in the computer science and machine learning literature [2, 1, 19, 25, 13].
First, we must define a family of RUMs that satisfies the conditions of Definition 1. Assume without loss of generality that the noise distribution E has unit variance. Then, consider the family of RUMs parameterized by θ in which candidates are ranked according to xi + εθi . By this definition, the standard deviation of the noise for a particular value of θ is simply 1/θ. Intuitively, larger values of θ reduce the effect of the noise, making the ranking more accurate. In Theorem 5 in Appendix A, we show as long as the noise distribution E has positive support on (−∞, ∞), this definition of Fθ meets the differentiability, asymptotic optimality, and monotonicity conditions in Definition 1. For distributions with finite support, many of our results can

[Next verbatim source excerpt]

Next, we show a few basic facts. Let fA (r) be the density function of the joint realization R = [X1 , . . . , Xn] = [x1 +ε1 /θA , . . . , xn +εn /θA] under the algorithmic ranking and fH (r) be the similarly defined density function under the human-generated ranking. Consider the “contraction” operation r0 = cont(r) such that ri0 = xi + (ri − xi) · θθH
. Essentially, the contraction defines a coupling between fA (·) and fH (·), since
A
for r0 = cont(r), fA (r0) dr0 = fH (r) dr. Let π(r) be the ranking induced by r. Note that contraction cannot introduce any new inversions in π(r)—that is, if i is ranked above j in π(r) for i < j, then i is ranked above j in π(cont(r)). Intuitively, this is because contraction pulls values closer to their means, and can therefore only correct existing inversions, not introduce new ones. This fact will allow us to prove some useful lemmas.

[Next verbatim source excerpt]

Lemma 3. For any i > 1, if the noise model E is well-ordered, for θA ≥ θH , τ ∼ FθA , and π ∼ FθH ,
Pr[τ1 = x1] − Pr[π1 = x1] ≥ Pr[τ1 = xi] − Pr[π1 = xi]

20

[form-feed in source extraction]
Proof. For j 6= i, let Sj→i ⊆ Rn be the set of realizations r such that π1 (r) = xj and π1 (cont(r)) = xi . Note that Sj→i = ∅ for j < i because contraction cannot create inversions. Then, we have that
XZ
XZ
XZ
Pr[τ1 = xi] − Pr[π1 = xi] =
fH (r)1r∈Sj→i dr −
fH (r)1r∈Si→j dr ≤
fH (r)1r∈Sj→i dr
j>i

Rn

j<i

Rn

j>i

Rn

Define
swapi (r) = r0 ,
where

[U+F8F1 delimiter glyph]
[U+F8F4 delimiter glyph]
[U+F8F2 delimiter glyph]rj
rj0 = r1
[U+F8F4 delimiter glyph]
[U+F8F3 delimiter glyph]
ri

j∈
/ {1, i}
j=i
j=1

Intuitively, the swapi operation simply swaps the realizations in positions 1 and i. Note that this is a bijection. Also, if r ∈ Sj→i , then swapi (r) ∈ Sj→1 , since
cont(swapi (r))1 ≥ cont(r)i ≥ max cont(r)j ≥ max cont(swapi (r))j
j

j ∈{1,i}
/

cont(swapi (r))1 ≥ cont(r)i ≥ cont(r)1 ≥ cont(swapi (r))i
Furthermore for r ∈ Sj→i , fH (r) ≤ fH (swapi (r)) since
f (ri − x1)f (r1 − xi)
fH (swapi (r))
=
≥1
fH (r)
f (r1 − x1)f (ri − xi)
because the noise is well-ordered and r ∈ Sj→i implies ri > r1 . Thus,
XZ
XZ
fH (r)1r∈Sj→i dr ≤
fH (swapi (r))1r∈Sj→i dr
j>i

Rn

j>i

≤

j>i

≤

Rn

XZ
j>i

≤

Rn

XZ

Rn

XZ
j>1

Rn

fH (swapi (r))1swapi (r)∈Sj→1 dr
fH (r)1r∈Sj→1 dr
fH (r)1r∈Sj→1 dr

= Pr[τ1 = x1] − Pr[π1 = x1]

Expanded PaperInterface specification

```
∀ {n : ℕ} {f : ℝ → ℝ},
  KR21Monoculture.StrictlyWellOrderedNoise f →
    Measurable f →
      (∀ (z : ℝ), 0 ≤ f z) →
        ∀ (hnormalized : ∫⁻ (z : ℝ), ENNReal.ofReal (f z) = 1)
          (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
          StrictAnti value →
            ∀ {thetaA thetaH : ℝ},
              0 < thetaH →
                thetaH ≤ thetaA →
                  ∀ {i : AppliedModelingLib.SocialChoice.Ranking.Candidate n},
                    i ≠ 0 →
                      (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f)
                            value thetaA).toMeasure =
                        MeasureTheory.Measure.map
                          (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                            AppliedModelingLib.SocialChoice.Ranking.rankByScore
                              fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                              value c + epsilon c / thetaA)
                          (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                            MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
                        (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f)
                              value thetaH).toMeasure =
                          MeasureTheory.Measure.map
                            (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
                              AppliedModelingLib.SocialChoice.Ranking.rankByScore
                                fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                                value c + epsilon c / thetaH)
                            (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                              MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
                          KR21Monoculture.firstChoiceProb
                              (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
                                (KR21Monoculture.w11CandidateNoiseLaw f) value thetaA)
                              i -
                            KR21Monoculture.firstChoiceProb
                              (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
                                (KR21Monoculture.w11CandidateNoiseLaw f) value thetaH)
                              i ≤
                            KR21Monoculture.firstChoiceProb
                              (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
                                (KR21Monoculture.w11CandidateNoiseLaw f) value thetaA)
                              0 -
                            KR21Monoculture.firstChoiceProb
                              (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
```

(KR21Monoculture.w11CandidateNoiseLaw f) value thetaH)
0

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.StrictlyWellOrderedNoise
- KR21Monoculture.firstChoiceProb
- KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF
- KR21Monoculture.w11CandidateNoiseLaw

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: ℝ → ℝ
3. assumption: KR21Monoculture.StrictlyWellOrderedNoise f
4. assumption: Measurable f
5. assumption: ∀ (z : ℝ), 0 ≤ f z
6. assumption: ∫⁻ (z : ℝ), ENNReal.ofReal (f z) = 1
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
8. assumption: StrictAnti value
9. parameter: ℝ
10. parameter: ℝ
11. assumption: 0 < thetaH
12. assumption: thetaH ≤ thetaA
13. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
14. assumption: i ≠ 0
15. conclusion: (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value
    thetaA).toMeasure =
  MeasureTheory.Measure.map
    (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
      AppliedModelingLib.SocialChoice.Ranking.rankByScore
        fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => value c + epsilon c / thetaA)
    (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
      MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
  (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value
      thetaH).toMeasure =
    MeasureTheory.Measure.map
      (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore
          fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) => value c + epsilon c / thetaH)
      (MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
        MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (f z)) ∧
    KR21Monoculture.firstChoiceProb
        (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value thetaA)
        i -
      KR21Monoculture.firstChoiceProb
        (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value thetaH)
        i ≤
    KR21Monoculture.firstChoiceProb
        (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value thetaA)
        0 -
      KR21Monoculture.firstChoiceProb
        (KR21Monoculture.paper_appendixA_scaledNoiseRankingPMF (KR21Monoculture.w11CandidateNoiseLaw f) value thetaH)
        0

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma3_source_arbitraryFinite_iid_delta_le_top_delta_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: With strictly well-ordered iid noise and thetaA>=thetaH, the target is Lemma 3’s top gain at least each non-top first-choice gain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 26

Source locator: cited publication, lines 1467-1800

Verbatim source input

Theorem 7. For any a ∈ R and Xi = xi + σεi where εi is Laplacian with unit variance,
d
Pr[Xi > Xj | Xi < a, Xj < a] ≥ 0.
da
Moreover, it is strictly positive for some a.

[Next verbatim source excerpt]

Theorem 7. For any a ∈ R and Xi = xi + σεi where εi is Laplacian with unit variance,
d
Pr[Xi > Xj | Xi < a, Xj < a] ≥ 0.
da
Moreover, it is strictly positive for some a.
Proof. First, we must derive an expression for Pr[Xi > Xj | Xi < a, Xj < a]. Recall that the Laplace
distribution parameterized by μ and λ has pdf
f (x; μ, λ) =
and cdf

λ
exp(−λ|x − μ|)
2

(
F (x; μ, λ) =

1
2 exp (−λ(μ − x))
1 − 12 exp (−λ(x − μ))

21

x<μ
x≥μ

[form-feed in source extraction]
Note that xi and xj be the respective means of Xi and Xj , with xi > xj . Because the Laplace distribution
is piecewise defined, we must consider 3 cases and show that in all 3 cases, (C.2) holds. Note that

Expanded PaperInterface specification

```
∀ {sigma xi xj : ℝ},
  0 < sigma →
    xj < xi →
      have innovation : MeasureTheory.Measure ℝ :=
        MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|));
      have score : ℝ × ℝ → AppliedModelingLib.SocialChoice.Ranking.Candidate 0 → ℝ :=
        fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
        if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2;
      have rank : ℝ × ℝ → AppliedModelingLib.SocialChoice.Ranking.Ranking 0 := fun (epsilon : ℝ × ℝ) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore (score epsilon);
      have rankLaw : MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Ranking 0) :=
        MeasureTheory.Measure.map rank (innovation.prod innovation);
      Measurable rank ∧
        rankLaw Set.univ = 1 ∧
          (∀ (epsilon : ℝ × ℝ),
              xj + sigma * epsilon.2 < xi + sigma * epsilon.1 →
                AppliedModelingLib.SocialChoice.Ranking.firstChoice (rank epsilon) = 0) ∧
            (∀ (a : ℝ),
                0 <
                    ((innovation.prod innovation)
                        {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < a ∧ xj + sigma * epsilon.2 < a}).toReal ∧
                  ∃ (d : ℝ),
                    HasDerivAt
                        (fun (u : ℝ) =>
                          ((innovation.prod innovation)
                              {epsilon : ℝ × ℝ |
                                xi + sigma * epsilon.1 < u ∧
                                  xj + sigma * epsilon.2 < u ∧
                                    xj + sigma * epsilon.2 < xi + sigma * epsilon.1}).toReal /
                            ((innovation.prod innovation)
                                {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < u ∧ xj + sigma * epsilon.2 < u}).toReal)
                        d a ∧
                      0 ≤ d) ∧
              ∃ (a : ℝ) (d : ℝ),
                HasDerivAt
                    (fun (u : ℝ) =>
                      ((innovation.prod innovation)
                          {epsilon : ℝ × ℝ |
                            xi + sigma * epsilon.1 < u ∧
                              xj + sigma * epsilon.2 < u ∧ xj + sigma * epsilon.2 < xi + sigma * epsilon.1}).toReal /
                        ((innovation.prod innovation)
                            {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < u ∧ xj + sigma * epsilon.2 < u}).toReal)
                    d a ∧
                  0 < d
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. assumption: 0 < sigma
5. assumption: xj < xi
6. conclusion: (Measurable fun (epsilon : ℝ × ℝ) =>
    AppliedModelingLib.SocialChoice.Ranking.rankByScore
     ((fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
        if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2)
       epsilon)) ∧
  (MeasureTheory.Measure.map
       (fun (epsilon : ℝ × ℝ) =>
         AppliedModelingLib.SocialChoice.Ranking.rankByScore
          ((fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
             if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2)
           epsilon))
       ((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
         (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|)))))
      Set.univ =
    1 ∧
   (∀ (epsilon : ℝ × ℝ),
      xj + sigma * epsilon.2 < xi + sigma * epsilon.1 →
       AppliedModelingLib.SocialChoice.Ranking.firstChoice
          ((fun (epsilon : ℝ × ℝ) =>
             AppliedModelingLib.SocialChoice.Ranking.rankByScore
              ((fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
                 if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2)
               epsilon))
           epsilon) =
         0) ∧
     (∀ (a : ℝ),
        0 <
          (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
              (MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
             {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < a ∧ xj + sigma * epsilon.2 < a}).toReal ∧
         ∃ (d : ℝ),
           HasDerivAt
              (fun (u : ℝ) =>
                (((MeasureTheory.volume.withDensity fun (z : ℝ) =>
                          ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
                    (MeasureTheory.volume.withDensity fun (z : ℝ) =>
                      ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
                   {epsilon : ℝ × ℝ |
                     xi + sigma * epsilon.1 < u ∧
                       xj + sigma * epsilon.2 < u ∧ xj + sigma * epsilon.2 < xi + sigma * epsilon.1}).toReal /
                 (((MeasureTheory.volume.withDensity fun (z : ℝ) =>
                          ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
                    (MeasureTheory.volume.withDensity fun (z : ℝ) =>
                      ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
                   {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < u ∧ xj + sigma * epsilon.2 < u}).toReal)
              d a ∧
            0 ≤ d) ∧
       ∃ (a : ℝ) (d : ℝ),
         HasDerivAt
            (fun (u : ℝ) =>
              (((MeasureTheory.volume.withDensity fun (z : ℝ) => ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
                  (MeasureTheory.volume.withDensity fun (z : ℝ) =>
                    ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
                 {epsilon : ℝ × ℝ |
                   xi + sigma * epsilon.1 < u ∧
                     xj + sigma * epsilon.2 < u ∧ xj + sigma * epsilon.2 < xi + sigma * epsilon.1}).toReal /
               (((MeasureTheory.volume.withDensity fun (z : ℝ) =>
                        ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))).prod
                  (MeasureTheory.volume.withDensity fun (z : ℝ) =>
                    ENNReal.ofReal (√2 / 2 * Real.exp (-√2 * |z|))))
                 {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < u ∧ xj + sigma * epsilon.2 < u}).toReal)
            d a ∧
          0 < d
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem7_source_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For unit-variance Laplace innovations, the target gives source Theorem 7’s nonnegative derivative of the truncated pairwise-win probability for every a and strict positivity for some a.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 27

Source locator: cited publication, lines 1494-1764

Verbatim source input

is piecewise defined, we must consider 3 cases and show that in all 3 cases, (C.2) holds. Note that
Ra
f (x; xi , λ)F (x; xj , λ) dx
(C.3)
Pr[Xi > Xj | Xi < a, Xj < a] = −∞
F (a; xi , λ)F (a; xj , λ)

Expanded PaperInterface specification

```
∀ {lam xi xj a : ℝ},
  0 < lam →
    have density : ℝ → ℝ → ℝ := fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|);
    have cdf : ℝ → ℝ → ℝ := fun (location x : ℝ) =>
      if x < location then 1 / 2 * Real.exp (-lam * (location - x)) else 1 - 1 / 2 * Real.exp (-lam * (x - location));
    have scoreI : MeasureTheory.Measure ℝ :=
      MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (density xi x);
    have scoreJ : MeasureTheory.Measure ℝ :=
      MeasureTheory.volume.withDensity fun (x : ℝ) => ENNReal.ofReal (density xj x);
    have pairLaw : MeasureTheory.Measure (ℝ × ℝ) := scoreI.prod scoreJ;
    0 < (pairLaw {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal ∧
      (pairLaw {p : ℝ × ℝ | p.1 < a ∧ p.2 < p.1}).toReal / (pairLaw {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal =
        (∫ (x : ℝ) in Set.Iic a, density xi x * cdf xj x) / (cdf xi a * cdf xj a)
```

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 0 < lam
6. conclusion: 0 <
    (((MeasureTheory.volume.withDensity fun (x : ℝ) =>
            ENNReal.ofReal ((fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xi x)).prod
          (MeasureTheory.volume.withDensity fun (x : ℝ) =>
            ENNReal.ofReal ((fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xj x)))
        {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal ∧
  (((MeasureTheory.volume.withDensity fun (x : ℝ) =>
              ENNReal.ofReal ((fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xi x)).prod
            (MeasureTheory.volume.withDensity fun (x : ℝ) =>
              ENNReal.ofReal ((fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xj x)))
          {p : ℝ × ℝ | p.1 < a ∧ p.2 < p.1}).toReal /
      (((MeasureTheory.volume.withDensity fun (x : ℝ) =>
              ENNReal.ofReal ((fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xi x)).prod
            (MeasureTheory.volume.withDensity fun (x : ℝ) =>
              ENNReal.ofReal ((fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xj x)))
          {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal =
    (∫ (x : ℝ) in Set.Iic a,
       (fun (location x : ℝ) => lam / 2 * Real.exp (-lam * |x - location|)) xi x *
         (fun (location x : ℝ) =>
            if x < location then 1 / 2 * Real.exp (-lam * (location - x))
            else 1 - 1 / 2 * Real.exp (-lam * (x - location)))
           xj x) /
      ((fun (location x : ℝ) =>
          if x < location then 1 / 2 * Real.exp (-lam * (location - x))
          else 1 - 1 / 2 * Real.exp (-lam * (x - location)))
         xi a *
       (fun (location x : ℝ) =>
          if x < location then 1 / 2 * Real.exp (-lam * (location - x))
          else 1 - 1 / 2 * Real.exp (-lam * (x - location)))
         xj a)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC3_laplace_strict_conditional_ratio_eq_density_cdf_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target identifies the truncated pair-event probability with the source integral of the Laplace density times CDF divided by the two CDF factors.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 28

Source locator: cited publication, lines 1761-1765

Verbatim source input

⇐⇒ exp(−λ(2a − xi − xj)) − 8 + 4 exp(−λ(a − xi)) − exp(−2λ(a − xi))
+ (4 + 2λ(xi − xj))(1 + exp(−λ(xi − xj))) − (4 + 2λ(xi − xj)) exp(−λ(a − xj))
>0

(C.4)

Expanded PaperInterface specification

```
∀ {lam xi xj a : ℝ},
  0 < lam →
    xj < xi →
      xi < a →
        0 <
          Real.exp (-lam * (2 * a - xi - xj)) - 8 + 4 * Real.exp (-lam * (a - xi)) - Real.exp (-2 * lam * (a - xi)) +
              (4 + 2 * lam * (xi - xj)) * (1 + Real.exp (-lam * (xi - xj))) -
            (4 + 2 * lam * (xi - xj)) * Real.exp (-lam * (a - xj))
```

Lean-elaborated claim roles

1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 0 < lam
6. assumption: xj < xi
7. assumption: xi < a
8. conclusion: Real.lt† 0
  (Real.exp (-lam * (2 * a - xi - xj)) - 8 + 4 * Real.exp (-lam * (a - xi)) - Real.exp (-2 * lam * (a - xi)) +
      (4 + 2 * lam * (xi - xj)) * (1 + Real.exp (-lam * (xi - xj))) -
    (4 + 2 * lam * (xi - xj)) * Real.exp (-lam * (a - xj)))

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC4_laplace_case3_expression_pos

Recorded source-to-Spec screening **Verdict:** matches

Reason: Under the source case xj<xi<a and lambda>0, the target is the printed C.4 exponential expression with strict positivity.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 29

Source locator: cited publication, lines 1801-2160

Verbatim source input

Theorem 8. For any a ∈ R and Xi = xi + σεi where εi ∼ N (0, 1),
d
Pr[Xi > Xj | Xi < a, Xj < a] > 0.
da

[Next verbatim source excerpt]

Theorem 8. For any a ∈ R and Xi = xi + σεi where εi ∼ N (0, 1),
d
Pr[Xi > Xj | Xi < a, Xj < a] > 0.
da
√
0
Proof. Assume σ = 1/ 2. This
√ is without loss of generality because for any instance with arbitrary σ , there
is an instance with σ = 1/ 2 that yields the same distribution over rankings (simply by scaling all item
values by σ/σ 0). First, we have
Ra
Pr[Xi = x] Pr[Xj < x] dx
Pr[Xi > Xj | Xi < a, Xj < a] = −∞
P r[Xi < a] Pr[Xj < a]
Ra
√
exp(−(x − xi)2)/ π · (1 + erf(x − xj))/2 dx
−∞
=
(1 + erf(a − xi))/2 · (1 + erf(a − xj))/2
Ra
exp(−(x − xi)2)(1 + erf(x − xj)) dx
2
= √ −∞
(1 + erf(a − xi)) · (1 + erf(a − xj))
π
The derivative with respect to a is positive if and only if
(1 + erf(a − xi))(1 + erf(a − xj)) exp(−(a − xi)2)(1 + erf(a − xj))
Z a
>
exp(−(x − xi)2)(1 + erf(x − xj)) dx
−∞

[U+0001 control character]
2
· √ (1 + erf(a − xi)) exp(−(a − xj)2) + (1 + erf(a − xj)) exp(−(a − xi)2)
π

(C.5)

Let t = a − xi and δ = xi − xj . Then, using the fact that
Z a

Z a−xi

Expanded PaperInterface specification

```
∀ {sigma xi xj : ℝ},
  0 < sigma →
    xj < xi →
      have innovation : MeasureTheory.Measure ℝ := ProbabilityTheory.gaussianReal 0 1;
      have score : ℝ × ℝ → AppliedModelingLib.SocialChoice.Ranking.Candidate 0 → ℝ :=
        fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
        if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2;
      have rank : ℝ × ℝ → AppliedModelingLib.SocialChoice.Ranking.Ranking 0 := fun (epsilon : ℝ × ℝ) =>
        AppliedModelingLib.SocialChoice.Ranking.rankByScore (score epsilon);
      have rankLaw : MeasureTheory.Measure (AppliedModelingLib.SocialChoice.Ranking.Ranking 0) :=
        MeasureTheory.Measure.map rank (innovation.prod innovation);
      Measurable rank ∧
        rankLaw Set.univ = 1 ∧
          (∀ (epsilon : ℝ × ℝ),
              xj + sigma * epsilon.2 < xi + sigma * epsilon.1 →
                AppliedModelingLib.SocialChoice.Ranking.firstChoice (rank epsilon) = 0) ∧
            ∀ (a : ℝ),
              0 <
                  ((innovation.prod innovation)
                      {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < a ∧ xj + sigma * epsilon.2 < a}).toReal ∧
                ∃ (d : ℝ),
                  HasDerivAt
                      (fun (u : ℝ) =>
                        ((innovation.prod innovation)
                            {epsilon : ℝ × ℝ |
                              xi + sigma * epsilon.1 < u ∧
                                xj + sigma * epsilon.2 < u ∧
                                  xj + sigma * epsilon.2 < xi + sigma * epsilon.1}).toReal /
                          ((innovation.prod innovation)
                              {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < u ∧ xj + sigma * epsilon.2 < u}).toReal)
                      d a ∧
                    0 < d
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. assumption: 0 < sigma
5. assumption: xj < xi
6. conclusion: (Measurable fun (epsilon : ℝ × ℝ) =>
   AppliedModelingLib.SocialChoice.Ranking.rankByScore
    ((fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
       if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2)
      epsilon)) ∧
  (MeasureTheory.Measure.map
       (fun (epsilon : ℝ × ℝ) =>
         AppliedModelingLib.SocialChoice.Ranking.rankByScore
           ((fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
              if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2)
             epsilon))
       ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1)))
      Set.univ =
    1 ∧
   (∀ (epsilon : ℝ × ℝ),
      xj + sigma * epsilon.2 < xi + sigma * epsilon.1 →
       AppliedModelingLib.SocialChoice.Ranking.firstChoice
          ((fun (epsilon : ℝ × ℝ) =>
             AppliedModelingLib.SocialChoice.Ranking.rankByScore
               ((fun (epsilon : ℝ × ℝ) (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 0) =>
                  if c = 0 then xi + sigma * epsilon.1 else xj + sigma * epsilon.2)
                 epsilon))
            epsilon) =
         0) ∧
     ∀ (a : ℝ),
       0 <
         (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
             {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < a ∧ xj + sigma * epsilon.2 < a}).toReal ∧
        ∃ (d : ℝ),
          HasDerivAt
             (fun (u : ℝ) =>
               (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
                   {epsilon : ℝ × ℝ |
                     xi + sigma * epsilon.1 < u ∧
                       xj + sigma * epsilon.2 < u ∧ xj + sigma * epsilon.2 < xi + sigma * epsilon.1}).toReal /
                 (((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))
                   {epsilon : ℝ × ℝ | xi + sigma * epsilon.1 < u ∧ xj + sigma * epsilon.2 < u}).toReal)
             d a ∧
           0 < d
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem8_source_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For standard Gaussian innovations, the target gives source Theorem 8's strictly positive derivative of the truncated pairwise-win probability for every a.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 30

Source locator: cited publication, lines 1827-1839

Verbatim source input

The derivative with respect to a is positive if and only if
(1 + erf(a − xi))(1 + erf(a − xj)) exp(−(a − xi)2)(1 + erf(a − xj))
Z a
>
exp(−(x − xi)2)(1 + erf(x − xj)) dx
−∞

[U+0001 control character]
2
· √ (1 + erf(a − xi)) exp(−(a − xj)2) + (1 + erf(a − xj)) exp(−(a − xi)2)
π

(C.5)

Expanded PaperInterface specification

```
∀ {xi xj a : ℝ},
  xj < xi →
    have scoreI : MeasureTheory.Measure ℝ := ProbabilityTheory.gaussianReal xi (1 / 2);
    have scoreJ : MeasureTheory.Measure ℝ := ProbabilityTheory.gaussianReal xj (1 / 2);
    have denominator : ℝ := ((scoreI.prod scoreJ) {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal;
    0 < denominator ∧
      ((scoreI.prod scoreJ) {p : ℝ × ℝ | p.1 < a ∧ p.2 < a ∧ p.2 < p.1}).toReal / denominator =
        (2 / √Real.pi *
           ∫ (x : ℝ) in Set.Iic a, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) /
          ((1 + KR21Monoculture.theorem8Erf (a - xi)) * (1 + KR21Monoculture.theorem8Erf (a - xj))) ∧
       0 <
          (1 + KR21Monoculture.theorem8Erf (a - xi)) * (1 + KR21Monoculture.theorem8Erf (a - xj)) *
               Real.exp (-(a - xi) ^ 2) *
             (1 + KR21Monoculture.theorem8Erf (a - xj)) -
           (∫ (x : ℝ) in Set.Iic a, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) *
              (2 / √Real.pi) *
             ((1 + KR21Monoculture.theorem8Erf (a - xi)) * Real.exp (-(a - xj) ^ 2) +
              (1 + KR21Monoculture.theorem8Erf (a - xj)) * Real.exp (-(a - xi) ^ 2)) ∧
        ∃ (d : ℝ),
          HasDerivAt
            (fun (u : ℝ) =>
              (2 / √Real.pi *
                 ∫ (x : ℝ) in Set.Iic u, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) /
                ((1 + KR21Monoculture.theorem8Erf (u - xi)) * (1 + KR21Monoculture.theorem8Erf (u - xj))))
            d a ∧
           (0 < d ↔
             0 <
              (1 + KR21Monoculture.theorem8Erf (a - xi)) * (1 + KR21Monoculture.theorem8Erf (a - xj)) *
                   Real.exp (-(a - xi) ^ 2) *
                 (1 + KR21Monoculture.theorem8Erf (a - xj)) -
               (∫ (x : ℝ) in Set.Iic a, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) *
                  (2 / √Real.pi) *
                 ((1 + KR21Monoculture.theorem8Erf (a - xi)) * Real.exp (-(a - xj) ^ 2) +
                  (1 + KR21Monoculture.theorem8Erf (a - xj)) * Real.exp (-(a - xi) ^ 2)))
```

Retained governing declarations

- KR21Monoculture.theorem8Erf

Lean-elaborated claim roles

1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. assumption: xj < xi
5. conclusion: 0 <
   (((ProbabilityTheory.gaussianReal xi (1 / 2)).prod (ProbabilityTheory.gaussianReal xj (1 / 2)))
      {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal ∧
  (((ProbabilityTheory.gaussianReal xi (1 / 2)).prod (ProbabilityTheory.gaussianReal xj (1 / 2)))
         {p : ℝ × ℝ | p.1 < a ∧ p.2 < a ∧ p.2 < p.1}).toReal /
       (((ProbabilityTheory.gaussianReal xi (1 / 2)).prod (ProbabilityTheory.gaussianReal xj (1 / 2)))
          {p : ℝ × ℝ | p.1 < a ∧ p.2 < a}).toReal =
     (2 / √Real.pi * ∫ (x : ℝ) in Set.Iic a, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) /
       ((1 + KR21Monoculture.theorem8Erf (a - xi)) * (1 + KR21Monoculture.theorem8Erf (a - xj))) ∧
   0 <
      (1 + KR21Monoculture.theorem8Erf (a - xi)) * (1 + KR21Monoculture.theorem8Erf (a - xj)) *
           Real.exp (-(a - xi) ^ 2) *
         (1 + KR21Monoculture.theorem8Erf (a - xj)) -
       (∫ (x : ℝ) in Set.Iic a, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) *
          (2 / √Real.pi) *
         ((1 + KR21Monoculture.theorem8Erf (a - xi)) * Real.exp (-(a - xj) ^ 2) +
          (1 + KR21Monoculture.theorem8Erf (a - xj)) * Real.exp (-(a - xi) ^ 2)) ∧
     ∃ (d : ℝ),
       HasDerivAt
         (fun (u : ℝ) =>
           (2 / √Real.pi *
              ∫ (x : ℝ) in Set.Iic u, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) /
             ((1 + KR21Monoculture.theorem8Erf (u - xi)) * (1 + KR21Monoculture.theorem8Erf (u - xj))))
         d a ∧

```
(0 < d ↔
  0 <
    (1 + KR21Monoculture.theorem8Erf (a - xi)) * (1 + KR21Monoculture.theorem8Erf (a - xj)) *
        Real.exp (-(a - xi) ^ 2) *
      (1 + KR21Monoculture.theorem8Erf (a - xj)) -
    (∫ (x : ℝ) in Set.Iic a, Real.exp (-(x - xi) ^ 2) * (1 + KR21Monoculture.theorem8Erf (x - xj))) *
      (2 / √Real.pi) *
     ((1 + KR21Monoculture.theorem8Erf (a - xi)) * Real.exp (-(a - xj) ^ 2) +
      (1 + KR21Monoculture.theorem8Erf (a - xj)) * Real.exp (-(a - xi) ^ 2)))
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC5_gaussian_source_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target gives the source Gaussian erf conditional ratio and its C.5 derivative-positivity numerator criterion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 31

Source locator: cited publication, lines 1874-1914

Verbatim source input

(C.5) becomes
√
√
Z t
π
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
π
·
>
(1
+
erf(t))
+
exp(−x2) erf(x + δ) dx
2 (1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)
2
−∞
Z t
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
2
√
⇐⇒
−
(1
+
erf(t))
−
exp(−x2) erf(x + δ) dx > 0
(1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)
π −∞
(C.6)

Expanded PaperInterface specification

```
∀ {delta t : ℝ},
  0 < delta →
    0 <
      (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 * Real.exp (-t ^ 2) /
           ((1 + KR21Monoculture.theorem8Erf t) * Real.exp (-(t + delta) ^ 2) +
             (1 + KR21Monoculture.theorem8Erf (t + delta)) * Real.exp (-t ^ 2)) -
         (1 + KR21Monoculture.theorem8Erf t) -
        2 / √Real.pi * ∫ (x : ℝ) in Set.Iic t, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta)
```

Retained governing declarations

- KR21Monoculture.theorem8Erf

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. assumption: 0 < delta
4. conclusion: Real.lt✝ 0
  ((1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 * Real.exp (-t ^ 2) /
       ((1 + KR21Monoculture.theorem8Erf t) * Real.exp (-(t + delta) ^ 2) +
         (1 + KR21Monoculture.theorem8Erf (t + delta)) * Real.exp (-t ^ 2)) -
     (1 + KR21Monoculture.theorem8Erf t) -
    2 / √Real.pi * ∫ (x : ℝ) in Set.Iic t, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC6_gaussian_reduced_expression_pos

Recorded source-to-Spec screening **Verdict:** matches

Reason: After t=a-xi and delta=xi-xj>0, the target is the source C.6 reduced expression asserted positive.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 32

Source locator: cited publication, lines 1914-1951

Verbatim source input

We’ll show that these conditions hold for the LHS of (C.6).
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
2
− (1 + erf(t)) − √
t→−∞ (1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)
π

Z t

lim

(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
t→−∞ (1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)

= lim

exp(−x2) erf(x + δ) dx

−∞

(C.7)

Observe that both the numerator and denominator of (C.7) are positive, so this limit must be at least 0.
We can upper bound it by
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
≤
lim
t→−∞
t→−∞ (1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)
(1 + erf(t + δ)) exp(−t2)
= lim (1 + erf(t))(1 + erf(t + δ))
lim

t→−∞

=0
Thus, the limit is 0. Now, we must show that the derivative is positive. The derivative is

Expanded PaperInterface specification

```
∀ (delta : ℝ),
  Filter.Tendsto
      (fun (t : ℝ) =>
        (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 * Real.exp (-t ^ 2) /
          ((1 + KR21Monoculture.theorem8Erf t) * Real.exp (-(t + delta) ^ 2) +
            (1 + KR21Monoculture.theorem8Erf (t + delta)) * Real.exp (-t ^ 2)))
      Filter.atBot (nhds 0) ∧
    Filter.Tendsto (fun (t : ℝ) => ∫ (x : ℝ) in Set.Iic t, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
      Filter.atBot (nhds 0)
```

Retained governing declarations

- KR21Monoculture.theorem8Erf

Lean-elaborated claim roles

```
1. parameter: ℝ
2. conclusion: Filter.Tendsto
    (fun (t : ℝ) =>
      (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 * Real.exp (-t ^ 2) /
        ((1 + KR21Monoculture.theorem8Erf t) * Real.exp (-(t + delta) ^ 2) +
          (1 + KR21Monoculture.theorem8Erf (t + delta)) * Real.exp (-t ^ 2)))
    Filter.atBot (nhds 0) ∧
  Filter.Tendsto (fun (t : ℝ) => ∫ (x : ℝ) in Set.Iic t, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
    Filter.atBot (nhds 0)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC7_gaussian_reduced_expression_tendsto_atBot_zero

Recorded source-to-Spec screening **Verdict:** matches

Reason: The source’s C.7 equality identifies the rational expression and Gaussian integral as the same left-tail limit, and its immediately following bound concludes that limit is zero; the Lean target states both resulting zero limits.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 33

Source locator: cited publication, lines 1951-1985

Verbatim source input

Thus, the limit is 0. Now, we must show that the derivative is positive. The derivative is
[U+0015 control character]
[U+0014 control character]
Z t
2
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
d
2
− (1 + erf(t)) − √
exp(−x) erf(x + δ) dx
dt (1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)
π −∞
[U+0014 control character]
[U+0015 control character]
d
(1 + erf(t))(1 + erf(t + δ))2 exp(−t2)
2
2
=
− √ exp(−t2) − √ exp(−t2) erf(t + δ)
dt (1 + erf(t)) exp(−(t + δ)2) + (1 + erf(t + δ)) exp(−t2)
π
π
(C.8)

Expanded PaperInterface specification

```
∀ (delta t : ℝ),
  HasDerivAt
    (fun (u : ℝ) =>
      (1 + KR21Monoculture.theorem8Erf u) * (1 + KR21Monoculture.theorem8Erf (u + delta)) ^ 2 * Real.exp (-u ^ 2) /
          ((1 + KR21Monoculture.theorem8Erf u) * Real.exp (-(u + delta) ^ 2) +
            (1 + KR21Monoculture.theorem8Erf (u + delta)) * Real.exp (-u ^ 2)) -
        (1 + KR21Monoculture.theorem8Erf u) -
       2 / √Real.pi * ∫ (x : ℝ) in Set.Iic u, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
    (deriv
        (fun (u : ℝ) =>
          (1 + KR21Monoculture.theorem8Erf u) * (1 + KR21Monoculture.theorem8Erf (u + delta)) ^ 2 *
              Real.exp (-u ^ 2) /
            ((1 + KR21Monoculture.theorem8Erf u) * Real.exp (-(u + delta) ^ 2) +
              (1 + KR21Monoculture.theorem8Erf (u + delta)) * Real.exp (-u ^ 2)))
        t -
      2 / √Real.pi * Real.exp (-t ^ 2) -
     2 / √Real.pi * Real.exp (-t ^ 2) * KR21Monoculture.theorem8Erf (t + delta))
    t
```

Retained governing declarations

- KR21Monoculture.theorem8Erf

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. conclusion: HasFDerivAtFilter
  (fun (u : ℝ) =>
    (1 + KR21Monoculture.theorem8Erf u) * (1 + KR21Monoculture.theorem8Erf (u + delta)) ^ 2 * Real.exp (-u ^ 2) /
          ((1 + KR21Monoculture.theorem8Erf u) * Real.exp (-(u + delta) ^ 2) +
            (1 + KR21Monoculture.theorem8Erf (u + delta)) * Real.exp (-u ^ 2)) -
        (1 + KR21Monoculture.theorem8Erf u) -
       2 / √Real.pi * ∫ (x : ℝ) in Set.Iic u, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
  (ContinuousLinearMap.toSpanSingleton ℝ
    (deriv
        (fun (u : ℝ) =>
          (1 + KR21Monoculture.theorem8Erf u) * (1 + KR21Monoculture.theorem8Erf (u + delta)) ^ 2 *
              Real.exp (-u ^ 2) /
            ((1 + KR21Monoculture.theorem8Erf u) * Real.exp (-(u + delta) ^ 2) +
              (1 + KR21Monoculture.theorem8Erf (u + delta)) * Real.exp (-u ^ 2)))
        t -
      2 / √Real.pi * Real.exp (-t ^ 2) -
     2 / √Real.pi * Real.exp (-t ^ 2) * KR21Monoculture.theorem8Erf (t + delta)))
  (nhds t ×ˢ pure t)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC8_gaussian_reduced_expression_hasDerivAt

Recorded source-to-Spec screening **Verdict:** matches
Reason: The target is the source C.8 derivative of the C.6 expression: derivative of the rational term minus the two displayed erf-integral derivatives.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 34

Source locator: cited publication, lines 1801-1914

Verbatim source input

we get that (C.8) is positive if and only if
√
δ π exp((t + δ)2)(1 + erf(t))(1 + erf(t + δ)) − exp(2δt + t2)(1 + erf(t + δ)) + (1 + erf(t)) > 0
√
1 + erf(t)
⇐⇒δ π exp((t + δ)2)(1 + erf(t)) +
− exp(2δt + t2) > 0
1 + erf(t + δ)
√
1 + erf(t)
⇐⇒δ π exp(t2)(1 + erf(t)) + exp(−2δt − t2)
−1>0
1 + erf(t + δ)
[U+0014 control character]
[U+0015 control character]
√
exp(−2δt − δ 2)
⇐⇒(1 + erf(t)) δ π exp(t2) +
−1>0
1 + erf(t + δ)
[U+0015 control character]
[U+0014 control character]
1 + erf(t) √
exp(−(t + δ)2)
⇐⇒
δ
π
+
−1>0
(C.9)
exp(−t2)
1 + erf(t + δ)

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

In the source normalization sigma = 1/sqrt(2), with t = a-x_i and delta = x_j-x_i > 0, let e(u)=erf(u) and H_delta(u) = ((1+e(u))(1+e(u+delta))^2
↪ exp(-u^2))/((1+e(u))exp(-(u+delta)^2)+(1+e(u+delta))exp(-u^2)) - (1+e(u)) - (2/sqrt(pi))*integral_{x<=u} exp(-x^2)e(x+delta) dx. Then H_delta has
↪ derivative P_delta(t)*B_delta(t), where P_delta(t) = 2(1+e(t))(1+e(t+delta))^2
↪ exp((t+delta)^2)/(sqrt(pi)*((e(t)+1)exp(t^2)+(e(t+delta)+1)exp((t+delta)^2))^2) and B_delta(t) = ((1+e(t))/exp(-t^2))*(delta*sqrt(pi) +
↪ ((1+e(t+delta))/exp(-(t+delta)^2))^(-1)) - 1. Both P_delta(t) and B_delta(t) are strictly positive. This is the corrected factorization and parenthesized
↪ C.9 bracket, not the printed prefactor or unparenthesized display.

Archival source anchor: cited publication, lines 1951-2039

Expanded PaperInterface specification

```
∀ {delta t : ℝ},
  0 < delta →
    HasDerivAt
      (fun (u : ℝ) =>
        (1 + KR21Monoculture.theorem8Erf u) * (1 + KR21Monoculture.theorem8Erf (u + delta)) ^ 2 * Real.exp (-u ^ 2) /
            ((1 + KR21Monoculture.theorem8Erf u) * Real.exp (-(u + delta) ^ 2) +
              (1 + KR21Monoculture.theorem8Erf (u + delta)) * Real.exp (-u ^ 2)) -
          (1 + KR21Monoculture.theorem8Erf u) -
        2 / √Real.pi * ∫ (x : ℝ) in Set.Iic u, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
      (2 * (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 *
          Real.exp ((t + delta) ^ 2) /
        (√Real.pi *
          ((KR21Monoculture.theorem8Erf t + 1) * Real.exp (t ^ 2) +
            (KR21Monoculture.theorem8Erf (t + delta) + 1) * Real.exp ((t + delta) ^ 2)) ^
          2) *
       ((1 + KR21Monoculture.theorem8Erf t) / Real.exp (-t ^ 2) *
          (delta * √Real.pi + ((1 + KR21Monoculture.theorem8Erf (t + delta)) / Real.exp (-(t + delta) ^ 2))⁻¹) -
        1))
      t ∧
    0 <
      2 * (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 *
          Real.exp ((t + delta) ^ 2) /
        (√Real.pi *
          ((KR21Monoculture.theorem8Erf t + 1) * Real.exp (t ^ 2) +
            (KR21Monoculture.theorem8Erf (t + delta) + 1) * Real.exp ((t + delta) ^ 2)) ^
          2) ∧
     0 <
      (1 + KR21Monoculture.theorem8Erf t) / Real.exp (-t ^ 2) *
        (delta * √Real.pi + ((1 + KR21Monoculture.theorem8Erf (t + delta)) / Real.exp (-(t + delta) ^ 2))⁻¹) -
      1
```

Retained governing declarations

- KR21Monoculture.theorem8Erf

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. assumption: 0 < delta
4. conclusion: HasDerivAt
    (fun (u : ℝ) =>
      (1 + KR21Monoculture.theorem8Erf u) * (1 + KR21Monoculture.theorem8Erf (u + delta)) ^ 2 * Real.exp (-u ^ 2) /
           ((1 + KR21Monoculture.theorem8Erf u) * Real.exp (-(u + delta) ^ 2) +
             (1 + KR21Monoculture.theorem8Erf (u + delta)) * Real.exp (-u ^ 2)) -
         (1 + KR21Monoculture.theorem8Erf u) -
       2 / √Real.pi * ∫ (x : ℝ) in Set.Iic u, Real.exp (-x ^ 2) * KR21Monoculture.theorem8Erf (x + delta))
    (2 * (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 *
         Real.exp ((t + delta) ^ 2) /
       (√Real.pi *
         ((KR21Monoculture.theorem8Erf t + 1) * Real.exp (t ^ 2) +
           (KR21Monoculture.theorem8Erf (t + delta) + 1) * Real.exp ((t + delta) ^ 2)) ^
           2) *
      ((1 + KR21Monoculture.theorem8Erf t) / Real.exp (-t ^ 2) *
         (delta * √Real.pi + ((1 + KR21Monoculture.theorem8Erf (t + delta)) / Real.exp (-(t + delta) ^ 2))⁻¹) -
       1))
    t ∧
  0 <
     2 * (1 + KR21Monoculture.theorem8Erf t) * (1 + KR21Monoculture.theorem8Erf (t + delta)) ^ 2 *
         Real.exp ((t + delta) ^ 2) /
       (√Real.pi *
         ((KR21Monoculture.theorem8Erf t + 1) * Real.exp (t ^ 2) +
           (KR21Monoculture.theorem8Erf (t + delta) + 1) * Real.exp ((t + delta) ^ 2)) ^
           2) ∧
    0 <
      (1 + KR21Monoculture.theorem8Erf t) / Real.exp (-t ^ 2) *
         (delta * √Real.pi + ((1 + KR21Monoculture.theorem8Erf (t + delta)) / Real.exp (-(t + delta) ^ 2))⁻¹) -
       1
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC8_C9_corrected_gaussian_target

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The full H_delta derivative, corrected P_delta*B_delta factorization, and both positivity facts match the bound approval for the non-archival C.8/C.9 repair.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 35

Source locator: cited publication, lines 2017-2056

Verbatim source input

Define
g(t) ,

1 + erf(t)
.
exp(−t2)

Then, (C.9) is
[U+0014 control character]
√
g(t) δ π +

[U+0015 control character]
1
−1>0
g(t + δ)
√
1
1
⇐⇒
−
<δ π
g(t) g(t + δ)
26

[form-feed in source extraction]
By the Mean Value Theorem,
1
1
d 1
−
= −δ
g(t) g(t + δ)
dt g(t) t=t∗
for some t ≤ t∗ ≤ t + δ. Thus, it suffices to show that
√
d 1
>− π
dt g(t)

(C.10)

Expanded PaperInterface specification

```
∀ (t : ℝ),
  ∃ (d : ℝ), HasDerivAt (fun (u : ℝ) => ((1 + KR21Monoculture.theorem8Erf u) / Real.exp (-u ^ 2))⁻¹) d t ∧ -√Real.pi < d
```

Retained governing declarations

- KR21Monoculture.theorem8Erf

Lean-elaborated claim roles

1. parameter: ℝ
2. conclusion: ∃ (d : ℝ), HasDerivAt (fun (u : ℝ) => ((1 + KR21Monoculture.theorem8Erf u) / Real.exp (-u ^ 2))⁻¹) d t ∧ -√Real.pi < d

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationC10_gaussian_g_inv_derivative_lower_bound

Recorded source-to-Spec screening **Verdict:** matches

Reason: With g(t) = (1 + erf(t)) / exp(-t^2), the source's condition (C.10) is exactly that the derivative of 1/g is strictly greater than -sqrt(pi), as stated by the Lean target for every t.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 36

Source locator: cited publication, lines 133-138

Verbatim source input

Theorem 9. The family of distributions Fθ produced by the Mallows Model with Kendall tau distance with
θ = φ − 1 satisfies the conditions of Definition 1.

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n)
  (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
  AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value →
    (∀ (phi theta : ℝ),
       1 < phi →
         theta = phi - 1 →
           have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
           M.q = phi⁻¹ ∧
             ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
               (M.law pi).toReal =
                 phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                   ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                     phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
      (∀ (theta : ℝ),
         0 < theta →
           ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
             ContinuousAt (fun (theta' : ℝ) => ((KR21Monoculture.concreteMallowsSpec center theta').law pi).toReal)
               theta ∧
             DifferentiableAt ℝ
               (fun (theta' : ℝ) => ((KR21Monoculture.concreteMallowsSpec center theta').law pi).toReal) theta) ∧
        Filter.Tendsto (fun (theta : ℝ) => ((KR21Monoculture.concreteMallowsSpec center theta).law center).toReal)
          Filter.atTop (nhds 1) ∧
          ∀ (thetaA thetaH : ℝ),
            0 < thetaH →
              thetaH < thetaA →
                (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
                   remaining.Nonempty →
                     KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaH).law value
                       remaining ≤
                       KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaA).law value
                         remaining) ∧
                  KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaH).law value
                    Finset.univ <
                    KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaA).law value
                      Finset.univ
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.expectedBestInSet

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
4. assumption: AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
5. conclusion: (∀ (phi theta : ℝ),
     1 < phi →
       theta = phi - 1 →
         have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
         M.q = phi⁻¹ ∧
           ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
             (M.law pi).toReal =
               phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                 ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                   phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
    (∀ (theta : ℝ),
       0 < theta →
         ∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
           ContinuousAt (fun (theta' : ℝ) => ((KR21Monoculture.concreteMallowsSpec center theta').law pi).toReal) theta ∧
             DifferentiableAt ℝ (fun (theta' : ℝ) => ((KR21Monoculture.concreteMallowsSpec center theta').law pi).toReal)
               theta) ∧
      Filter.Tendsto (fun (theta : ℝ) => ((KR21Monoculture.concreteMallowsSpec center theta).law center).toReal)
        Filter.atTop (nhds 1) ∧
        ∀ (thetaA thetaH : ℝ),
          0 < thetaH →
```

```
thetaH < thetaA →
  (∀ (remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate n)),
     remaining.Nonempty →
       KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaH).law value
           remaining ≤
         KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaA).law value
           remaining) ∧
    KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaH).law value
       Finset.univ <
     KR21Monoculture.expectedBestInSet (KR21Monoculture.concreteMallowsSpec center thetaA).law value
       Finset.univ
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.theorem9_source_mallows_definition1_literal_semantic_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: With theta=phi-1, the target gives the normalized Kendall-tau Mallows law and every Definition 1 condition claimed by source Theorem 9.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 37

Source locator: cited publication, lines 133-138

Verbatim source input

Rearranging,
(−S)

Prφ [π1

= xi]

(−S)
Prφ0 [π1
= xi]

(−S)

Prφ [π1

= xj]
(−S)
Prφ0 [π1
= xj]

<

(D.1)

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For a common center on exactly three candidates, phi_accurate > phi_noisy > 1, theta_r = phi_r-1, and any nonempty remaining set containing center-ordered candidates better before worse, the two source Mallows laws have q_r = phi_r^(-1) and normalized mass phi_r^(-KendallTau(center,pi)) / sum_tau phi_r^(-KendallTau(center,tau)). Writing P_r(c) for the probability that c is best in that remaining set under law r, P_accurate(better)P_noisy(worse) > P_accurate(worse)P_noisy(better). No arbitrary-universe claim or printed reverse D.1 inequality is asserted.

Archival source anchor: cited publication, lines 2317-2376

Expanded PaperInterface specification

```
∀ (center : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) (phiNoisy thetaNoisy phiAccurate thetaAccurate : ℝ),
  1 < phiNoisy →
    phiNoisy < phiAccurate →
      thetaNoisy = phiNoisy - 1 →
        thetaAccurate = phiAccurate - 1 →
          ∀ {remaining : Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)},
            remaining.Nonempty →
              ∀ {better worse : AppliedModelingLib.SocialChoice.Ranking.Candidate 1},
                better ∈ remaining →
                  worse ∈ remaining →
                    AppliedModelingLib.SocialChoice.Ranking.rankOf center better <
                      AppliedModelingLib.SocialChoice.Ranking.rankOf center worse →
                    have MAccurate : KR21Monoculture.MallowsSpec 1 :=
                      KR21Monoculture.concreteMallowsSpec center thetaAccurate;
                    have MNoisy : KR21Monoculture.MallowsSpec 1 :=
                      KR21Monoculture.concreteMallowsSpec center thetaNoisy;
                    MAccurate.q = phiAccurate⁻¹ ∧
                      MNoisy.q = phiNoisy⁻¹ ∧
                        (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
                          (MAccurate.law pi).toReal =
                            phiAccurate⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                              ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking 1,
                                phiAccurate⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                          (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
                            (MNoisy.law pi).toReal =
                              phiNoisy⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                                ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking 1,
                                  phiNoisy⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                            0 <
                              ((AppliedModelingLib.pmfProb MAccurate.law
                                  fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
                                  better = KR21Monoculture.bestInSet pi remaining) *
                                AppliedModelingLib.pmfProb MNoisy.law
                                  fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
                                  worse = KR21Monoculture.bestInSet pi remaining) -
                               (AppliedModelingLib.pmfProb MAccurate.law
                                  fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
                                  worse = KR21Monoculture.bestInSet pi remaining) *
                                AppliedModelingLib.pmfProb MNoisy.law
                                  fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
                                  better = KR21Monoculture.bestInSet pi remaining
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau

- AppliedModelingLib.SocialChoice.Ranking.rankOf
- AppliedModelingLib.pmfProb
- KR21Monoculture.MallowsSpec
- KR21Monoculture.bestInSet
- KR21Monoculture.concreteMallowsSpec

Lean-elaborated claim roles

1. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking 1
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. parameter: ℝ
6. assumption: 1 < phiNoisy
7. assumption: phiNoisy < phiAccurate
8. assumption: thetaNoisy = phiNoisy - 1
9. assumption: thetaAccurate = phiAccurate - 1
10. parameter: Finset (AppliedModelingLib.SocialChoice.Ranking.Candidate 1)
11. assumption: remaining.Nonempty
12. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate 1
13. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate 1
14. assumption: better ∈ remaining
15. assumption: worse ∈ remaining
16. assumption: AppliedModelingLib.SocialChoice.Ranking.rankOf center better <
  AppliedModelingLib.SocialChoice.Ranking.rankOf center worse
17. conclusion: (KR21Monoculture.concreteMallowsSpec center thetaAccurate).q = phiAccurate⁻¹ ∧
  (KR21Monoculture.concreteMallowsSpec center thetaNoisy).q = phiNoisy⁻¹ ∧
    (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
       ((KR21Monoculture.concreteMallowsSpec center thetaAccurate).law pi).toReal =
         phiAccurate⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
           ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking 1,
             phiAccurate⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
      (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1),
         ((KR21Monoculture.concreteMallowsSpec center thetaNoisy).law pi).toReal =
           phiNoisy⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
             ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking 1,
               phiNoisy⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
        0 <
         ((AppliedModelingLib.pmfProb (KR21Monoculture.concreteMallowsSpec center thetaAccurate).law
              fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
              better = KR21Monoculture.bestInSet pi remaining) *
            AppliedModelingLib.pmfProb (KR21Monoculture.concreteMallowsSpec center thetaNoisy).law
              fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
              worse = KR21Monoculture.bestInSet pi remaining) -
           (AppliedModelingLib.pmfProb (KR21Monoculture.concreteMallowsSpec center thetaAccurate).law
              fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
              worse = KR21Monoculture.bestInSet pi remaining) *
            AppliedModelingLib.pmfProb (KR21Monoculture.concreteMallowsSpec center thetaNoisy).law
              fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking 1) =>
              better = KR21Monoculture.bestInSet pi remaining

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationD1_source_phi_likelihood_ratio_complete

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: For exactly three candidates, the visible phi^-1 laws and center order yield the approved corrected strict better/worse product inequality, not archival D.1's reverse direction.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 38

Source locator: cited publication, lines 133-138

Verbatim source input

We must show that when π, τ ∼ Fθ ,
E [π1 − π2 | π1 6= τ1] > 0.

(E.1)

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For Candidate n = Fin(n+2) with n > 0, a center ranking, phi > 1, theta = phi-1, and a value profile strictly ordered by that center, the source Mallows law has q = phi^(-1) and normalized mass phi^(-KendallTau(center,pi)) / sum_tau phi^(-KendallTau(center,tau)). For two iid draws pi,tau, the top-disagreement event has positive probability and E[(value(first(pi))-value(second(pi)))*1_{first(pi) != first(tau)}] divided by that probability is strictly positive.

Archival source anchor: cited publication, lines 2444-2447

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      0 < n →
        ∀ {value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ},
          AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value →
            have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
            M.q = phi⁻¹ ∧
              (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                  (M.law pi).toReal =
                    phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                      ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                        phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                0 < AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent ∧
                  0 <
                    (AppliedModelingLib.pmfPairIndicatorExp M.law M.law KR21Monoculture.disagreementEvent
                        fun
                          (pair :
                            AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                              AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                        value (AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1) -
                          value (AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1)) /
                      AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.pmfPairIndicatorExp
- AppliedModelingLib.pmfPairProb
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.decidableDisagreementEvent
- KR21Monoculture.disagreementEvent

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. assumption: 0 < n
8. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
9. assumption: AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
10. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧

```
(∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
   ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
    phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
      ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
        phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
 0 <
    AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
      (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent ∧
   0 <
    (AppliedModelingLib.pmfPairIndicatorExp (KR21Monoculture.concreteMallowsSpec center theta).law
       (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
       fun
         (pair :
           AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
             AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
       value (AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1) -
         value (AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1)) /
      AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
        (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationE1_source_phi_literal_conditional_semantic_complete

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The target is the approved N>=3 Mallows conditional-gain endpoint with positive top-disagreement denominator and strictly positive quotient.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 39

Source locator: cited publication, lines 133-138

Verbatim source input

Since xi > xj for i < j, it suffices to show that for all i < j,
Pr[π1 = xi ∩ π2 = xj | π1 6= τ1] ≥ Pr[π1 = xj ∩ π2 = xi | π1 6= τ1],

(E.2)

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For Candidate n = Fin(n+2) with n > 0, a center ranking, phi > 1, and theta = phi-1, the source Mallows law has q = phi^(-1), normalized mass phi^(-KendallTau(center,pi)) / sum_tau phi^(-KendallTau(center,tau)), and positive iid top-disagreement probability. Conditional on top disagreement, for every center-ordered pair c before d, the probability of first-two order d,c is at most that of c,d; for at least one center-ordered pair the inequality is strict.

Archival source anchor: cited publication, lines 2469-2474

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      0 < n →
        have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
        M.q = phi⁻¹ ∧
          (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
              (M.law pi).toReal =
                phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                  ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                    phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
            0 < AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent ∧
              (∀ (c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                  AppliedModelingLib.SocialChoice.Ranking.rankOf M.center c <
                      AppliedModelingLib.SocialChoice.Ranking.rankOf M.center d →
                    (AppliedModelingLib.pmfPairIndicatorExp M.law M.law KR21Monoculture.disagreementEvent
                          fun
                            (pair :
                              AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                                AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                          if
                              d = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                                c = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                            1
                          else 0) /
                        AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent ≤
                      (AppliedModelingLib.pmfPairIndicatorExp M.law M.law KR21Monoculture.disagreementEvent
                          fun
                            (pair :
                              AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                                AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                          if
                              c = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                                d = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                            1
                          else 0) /
                        AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent) ∧
                ∃ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) (d :
                  AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                  AppliedModelingLib.SocialChoice.Ranking.rankOf M.center c <
                      AppliedModelingLib.SocialChoice.Ranking.rankOf M.center d ∧
                    (AppliedModelingLib.pmfPairIndicatorExp M.law M.law KR21Monoculture.disagreementEvent
                          fun
                            (pair :
                              AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                                AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                          if
                              d = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                                c = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                            1
                          else 0) /
                        AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent <
                      (AppliedModelingLib.pmfPairIndicatorExp M.law M.law KR21Monoculture.disagreementEvent
                          fun
                            (pair :
                              AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                                AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                          if
                              c = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                                d = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                            1
                          else 0) /
                        AppliedModelingLib.pmfPairProb M.law M.law KR21Monoculture.disagreementEvent
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate

- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.firstChoice
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.rankOf
- AppliedModelingLib.SocialChoice.Ranking.secondChoice
- AppliedModelingLib.pmfPairIndicatorExp
- AppliedModelingLib.pmfPairProb
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.decidableDisagreementEvent
- KR21Monoculture.disagreementEvent

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. assumption: 0 < n
8. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
  (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
     ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
       phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
         ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
           phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
    0 <
      AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
        (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent ∧
      (∀ (c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
         AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center c <
           AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center
             d →
           (AppliedModelingLib.pmfPairIndicatorExp (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
                fun
                  (pair :
                    AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                      AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                if
                    d = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                      c = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                  1
                else 0) /
              AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent ≤
            (AppliedModelingLib.pmfPairIndicatorExp (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
                fun
                  (pair :
                    AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                      AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                if
                    c = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                      d = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                  1
                else 0) /
              AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent) ∧
      ∃ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) (d :
        AppliedModelingLib.SocialChoice.Ranking.Candidate n),
        AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center c <
          AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center
            d ∧
          (AppliedModelingLib.pmfPairIndicatorExp (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
                fun
                  (pair :
                    AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                      AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                if
                    d = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
                      c = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
                  1
                else 0) /
              AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent <
            (AppliedModelingLib.pmfPairIndicatorExp (KR21Monoculture.concreteMallowsSpec center theta).law
                (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
                fun
                  (pair :
                    AppliedModelingLib.SocialChoice.Ranking.Ranking n ×
                      AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                if
```

```
      c = AppliedModelingLib.SocialChoice.Ranking.firstChoice pair.1 ∧
        d = AppliedModelingLib.SocialChoice.Ranking.secondChoice pair.1 then
    1
  else 0) /
AppliedModelingLib.pmfPairProb (KR21Monoculture.concreteMallowsSpec center theta).law
  (KR21Monoculture.concreteMallowsSpec center theta).law KR21Monoculture.disagreementEvent
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationE2_source_phi_literal_conditional_semantic_complete

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The target is the approved N>=3 conditional top-two comparison: every center-ordered pair is weakly ordered and one is strict.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 40

Source locator: cited publication, lines 133-138

Verbatim source input

and that this holds strictly for some i < j. We simplify (E.2) as follows:
Pr[π1 = xi ∩ π2 = xj | π1 6= τ1] > Pr[π1 = xj ∩ π2 = xi | π1 6= τ1]
Pr[π1 = xi ∩ π2 = xj ∩ π1 6= τ1]
Pr[π1 = xj ∩ π2 = xi ∩ π1 6= τ1]
>
Pr[π1 6= τ1]
Pr[π1 6= τ1]
⇐⇒ Pr[π1 = xi ∩ π2 = xj ∩ π1 6= τ1] > Pr[π1 = xj ∩ π2 = xi ∩ π1 6= τ1]
⇐⇒

⇐⇒ Pr[π1 = xi ∩ π2 = xj ∩ τ1 6= xi] > Pr[π1 = xj ∩ π2 = xi ∩ τ1 6= xj]
⇐⇒ Pr[π1 = xi ∩ π2 = xj] Pr[τ1 6= xi] > Pr[π1 = xj ∩ π2 = xi] Pr[τ1 6= xj]

(E.3)

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For Candidate n = Fin(n+2) with n > 0, a center ranking, phi > 1, and theta = phi-1, the source Mallows law has q = phi^(-1) and normalized mass ↪ phi^(-KendallTau(center,pi)) / sum_tau phi^(-KendallTau(center,tau)). For every center-ordered pair c before d, P(first=c,second=d)P(first!=c) >= ↪ P(first=d,second=c)P(first!=d), and this inequality is strict for at least one center-ordered pair. The target uses the corrected positive final product ↪ term; it does not certify the printed negative-sign intermediate expansion.

Archival source anchor: cited publication, lines 2474-2515

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      0 < n →
        have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
        M.q = phi⁻¹ ∧
          (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
             (M.law pi).toReal =
               phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                 ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                   phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
            (∀ (c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
               AppliedModelingLib.SocialChoice.Ranking.rankOf M.center c <
                 AppliedModelingLib.SocialChoice.Ranking.rankOf M.center d →
                 0 ≤
                   M.firstSecondChoiceProb c d * KR21Monoculture.firstChoiceMissProb M.law c -
                     M.firstSecondChoiceProb d c * KR21Monoculture.firstChoiceMissProb M.law d) ∧
              ∃ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) (d :
                AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                AppliedModelingLib.SocialChoice.Ranking.rankOf M.center c <
                  AppliedModelingLib.SocialChoice.Ranking.rankOf M.center d ∧
                 0 <
                   M.firstSecondChoiceProb c d * KR21Monoculture.firstChoiceMissProb M.law c -
                     M.firstSecondChoiceProb d c * KR21Monoculture.firstChoiceMissProb M.law d
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.rankOf
- KR21Monoculture.MallowsSpec
- KR21Monoculture.MallowsSpec.firstSecondChoiceProb
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.firstChoiceMissProb

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. assumption: 0 < n
8. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
   (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),

```
((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
  phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
    ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
      phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
(∀ (c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
   AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center c <
     AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center d →
     0 ≤
       (KR21Monoculture.concreteMallowsSpec center theta).firstSecondChoiceProb c d *
           KR21Monoculture.firstChoiceMissProb (KR21Monoculture.concreteMallowsSpec center theta).law c -
         (KR21Monoculture.concreteMallowsSpec center theta).firstSecondChoiceProb d c *
           KR21Monoculture.firstChoiceMissProb (KR21Monoculture.concreteMallowsSpec center theta).law d) ∧
 ∃ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n) (d :
   AppliedModelingLib.SocialChoice.Ranking.Candidate n),
   AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center c <
     AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center d ∧
     0 <
       (KR21Monoculture.concreteMallowsSpec center theta).firstSecondChoiceProb c d *
           KR21Monoculture.firstChoiceMissProb (KR21Monoculture.concreteMallowsSpec center theta).law c -
         (KR21Monoculture.concreteMallowsSpec center theta).firstSecondChoiceProb d c *
           KR21Monoculture.firstChoiceMissProb (KR21Monoculture.concreteMallowsSpec center theta).law d
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationE3_source_phi_complete

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The target is the approved N>=3 denominator-cleared first-two product comparison with a strict witness and the corrected positive final product term.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 41

Source locator: cited publication, lines 2703-2850

Verbatim source input

Lemma 4. Let {yi }ni=1 , {pi }ni=1 , and {qi }ni=1 be sequences such that
• yi is strictly increasing.
Pn
Pn
•
i=1 pi =
i=1 qi = 1.
• pqii is decreasing.
Pn
Pn
Then, i=1 pi yi < i=1 qi yi .

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For Candidate n = Fin(n+2), real sequences p, q, and a strictly increasing real sequence y, assume sum_i p_i = sum_i q_i = 1, p_i*q_j-p_j*q_i >= 0 for every i<j, and there exist i<j with p_i*q_j-p_j*q_i > 0. Then sum_i p_i*y_i < sum_i q_i*y_i. The explicit strict cross-ratio witness is required; strictness is not inferred from the non-strict pairwise condition alone.

Archival source anchor: cited publication, lines 2703-2713

Expanded PaperInterface specification

```
∀ (n : ℕ) {p q y : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ},
  ∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, p i = 1 →
    ∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, q i = 1 →
      (∀ (i j : AppliedModelingLib.SocialChoice.Ranking.Candidate n), i < j → 0 ≤ p i * q j - p j * q i) →
        (∃ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) (j :
            AppliedModelingLib.SocialChoice.Ranking.Candidate n), i < j ∧ 0 < p i * q j - p j * q i) →
          StrictMono y →
            ∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, p i * y i <
              ∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, q i * y i
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
3. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
4. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
5. assumption: ∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, p i = 1
6. assumption: ∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, q i = 1
7. assumption: ∀ (i j : AppliedModelingLib.SocialChoice.Ranking.Candidate n), i < j → 0 ≤ p i * q j - p j * q i
8. assumption: ∃ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n) (j : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
   i < j ∧ 0 < p i * q j - p j * q i
9. assumption: StrictMono y
10. conclusion: Real.lt† (∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, p i * y i)
    (∑ i : AppliedModelingLib.SocialChoice.Ranking.Candidate n, q i * y i)

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma4_weighted_average_lt_of_cross_ratio

Recorded source-to-Spec screening

Verdict: matches_approved_corrected_target
Reason: The target matches the approved corrected weighted-average result: nonnegative cross-ratios plus an explicit strict witness imply the strict expectation inequality.
Reviewer check: ☐ Matches formalized target
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 42

Source locator: cited publication, lines 133-138

Verbatim source input

Lemma 5. For xi > xj ,
Pr[π1 = xi ∩ π2 = xj] = φ Pr[π1 = xj ∩ π2 = xi].

(F.1)

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      ∀ {c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n},
        AppliedModelingLib.SocialChoice.Ranking.rankOf center c <
            AppliedModelingLib.SocialChoice.Ranking.rankOf center d →
          have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
          M.q = phi⁻¹ ∧
            (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                (M.law pi).toReal =
                  phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                    ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                      phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
              M.firstSecondChoiceProb c d = phi * M.firstSecondChoiceProb d c
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.rankOf
- KR21Monoculture.MallowsSpec
- KR21Monoculture.MallowsSpec.firstSecondChoiceProb
- KR21Monoculture.concreteMallowsSpec

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
8. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
9. assumption: AppliedModelingLib.SocialChoice.Ranking.rankOf center c < AppliedModelingLib.SocialChoice.Ranking.rankOf center d
10. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
  (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
      ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
        phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
          ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
            phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
    (KR21Monoculture.concreteMallowsSpec center theta).firstSecondChoiceProb c d =
      phi * (KR21Monoculture.concreteMallowsSpec center theta).firstSecondChoiceProb d c
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma5_source_phi_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: For center-ordered c before d, the target is the source F.1 identity Pr[first=c,second=d]=phi Pr[first=d,second=c].

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 43

Source locator: cited publication, lines 133-138

Verbatim source input

Lemma 6. For 1 ≤ i ≤ n,
Pr[π1 = xi] =

1 − φ−1
.
φi−1 (1 − φ−n)

(F.2)

[Next verbatim source excerpt]

More formally, we model the n candidates as having intrinsic values x1 , . . . , xn , where any employer would derive utility xi from hiring candidate i. Throughout the paper, we assume without loss of generality that x1 > x2 > · · · > xn . These values, however, are unknown to the employer; instead, they must use some noisy procedure to rank the candidates. We model such a procedure as a randomized mechanism R that takes in the true candidate values and draws a permutation π over those candidates from some distribution. Our main results hold for families of distributions over permutations as defined below:

[Next verbatim source excerpt]

The Mallows Model also appears frequently in the ranking literature [8, 11], and is much more analytically tractable than RUMs. Under the Mallows Model, the likelihood of a permutation is related to its distance from the true ranking π ∗ :
∗
1
(8)
Pr[π] = φ−d(π,π) ,
Z
where Z is a normalizing constant. In this model, φ > 1 is the accuracy parameter: the larger φ is, the more likely the ranking procedure is to output a ranking π that is close to the true ranking r. To instantiate this model, we need a notion of distance d(·, ·) over permutations. For this, we'll use Kendall tau distance, another standard notion in the literature, which is simply the number of pairs of elements in π that are incorrectly ordered [10]. In Appendix D, we verify that the family of distributions Fθ given by the Mallows Model satisfies Definition 1, defining θ = φ − 1 (for consistency, so θ is well-defined on (0, ∞)).

Expanded PaperInterface specification

```
∀ {N : ℕ},
  2 ≤ N →
    ∀ (center : AppliedModelingLib.SocialChoice.Ranking.Ranking (N - 2)) (phi theta : ℝ),
      1 < phi →
        theta = phi - 1 →
          ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate (N - 2)),
            have M : KR21Monoculture.MallowsSpec (N - 2) := KR21Monoculture.concreteMallowsSpec center theta;
            M.q = phi⁻¹ ∧
              (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking (N - 2)),
                 (M.law pi).toReal =
                   phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                     ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking (N - 2),
                       phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                KR21Monoculture.firstChoiceProb M.law c =
                  (1 - phi⁻¹) / (phi ^ ↑(AppliedModelingLib.SocialChoice.Ranking.rankOf M.center c) * (1 - phi⁻¹ ^ N))
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.rankOf
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.firstChoiceProb

Lean-elaborated claim roles

1. parameter: ℕ
2. assumption: 2 ≤ N
3. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking (N - 2)
4. parameter: ℝ
5. parameter: ℝ
6. assumption: 1 < phi
7. assumption: theta = phi - 1
8. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate (N - 2)
9. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
   (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking (N - 2)),
      ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
        phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /

```
    ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking (N - 2),
      phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
KR21Monoculture.firstChoiceProb (KR21Monoculture.concreteMallowsSpec center theta).law c =
  (1 - phi⁻¹) /
    (phi ^
       ↑(AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center
         c) *
     (1 - phi⁻¹ ^ N))
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma6_source_phi_rank_power_complete_spec_proof

Recorded source-to-Spec screening **Verdict:** matches

Reason: The source's total candidate count N is now explicit: Candidate (N - 2) has N elements, Lean rankOf is the source index i - 1, and the formula is (1 - phi^-1) / (phi^(i - 1) * (1 - phi^-N)); 2 <= N is the two-firm model's implicit feasibility condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 44

Source locator: cited publication, lines 133-138

Verbatim source input

Lemma 6. For 1 ≤ i ≤ n,
Pr[π1 = xi] =

1 − φ−1
.
φi−1 (1 − φ−n)

(F.2)

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
        have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
        M.q = phi⁻¹ ∧
          (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
              (M.law pi).toReal =
                phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                  ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                    phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
            KR21Monoculture.firstChoiceProb M.law c =
              (1 - phi⁻¹) / (phi ^ ↑(AppliedModelingLib.SocialChoice.Ranking.rankOf M.center c) * (1 - phi⁻¹ ^ (n + 2)))
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.rankOf
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec
- KR21Monoculture.firstChoiceProb

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
8. conclusion:

```
(KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
  (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
     ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
       phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
         ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
           phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
    KR21Monoculture.firstChoiceProb (KR21Monoculture.concreteMallowsSpec center theta).law c =
      (1 - phi⁻¹) /
        (phi ^
           ↑(AppliedModelingLib.SocialChoice.Ranking.rankOf (KR21Monoculture.concreteMallowsSpec center theta).center
             c) *
         (1 - phi⁻¹ ^ (n + 2)))
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.equationF2_source_phi_closed_form_complete

Recorded source-to-Spec screening

Verdict: matches
Reason: The target is the source F.2 first-choice law (1-phi^-1)/(phi^(rank) (1-phi^-N)) with the center rank indexed from zero.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 45

Source locator: cited publication, lines 133-138

Verbatim source input

Lemma 7. For the Mallows Model, UH (θA , θH) > UHH (θA , θH).

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) (phi theta : ℝ),
  1 < phi →
    theta = phi - 1 →
      ∀ {value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ},
        AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value →
          have M : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center theta;
          M.q = phi⁻¹ ∧
            (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
               (M.law pi).toReal =
                 phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                   ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                     phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
              AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent M.law M.law value <
                AppliedModelingLib.SocialChoice.Ranking.expectedFirstMoverUtility M.law value
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- AppliedModelingLib.SocialChoice.Ranking.expectedFirstMoverUtility
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phi
6. assumption: theta = phi - 1
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
8. assumption: AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
9. conclusion: (KR21Monoculture.concreteMallowsSpec center theta).q = phi⁻¹ ∧
  (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
     ((KR21Monoculture.concreteMallowsSpec center theta).law pi).toReal =
       phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
         ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
           phi⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
      (KR21Monoculture.concreteMallowsSpec center theta).law (KR21Monoculture.concreteMallowsSpec center theta).law
      value <
    AppliedModelingLib.SocialChoice.Ranking.expectedFirstMoverUtility
      (KR21Monoculture.concreteMallowsSpec center theta).law value
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma7_source_phi_complete

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target expands source Lemma 7, U_H>U_HH, as first-mover expected utility strictly exceeding the independent second-mover utility under the same Mallows law.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 46

Source locator: cited publication, lines 133-138

Verbatim source input

Lemma 8. Under the Mallows model, the probability that any two items i < j are correctly ranked increases monotonically with the accuracy parameter φ.

Expanded PaperInterface specification

```
∀ {n : ℕ} (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) {phiMore phiLess : ℝ},
  1 < phiLess →
    phiLess < phiMore →
      ∀ {c d : AppliedModelingLib.SocialChoice.Ranking.Candidate n},
        AppliedModelingLib.SocialChoice.Ranking.rankOf center c <
            AppliedModelingLib.SocialChoice.Ranking.rankOf center d →
          have Mless : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center (phiLess - 1);
          have Mmore : KR21Monoculture.MallowsSpec n := KR21Monoculture.concreteMallowsSpec center (phiMore - 1);
          Mless.q = phiLess⁻¹ ∧
            Mmore.q = phiMore⁻¹ ∧
              (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                  (Mless.law pi).toReal =
                    phiLess⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                      ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                        phiLess⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
                    (Mmore.law pi).toReal =
                      phiMore⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
                        ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
                          phiMore⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
                  (AppliedModelingLib.pmfProb Mless.law fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                      AppliedModelingLib.SocialChoice.Ranking.rankOf pi c <
                        AppliedModelingLib.SocialChoice.Ranking.rankOf pi d) <
                    AppliedModelingLib.pmfProb Mmore.law fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
                      AppliedModelingLib.SocialChoice.Ranking.rankOf pi c <
                        AppliedModelingLib.SocialChoice.Ranking.rankOf pi d
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.kendallTau
- AppliedModelingLib.SocialChoice.Ranking.rankOf
- AppliedModelingLib.pmfProb
- KR21Monoculture.MallowsSpec
- KR21Monoculture.concreteMallowsSpec

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 1 < phiLess
6. assumption: phiLess < phiMore
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
8. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n
9. assumption: AppliedModelingLib.SocialChoice.Ranking.rankOf center c < AppliedModelingLib.SocialChoice.Ranking.rankOf center d
10. conclusion: (KR21Monoculture.concreteMallowsSpec center (phiLess - 1)).q = phiLess⁻¹ ∧
  (KR21Monoculture.concreteMallowsSpec center (phiMore - 1)).q = phiMore⁻¹ ∧
    (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
       ((KR21Monoculture.concreteMallowsSpec center (phiLess - 1)).law pi).toReal =
         phiLess⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
           ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
             phiLess⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
      (∀ (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n),
         ((KR21Monoculture.concreteMallowsSpec center (phiMore - 1)).law pi).toReal =
           phiMore⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center pi /
             ∑ tau : AppliedModelingLib.SocialChoice.Ranking.Ranking n,
               phiMore⁻¹ ^ AppliedModelingLib.SocialChoice.Ranking.kendallTau center tau) ∧
        (AppliedModelingLib.pmfProb (KR21Monoculture.concreteMallowsSpec center (phiLess - 1)).law
           fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
           AppliedModelingLib.SocialChoice.Ranking.rankOf pi c < AppliedModelingLib.SocialChoice.Ranking.rankOf pi d) <
          AppliedModelingLib.pmfProb (KR21Monoculture.concreteMallowsSpec center (phiMore - 1)).law
            fun (pi : AppliedModelingLib.SocialChoice.Ranking.Ranking n) =>
            AppliedModelingLib.SocialChoice.Ranking.rankOf pi c < AppliedModelingLib.SocialChoice.Ranking.rankOf pi d
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.lemma8_source_mallows_phi_pairwise_correct_probability_lt

Recorded source-to-Spec screening **Verdict:** matches
Reason: For every center-ordered pair, the target is source Lemma 8’s strict increase in correct pairwise-ranking probability as phi increases.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 47

Source locator: cited publication, lines 497-500

Verbatim source input

Finally, there exist noise distributions that violate Definition 2 for any candidate distribution D. In
particular, the RUM family defined by the Gumbel distribution is well-known to be equivalent to the PlackettLuce model of ranking, which is generated by
↪ sequentially selecting candidate i with probability

Formalized review target

The archival source above is not asserted equivalent to this formalized target.

For any finite candidate set, common location, innovation scale s > 0, and theta > 0, let arrival_i be iid unit-rate exponential and set epsilon_i = location -
↪ s*log(arrival_i). The arrival law is the iid exponential product law, arrivals are almost surely positive, the innovation law is its coordinatewise
↪ pushforward, and ranking value_i + epsilon_i/theta has exactly the Plackett-Luce ranking law at inverse temperature theta/s. If the visible analytic
↪ premise Var(-log T)=pi^2/6 holds for unit-rate exponential T, then the specialization s=sqrt(6)/pi has coordinate variance one and Plackett-Luce
↪ inverse temperature theta/(sqrt(6)/pi). The archival unit-variance/same-theta formula is not asserted.

Archival source anchor: cited publication, lines 497-548

Expanded PaperInterface specification

```
∀ {n : ℕ} (location : ℝ) {s theta : ℝ},
  0 < s →
    0 < theta →
      ∀ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ),
       (KR21Monoculture.gumbelArrivalLaw n =
          MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
            ProbabilityTheory.expMeasure 1) ∧
         (∀ᵐ (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂KR21Monoculture.gumbelArrivalLaw n,
            ∀ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n), 0 < arrival i) ∧
          (∀ (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
             (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
            KR21Monoculture.commonLocationPositiveScaleGumbelNoise location s arrival i =
              location + -s * Real.log (arrival i)) ∧
           (∀ (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
              (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
             KR21Monoculture.commonLocationPositiveScaleGumbelScores location s theta value arrival i =
               value i + (location + -s * Real.log (arrival i)) / theta) ∧
            (MeasureTheory.Measure.map (KR21Monoculture.commonLocationPositiveScaleGumbelNoise location s)
                 (KR21Monoculture.gumbelArrivalLaw n) =
               MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                 MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -s * Real.log arrival)
                   (ProbabilityTheory.expMeasure 1)) ∧
             KR21Monoculture.commonLocationPositiveScaleGumbelRUMRankingPMF location s theta value =
               KR21Monoculture.plackettLuceRankingPMF (theta / s) value ∧
              (ProbabilityTheory.variance id KR21Monoculture.scaleOneGumbelMeasure = Real.pi ^ 2 / 6 →
                (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location =
                   MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                     MeasureTheory.Measure.map
                       (fun (arrival : ℝ) => location + -(√6 / Real.pi) * Real.log arrival)
                       (ProbabilityTheory.expMeasure 1)) ∧
                 (∀ (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
                    (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                   KR21Monoculture.sourceUnitVarianceGumbelScores theta value epsilon i =
                     value i + epsilon i / theta) ∧
                  (∀ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                     ProbabilityTheory.variance
                         (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) => epsilon i)
                         (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location) =
                       1) ∧
                   KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta value =
                     KR21Monoculture.plackettLuceRankingPMF (theta / (√6 / Real.pi)) value)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- KR21Monoculture.commonLocationPositiveScaleGumbelNoise
- KR21Monoculture.commonLocationPositiveScaleGumbelRUMRankingPMF
- KR21Monoculture.commonLocationPositiveScaleGumbelScores
- KR21Monoculture.gumbelArrivalLaw
- KR21Monoculture.plackettLuceRankingPMF
- KR21Monoculture.scaleOneGumbelMeasure
- KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw
- KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF
- KR21Monoculture.sourceUnitVarianceGumbelScores

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. assumption: 0 < s
6. assumption: 0 < theta
7. parameter: AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ
8. conclusion: (KR21Monoculture.gumbelArrivalLaw n =
   MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
     ProbabilityTheory.expMeasure 1) ∧
  (∀ᵐ (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂KR21Monoculture.gumbelArrivalLaw n,
     ∀ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n), 0 < arrival i) ∧
    (∀ (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
       (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
       KR21Monoculture.commonLocationPositiveScaleGumbelNoise location s arrival i =
         location + -s * Real.log (arrival i)) ∧
      (∀ (arrival : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
         (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
         KR21Monoculture.commonLocationPositiveScaleGumbelScores location s theta value arrival i =
           value i + (location + -s * Real.log (arrival i)) / theta) ∧
        (MeasureTheory.Measure.map (KR21Monoculture.commonLocationPositiveScaleGumbelNoise location s)
           (KR21Monoculture.gumbelArrivalLaw n) =
         MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
           MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -s * Real.log arrival)
             (ProbabilityTheory.expMeasure 1)) ∧
          KR21Monoculture.commonLocationPositiveScaleGumbelRUMRankingPMF location s theta value =
            KR21Monoculture.plackettLuceRankingPMF (theta / s) value ∧
           (ProbabilityTheory.variance id KR21Monoculture.scaleOneGumbelMeasure = Real.pi ^ 2 / 6 →
             (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location =
                MeasureTheory.Measure.pi fun (x : AppliedModelingLib.SocialChoice.Ranking.Candidate n) =>
                  MeasureTheory.Measure.map (fun (arrival : ℝ) => location + -(√6 / Real.pi) * Real.log arrival)
                    (ProbabilityTheory.expMeasure 1)) ∧
               (∀ (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ)
                  (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                  KR21Monoculture.sourceUnitVarianceGumbelScores theta value epsilon i =
                    value i + epsilon i / theta) ∧
                 (∀ (i : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
                    ProbabilityTheory.variance
                       (fun (epsilon : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) => epsilon i)
                       (KR21Monoculture.sourceUnitVarianceGumbelNoiseLaw location) =
                     1) ∧
                   KR21Monoculture.sourceUnitVarianceGumbelRUMRankingPMF location theta value =
                     KR21Monoculture.plackettLuceRankingPMF (theta / (√6 / Real.pi)) value)

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.section31_corrected_gumbel_plackett_luce_target

Recorded source-to-Spec screening **Verdict:** matches_approved_corrected_target

Reason: The iid exponential-arrival pushforward, location-scale scores, theta/s Plackett-Luce law, and conditional unit-variance specialization match the approved corrected Gumbel target.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 48

Source locator: cited publication, lines 176-182

Verbatim source input

for any candidate distribution D, meaning the Plackett-Luce model never meets Definition 2. Thus, under the Plackett-Luce model, monoculture has no effect—the optimal strategy is always to use the best available ranking, regardless of competitors' strategies.

Expanded PaperInterface specification

```
∀ {n : ℕ} (D : MeasureTheory.Measure (KR21Monoculture.ValueProfile n)) [MeasureTheory.IsProbabilityMeasure D]
  (center : AppliedModelingLib.SocialChoice.Ranking.Ranking n) {thetaA thetaH : ℝ},
  0 < thetaA →
    0 < thetaH →
      (∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
         MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D) →
        (∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
           AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value) →
          have M : KR21Monoculture.ValueProfile n → KR21Monoculture.Model n :=
            fun (value : KR21Monoculture.ValueProfile n) =>
              { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
                humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value };
          (∀ (self other : KR21Monoculture.Strategy),
             MeasureTheory.Integrable
               (fun (value : KR21Monoculture.ValueProfile n) => (M value).payoffAgainst self other) D) ∧
            (thetaH ≤ thetaA →
               ∫ (value : KR21Monoculture.ValueProfile n),
                   (M value).payoffAgainst KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm ∂D ≥
                 ∫ (value : KR21Monoculture.ValueProfile n),
                   (M value).payoffAgainst KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm ∂D ∧
               ∫ (value : KR21Monoculture.ValueProfile n),
                   (M value).payoffAgainst KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ∂D ≥
                 ∫ (value : KR21Monoculture.ValueProfile n),
                   (M value).payoffAgainst KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human ∂D) ∧
              (thetaA ≤ thetaH →
                 ∫ (value : KR21Monoculture.ValueProfile n),
                     (M value).payoffAgainst KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm ∂D ≥
                   ∫ (value : KR21Monoculture.ValueProfile n),
                     (M value).payoffAgainst KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm ∂D ∧
                 ∫ (value : KR21Monoculture.ValueProfile n),
                     (M value).payoffAgainst KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human ∂D ≥
                   ∫ (value : KR21Monoculture.ValueProfile n),
                     (M value).payoffAgainst KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ∂D)
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy
- KR21Monoculture.Model
- KR21Monoculture.Model.payoffAgainst
- KR21Monoculture.Strategy
- KR21Monoculture.ValueProfile
- KR21Monoculture.plackettLuceRankingPMF

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: MeasureTheory.Measure (KR21Monoculture.ValueProfile n)
3. assumption: MeasureTheory.IsProbabilityMeasure D
4. parameter: AppliedModelingLib.SocialChoice.Ranking.Ranking n
5. parameter: ℝ
6. parameter: ℝ
7. assumption: 0 < thetaA
8. assumption: 0 < thetaH
9. assumption: ∀ (c : AppliedModelingLib.SocialChoice.Ranking.Candidate n),
   MeasureTheory.Integrable (fun (value : KR21Monoculture.ValueProfile n) => value c) D
10. assumption: ∀ᵐ (value : AppliedModelingLib.SocialChoice.Ranking.Candidate n → ℝ) ∂D,
   AppliedModelingLib.SocialChoice.Ranking.StrictlyOrderedBy center value
11. conclusion: (∀ (self other : KR21Monoculture.Strategy),
     MeasureTheory.Integrable
       (fun (value : KR21Monoculture.ValueProfile n) =>
         ((fun (value : KR21Monoculture.ValueProfile n) =>
             { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
               humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
           value).payoffAgainst
         self other)
       D) ∧
   (thetaH ≤ thetaA →
```

```
  ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm ∂D ≥
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm ∂D ∧
   ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ∂D ≥
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human ∂D) ∧
(thetaA ≤ thetaH →
  ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.human KR21Monoculture.Strategy.algorithm ∂D ≥
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.algorithm ∂D ∧
   ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.human KR21Monoculture.Strategy.human ∂D ≥
    ∫ (value : KR21Monoculture.ValueProfile n),
      ((fun (value : KR21Monoculture.ValueProfile n) =>
            { algorithmRanking := KR21Monoculture.plackettLuceRankingPMF thetaA value,
              humanRanking := KR21Monoculture.plackettLuceRankingPMF thetaH value, value := value })
          value).payoffAgainst
        KR21Monoculture.Strategy.algorithm KR21Monoculture.Strategy.human ∂D)
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.section31_plackettLuce_outer_best_available_weakly_dominates

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target makes the source best-available-ranking conclusion explicit as weak payoff dominance of the larger Plackett-Luce accuracy against either opponent strategy.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 49

Source locator: cited publication, lines 972-974

Verbatim source input

Note that this distribution does not satisfy Definition 1 because it is neither differentiable nor supported on (−∞, ∞); however, we can provide a “smooth” approximation to this distribution by expressing it as the sum of arbitrarily tightly concentrated Gaussians with the same results.

[Next verbatim source excerpt]

Again, while this noise model doesn’t satisfy Definition 1, we can approximate it arbitrarily closely with the sum of tightly concentrated Gaussians. Let the θA = 1.1θ and θH = 0.9θ.

Expanded PaperInterface specification

```
(KR21Monoculture.appendixB1Value 0 = 7 / 4 ∧
   KR21Monoculture.appendixB1Value 1 = 1 / 2 ∧ KR21Monoculture.appendixB1Value 2 = 0) ∧
  (KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.plusOne = 1 ∧
     KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.zero = 0 ∧
      KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.minusOne = -1) ∧
   ((KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.plusOne).toReal = 1 / 20 ∧
      (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.zero).toReal = 9 / 10 ∧
       (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.minusOne).toReal = 1 / 20) ∧
     KR21Monoculture.appendixB1GaussianLatentMeasure =
        ((KR21Monoculture.appendixB1NoisePMF.toMeasure.prod KR21Monoculture.appendixB1NoisePMF.toMeasure).prod
            KR21Monoculture.appendixB1NoisePMF.toMeasure).prod
          ((ProbabilityTheory.gaussianReal 0 1).prod
            ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))) ∧
       (∀ (s theta : ℝ),
          0 < s →
            0 < theta →
              (KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist theta =
                KR21Monoculture.rumRankingPMFOfMeasure KR21Monoculture.appendixB1GaussianLatentMeasure
                  (fun (omega : KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple) =>
                    AppliedModelingLib.SocialChoice.Ranking.rankByScore
                      fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                      KR21Monoculture.appendixB1Value c +
                        (KR21Monoculture.appendixB1NoiseValue
                            (KR21Monoculture.appendixB1NoiseTripleFunction omega.1 c) +
                          s * KR21Monoculture.appendixBGaussianTripleFunction omega.2 c) /
                        theta)
                  (KR21Monoculture.appendixB1SourceGaussianMixtureRank_measurable s theta)) ∧
         (KR21Monoculture.appendixB2Value 0 = 3 ∧
            KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
          (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
             KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
              KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
               KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
            ((KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
               (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
                (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
                 (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal =
                   1 / 20) ∧
              KR21Monoculture.appendixB2GaussianLatentMeasure =
                 ((KR21Monoculture.appendixB2NoisePMF.toMeasure.prod
                        KR21Monoculture.appendixB2NoisePMF.toMeasure).prod
                    KR21Monoculture.appendixB2NoisePMF.toMeasure).prod
                  ((ProbabilityTheory.gaussianReal 0 1).prod
                    ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))) ∧
                 (∀ (s theta : ℝ),
                    0 < s →
                      0 < theta →
                        (KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist theta =
                          KR21Monoculture.rumRankingPMFOfMeasure KR21Monoculture.appendixB2GaussianLatentMeasure
                            (fun
                                (omega :
                                  KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple) =>
                              AppliedModelingLib.SocialChoice.Ranking.rankByScore
                                fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                                KR21Monoculture.appendixB2Value c +
                                  (KR21Monoculture.appendixB2NoiseValue
                                      (KR21Monoculture.appendixB2NoiseTripleFunction omega.1 c) +
                                    s * KR21Monoculture.appendixBGaussianTripleFunction omega.2 c) /
                                  theta)
                            (KR21Monoculture.appendixB2SourceGaussianMixtureRank_measurable s theta)) ∧
                   (∃ (delta : ℝ),
                      0 < delta ∧
                        ∀ (s : ℝ),
                          0 < s →
                            s < delta →
                              AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                                  ((KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist 1)
                                  ((KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist 1)
                                  KR21Monoculture.appendixB1Value -
                                AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
                                  ((KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist 1)
                                  KR21Monoculture.appendixB1Value <
                                0) ∧
                     ∃ (delta : ℝ),
                      0 < delta ∧
                        ∀ (s : ℝ),
                          0 < s →
                            s < delta →
```

```
0 <
  AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
      ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (9 / 10))
      ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (11 / 10))
      KR21Monoculture.appendixB2Value -
    AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
      ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (9 / 10))
      ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (9 / 10))
      KR21Monoculture.appendixB2Value
```

Retained governing declarations

- AppliedModelingLib.SocialChoice.Ranking.Candidate
- AppliedModelingLib.SocialChoice.Ranking.Ranking
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
- AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
- AppliedModelingLib.SocialChoice.Ranking.instMeasurableSpaceRanking
- AppliedModelingLib.SocialChoice.Ranking.rankByScore
- KR21Monoculture.AccuracyFamily
- KR21Monoculture.AppendixB1NoiseAtom
- KR21Monoculture.AppendixB1NoiseTriple
- KR21Monoculture.AppendixB2NoiseAtom
- KR21Monoculture.AppendixB2NoiseTriple
- KR21Monoculture.AppendixBGaussianTriple
- KR21Monoculture.PaperInterface.instMeasurableSpaceAppendixB1NoiseAtom_1
- KR21Monoculture.PaperInterface.instMeasurableSpaceAppendixB2NoiseAtom_1
- KR21Monoculture.appendixB1GaussianLatentMeasure
- KR21Monoculture.appendixB1NoisePMF
- KR21Monoculture.appendixB1NoiseTripleFunction
- KR21Monoculture.appendixB1NoiseValue
- KR21Monoculture.appendixB1SourceGaussianMixtureFamily
- KR21Monoculture.appendixB1SourceGaussianMixtureRank
- KR21Monoculture.appendixB1Value
- KR21Monoculture.appendixB2GaussianLatentMeasure
- KR21Monoculture.appendixB2NoisePMF
- KR21Monoculture.appendixB2NoiseTripleFunction
- KR21Monoculture.appendixB2NoiseValue
- KR21Monoculture.appendixB2SourceGaussianMixtureFamily
- KR21Monoculture.appendixB2SourceGaussianMixtureRank
- KR21Monoculture.appendixB2Value
- KR21Monoculture.appendixBGaussianTripleFunction
- KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom
- KR21Monoculture.instMeasurableSpaceAppendixB1NoiseAtom_2
- KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom
- KR21Monoculture.instMeasurableSpaceAppendixB2NoiseAtom_1
- KR21Monoculture.rumRankingPMFOfMeasure

Lean-elaborated claim roles

```
1. conclusion: (KR21Monoculture.appendixB1Value 0 = 7 / 4 ∧
    KR21Monoculture.appendixB1Value 1 = 1 / 2 ∧ KR21Monoculture.appendixB1Value 2 = 0) ∧
  (KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.plusOne = 1 ∧
     KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.zero = 0 ∧
       KR21Monoculture.appendixB1NoiseValue KR21Monoculture.AppendixB1NoiseAtom.minusOne = -1) ∧
   ((KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.plusOne).toReal = 1 / 20 ∧
      (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.zero).toReal = 9 / 10 ∧
        (KR21Monoculture.appendixB1NoisePMF KR21Monoculture.AppendixB1NoiseAtom.minusOne).toReal = 1 / 20) ∧
     KR21Monoculture.appendixB1GaussianLatentMeasure =
       ((KR21Monoculture.appendixB1NoisePMF.toMeasure.prod KR21Monoculture.appendixB1NoisePMF.toMeasure).prod
            KR21Monoculture.appendixB1NoisePMF.toMeasure).prod
         ((ProbabilityTheory.gaussianReal 0 1).prod
           ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))) ∧
      (∀ (s theta : ℝ),
         0 < s →
           0 < theta →
             (KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist theta =
               KR21Monoculture.rumRankingPMFOfMeasure KR21Monoculture.appendixB1GaussianLatentMeasure
                 (fun (omega : KR21Monoculture.AppendixB1NoiseTriple × KR21Monoculture.AppendixBGaussianTriple) =>
                   AppliedModelingLib.SocialChoice.Ranking.rankByScore
                     fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                     KR21Monoculture.appendixB1Value c +
                       (KR21Monoculture.appendixB1NoiseValue
                            (KR21Monoculture.appendixB1NoiseTripleFunction omega.1 c) +
                          s * KR21Monoculture.appendixBGaussianTripleFunction omega.2 c) /
                         theta)
                 (KR21Monoculture.appendixB1SourceGaussianMixtureRank_measurable s theta)) ∧
     (KR21Monoculture.appendixB2Value 0 = 3 ∧
        KR21Monoculture.appendixB2Value 1 = 2 ∧ KR21Monoculture.appendixB2Value 2 = 0) ∧
      (KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusOne = 1 ∧
         KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusOne = -1 ∧
           KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.plusTen = 10 ∧
             KR21Monoculture.appendixB2NoiseValue KR21Monoculture.AppendixB2NoiseAtom.minusTen = -10) ∧
       ((KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusOne).toReal = 9 / 20 ∧
          (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusOne).toReal = 9 / 20 ∧
            (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.plusTen).toReal = 1 / 20 ∧
              (KR21Monoculture.appendixB2NoisePMF KR21Monoculture.AppendixB2NoiseAtom.minusTen).toReal =
                1 / 20) ∧
         KR21Monoculture.appendixB2GaussianLatentMeasure =
           ((KR21Monoculture.appendixB2NoisePMF.toMeasure.prod
                KR21Monoculture.appendixB2NoisePMF.toMeasure).prod
             KR21Monoculture.appendixB2NoisePMF.toMeasure).prod
           ((ProbabilityTheory.gaussianReal 0 1).prod
             ((ProbabilityTheory.gaussianReal 0 1).prod (ProbabilityTheory.gaussianReal 0 1))) ∧
         (∀ (s theta : ℝ),
            0 < s →
              0 < theta →
                (KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist theta =
                  KR21Monoculture.rumRankingPMFOfMeasure KR21Monoculture.appendixB2GaussianLatentMeasure
                    (fun
                       (omega :
                         KR21Monoculture.AppendixB2NoiseTriple × KR21Monoculture.AppendixBGaussianTriple) =>
                      AppliedModelingLib.SocialChoice.Ranking.rankByScore
                        fun (c : AppliedModelingLib.SocialChoice.Ranking.Candidate 1) =>
                        KR21Monoculture.appendixB2Value c +
                          (KR21Monoculture.appendixB2NoiseValue
                               (KR21Monoculture.appendixB2NoiseTripleFunction omega.1 c) +
                             s * KR21Monoculture.appendixBGaussianTripleFunction omega.2 c) /
                            theta)
                    (KR21Monoculture.appendixB2SourceGaussianMixtureRank_measurable s theta)) ∧
           (∃ (delta : ℝ),
              0 < delta ∧
                ∀ (s : ℝ),
                  0 < s →
                    s < delta →
                      AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                           ((KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist 1)
                           ((KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist 1)
                           KR21Monoculture.appendixB1Value -
                         AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverShared
                           ((KR21Monoculture.appendixB1SourceGaussianMixtureFamily s).dist 1)
                           KR21Monoculture.appendixB1Value <
                         0) ∧
             ∃ (delta : ℝ),
               0 < delta ∧
                 ∀ (s : ℝ),
                   0 < s →
                     s < delta →
                       0 <
                         AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                           ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (9 / 10))
                           ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (11 / 10))
                           KR21Monoculture.appendixB2Value -
                          AppliedModelingLib.SocialChoice.Ranking.expectedSecondMoverIndependent
                           ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (9 / 10))
                           ((KR21Monoculture.appendixB2SourceGaussianMixtureFamily s).dist (9 / 10))
                           KR21Monoculture.appendixB2Value
```

Lean proof endpoint (built separately)

KR21Monoculture.PaperInterface.appendixB_smoothing_source_core

Recorded source-to-Spec screening **Verdict:** matches

Reason: The target gives iid Gaussian-mixture smoothings of both displayed discrete noises and positive radii

preserving their respective strict counterexample gaps.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation